IS6400 Business Data Analytics

Dr. LIU Junming

City University of Hong Kong

Lecture 10

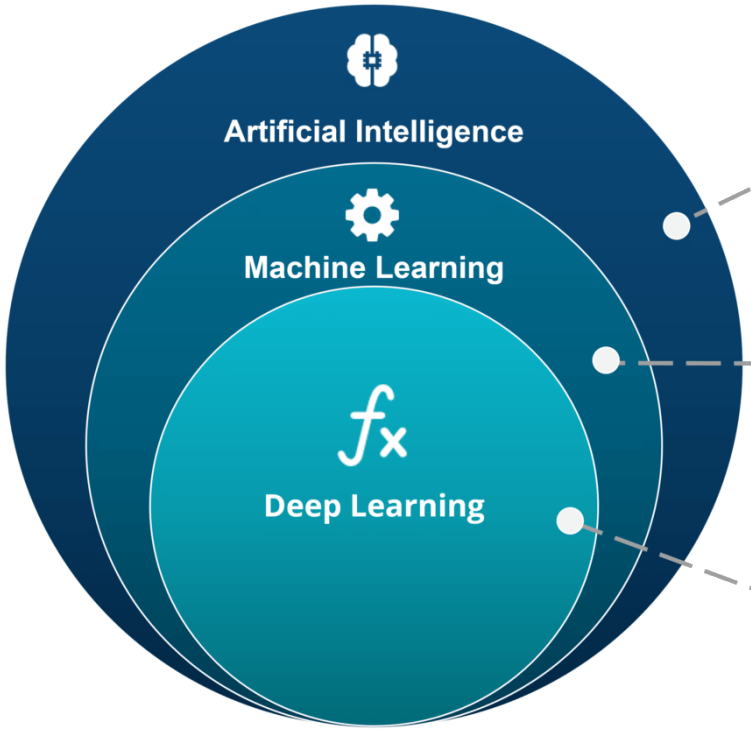
- Intro-Deep Learning
  - ANN
  - RNN, LSTM
  - NLP
- Image Mining

Reading Material:

-

What is Deep Learning?

Deep learning (also known as **deep structured learning**) is part of a broader family of machine learning methods based on artificial neural networks.

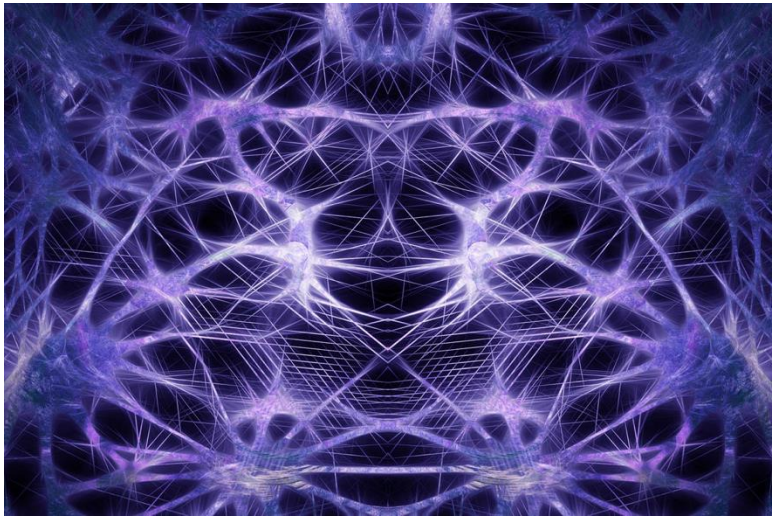


ARTIFICIAL INTELLIGENCE
A technique which enables machines to mimic human behaviour

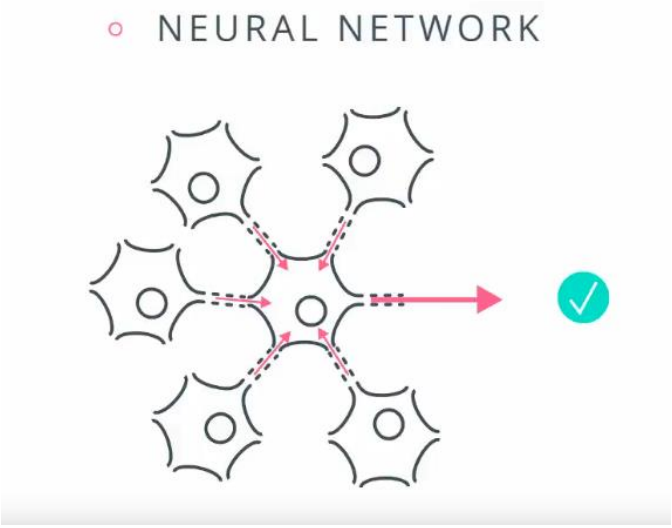
MACHINE LEARNING
Subset of AI technique which use statistical methods to enable machines to improve with experience

DEEP LEARNING
Subset of ML which make the computation of multi-layer neural network feasible

Artificial Neural Networks

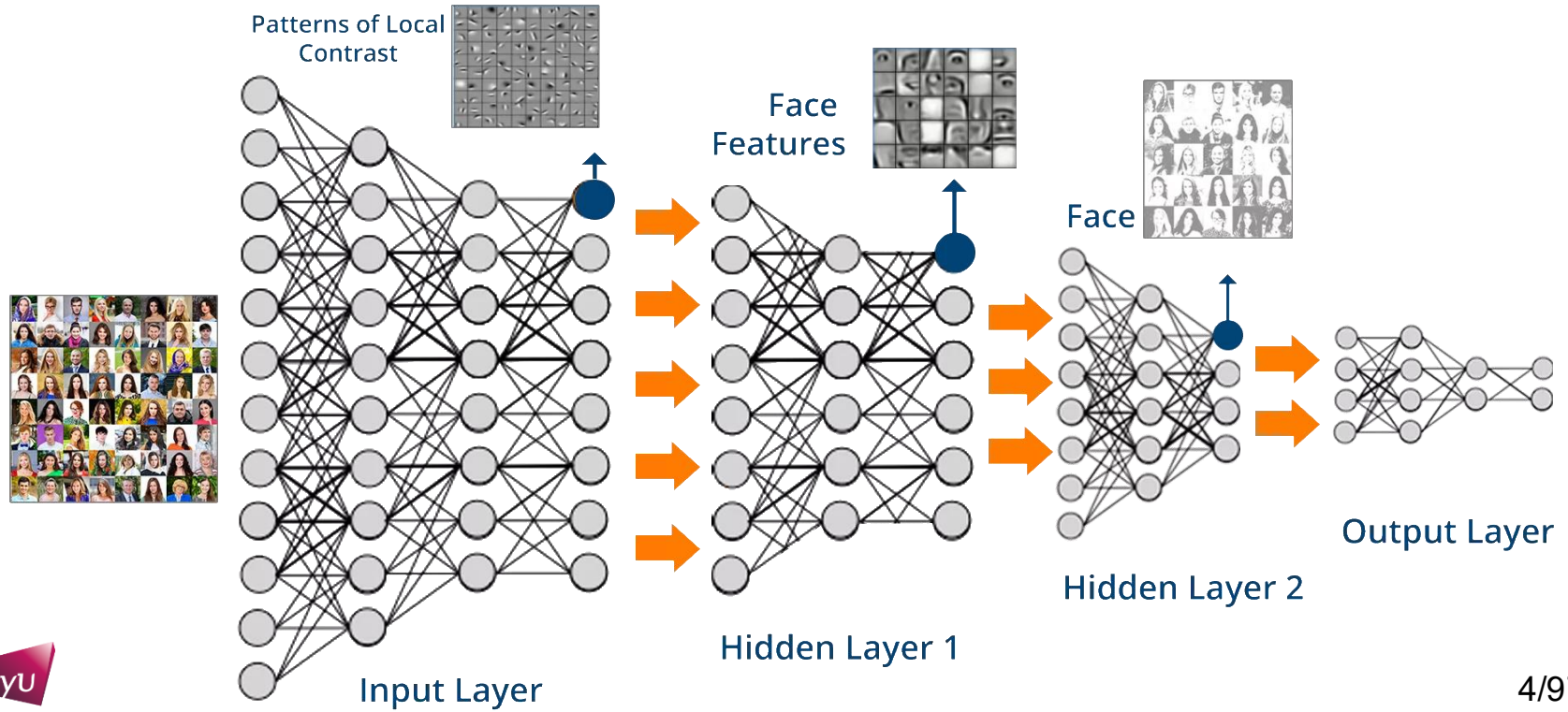






What is Deep Learning?

Deep learning use neural networks to simulate human like decision making

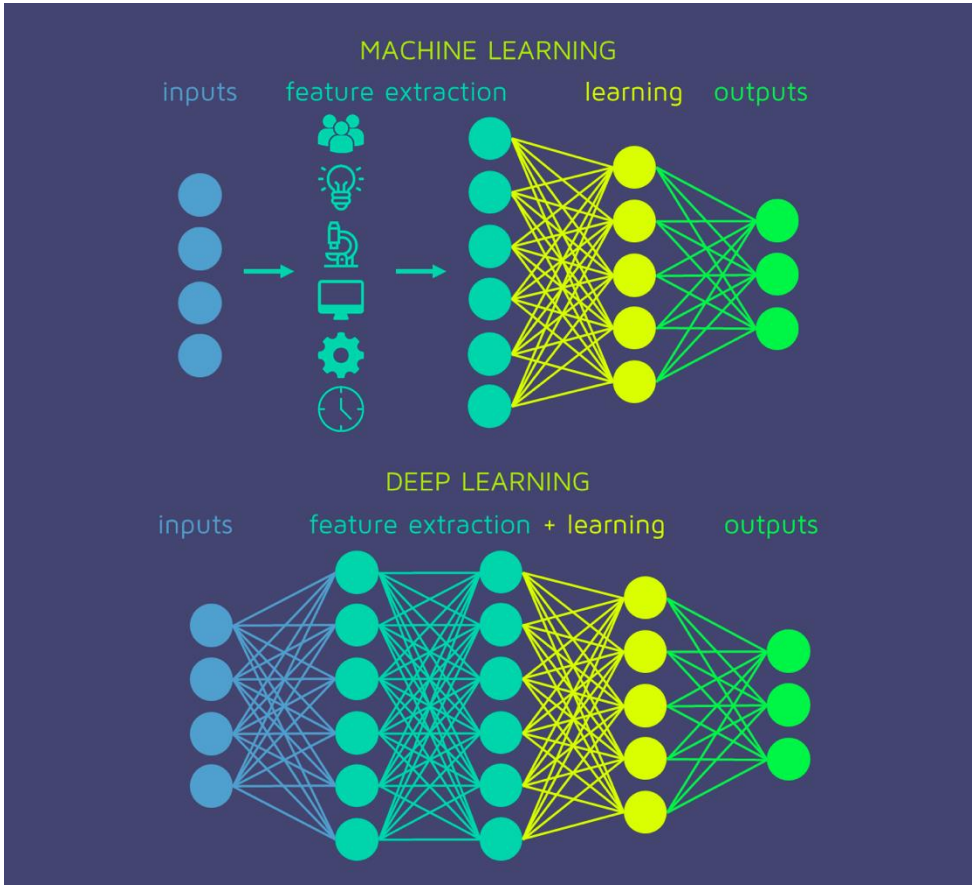


Why Deep Learning?

Limitations of fundamental machine learning algorithms

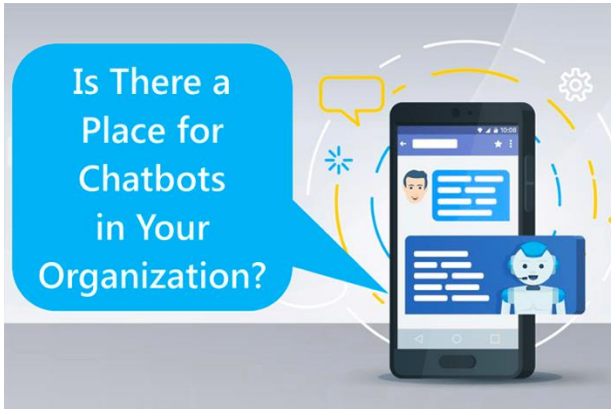
- Curse of Dimension!
- Large dataset
- Challenges of feature extraction
- Not able to solve complex problems such as image recognition, speech recognition, NLP,etc

Why Deep Learning?

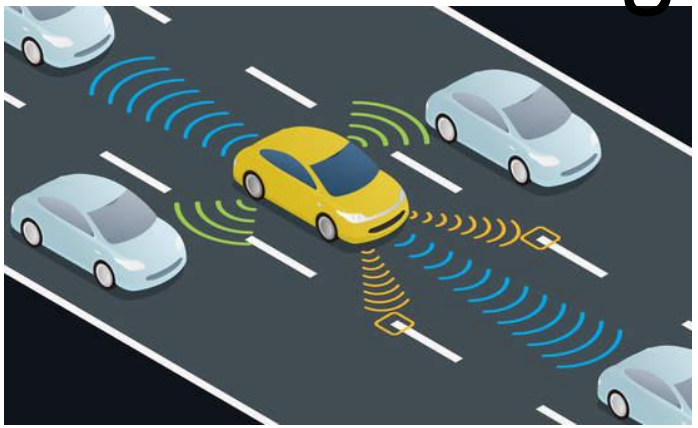


Why Deep Learning?

Chatbot



Self-driving



Medicare



Image recognition



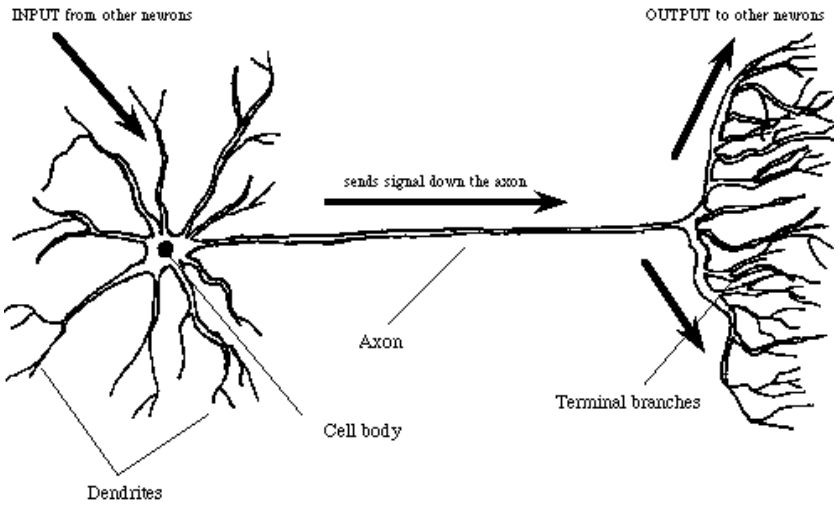
CityU
香港城市大學
City University of Hong Kong
專業 創新 與時俱進
Professional·Creative
For The World

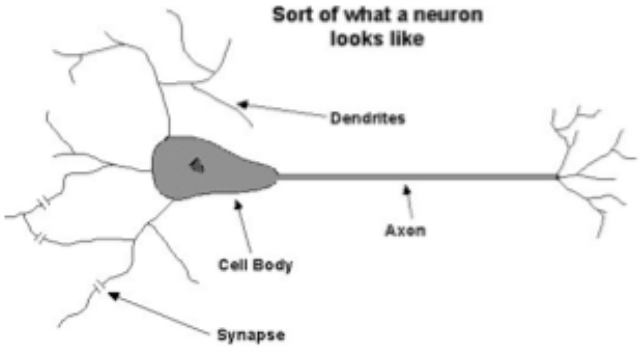
Why Deep Learning and NN?

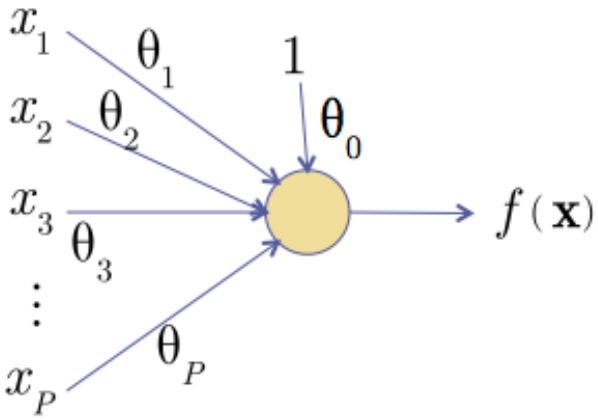
Huge market demand



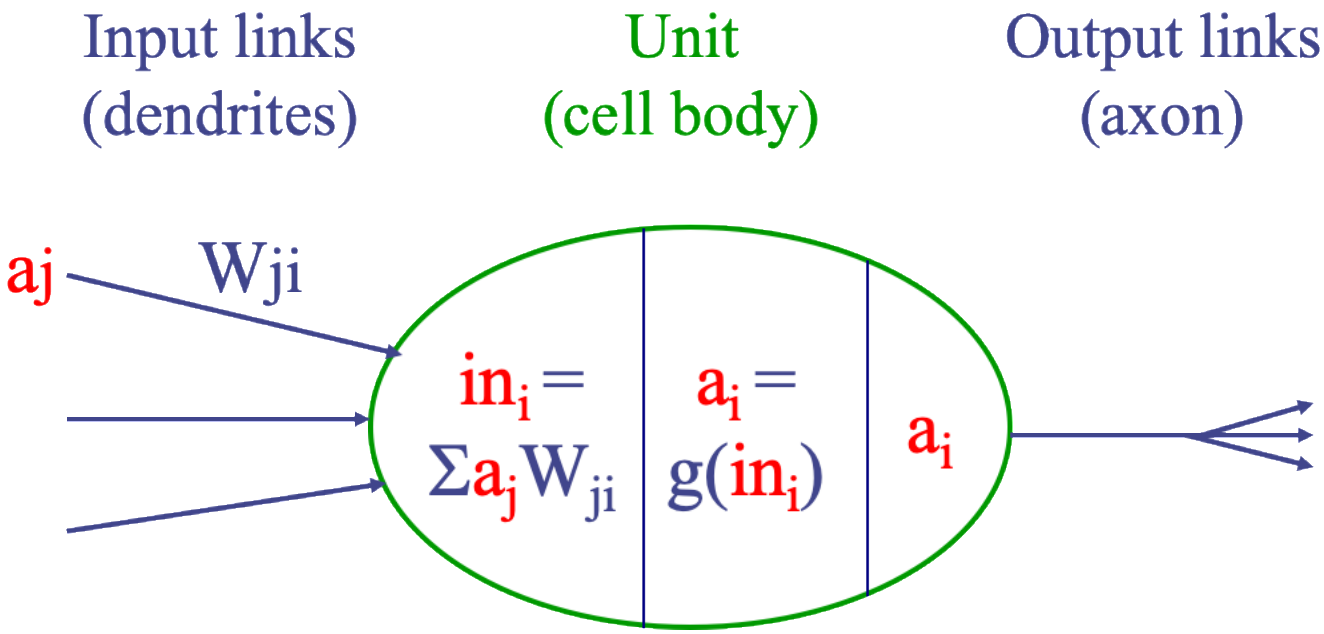
Biological Background



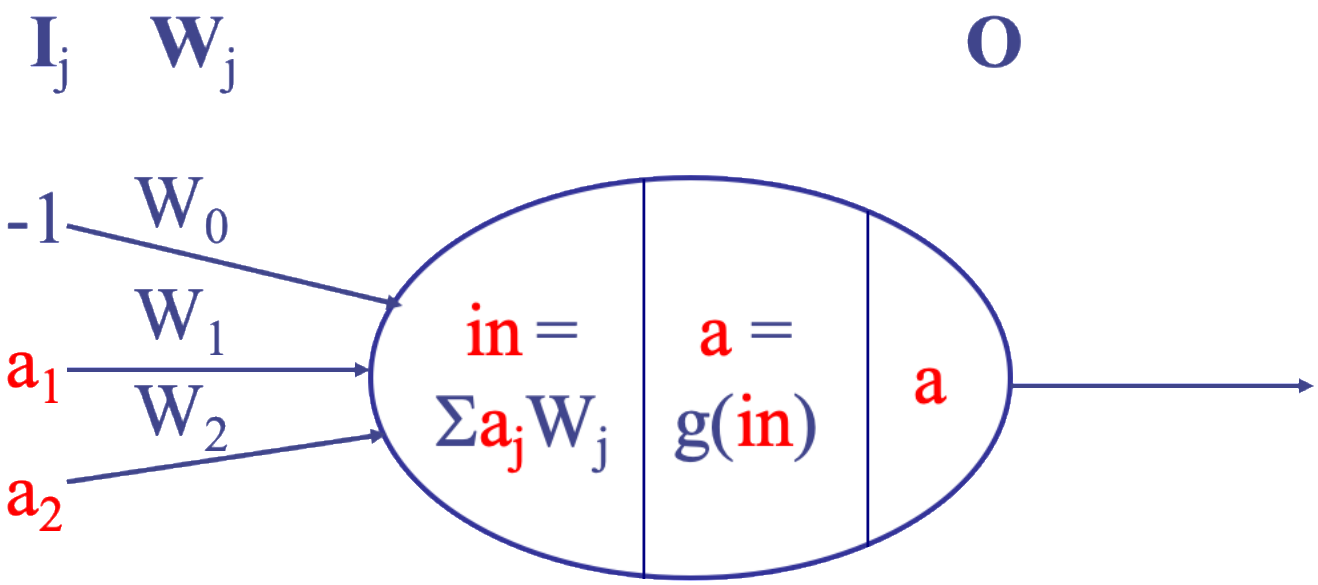




Artificial Neuron



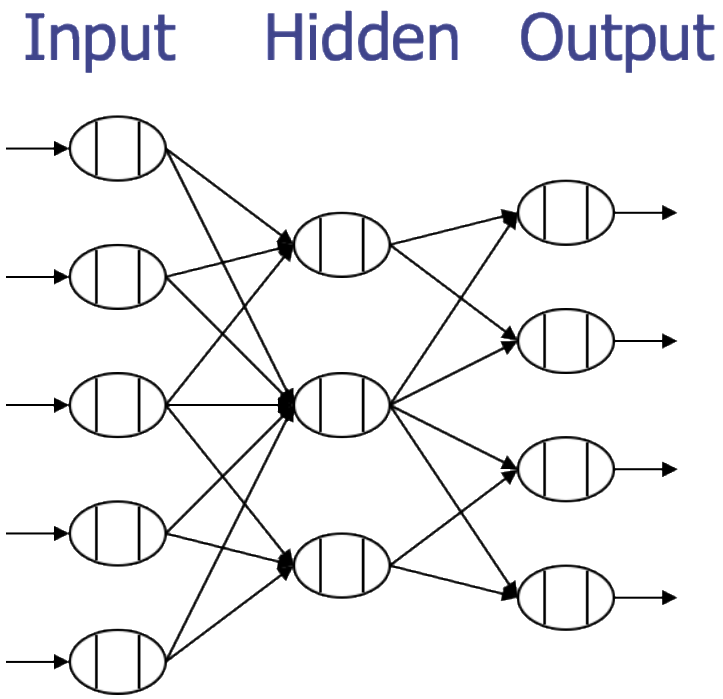
Class I: Perceptron



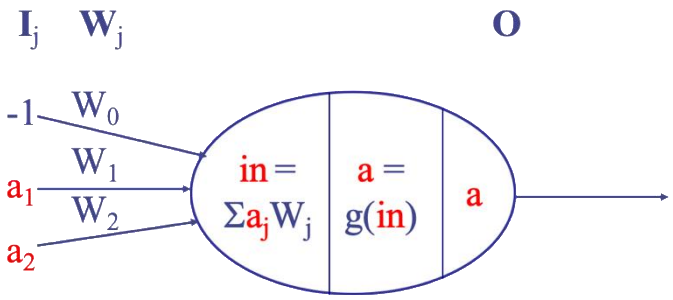
$$a = g(-W_0 + W_1 a_1 + W_2 a_2) \qquad g(in) = \begin{cases} 0, & in<0 \\ 1, & in>0 \end{cases}$$

Class II: Multi-layer Feed-forward Networks

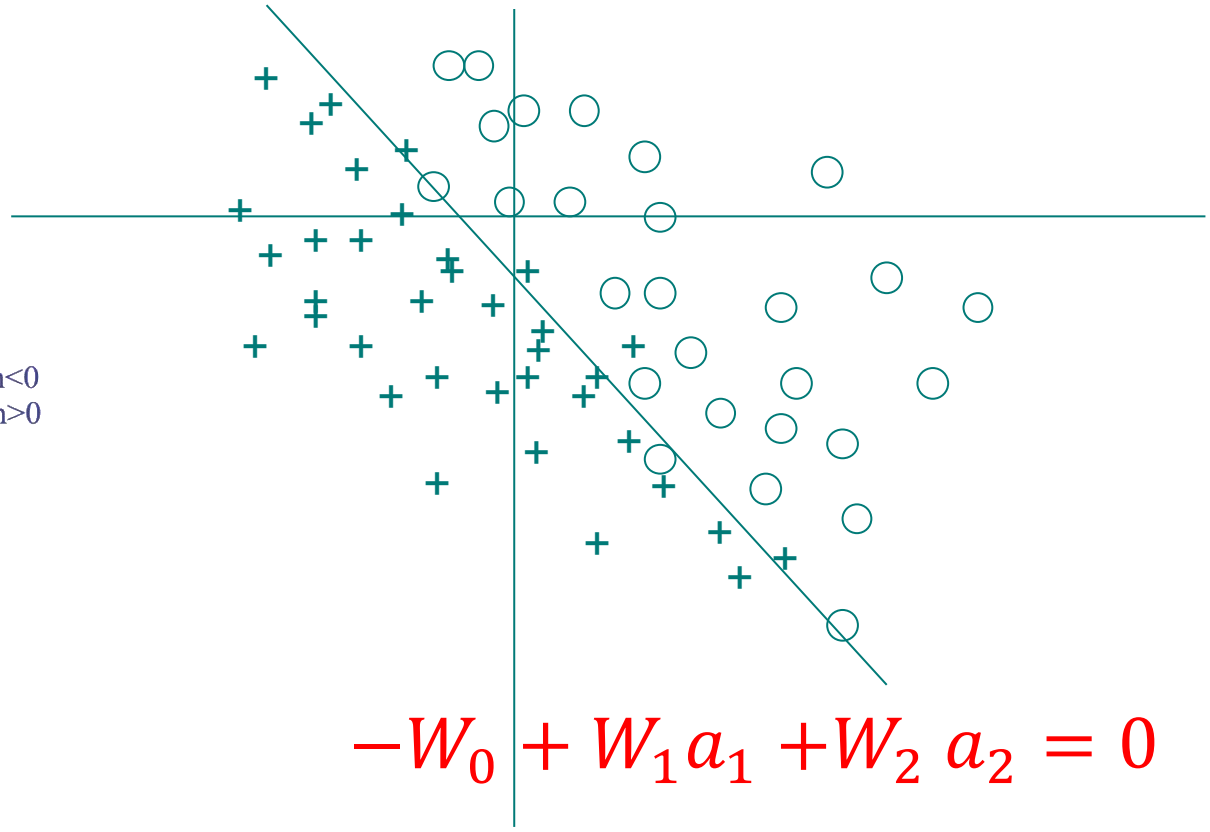
- Multiple layers:
  - hidden layer(s)
- Feed-forward:
  - Output links only connected to input links in the next layer
- Complex non-linear functions can be represented



Use a trained model



$a = g(-W_0 + W_1 a_1 + W_2 a_2)$ $\quad g(in) = \begin{cases} 0, & in<0 \\ 1, & in>0 \end{cases}$



$$-W_0 + W_1 a_1 + W_2 a_2 = 0$$

Perceptron

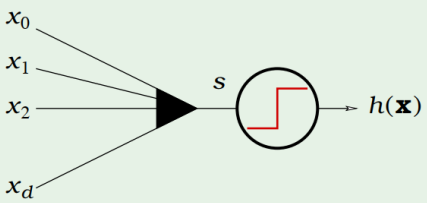
Linear models

$$s = \sum_{i=0}^{d} w_i x_i$$

Perceptron

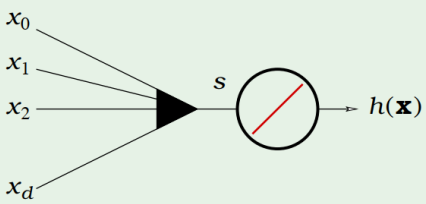
linear classification

$$h(\mathbf{x}) = \text{sign}(s)$$



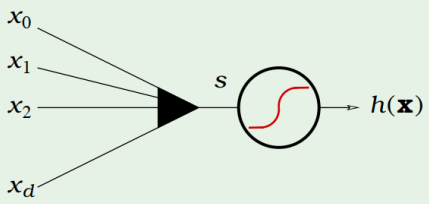
linear regression

$$h(\mathbf{x}) = s$$



logistic regression

$$h(\mathbf{x}) = g(s)$$



$$g(s) = 1/(1+\exp(-s))$$

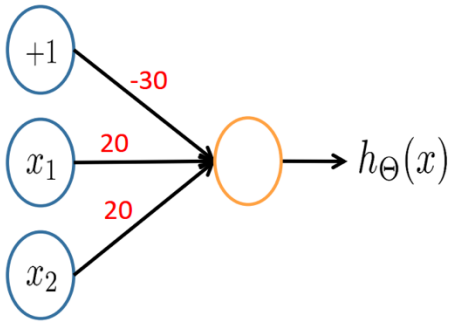
Perceptron

Logical functions

Simple example: AND

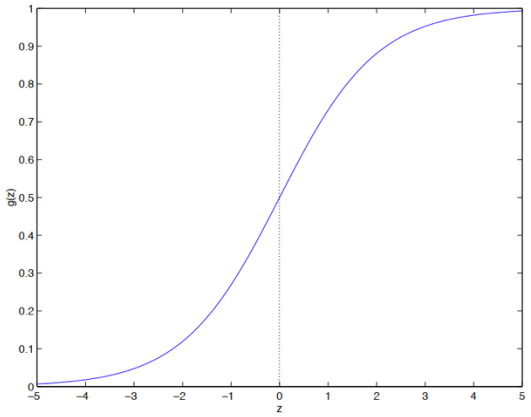
$x_1, x_2 \in \{0, 1\}$

$y = x_1 \text{ AND } x_2$



$h_\theta(x) = g(-30 + 20x_1 + 20x_2)$

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$



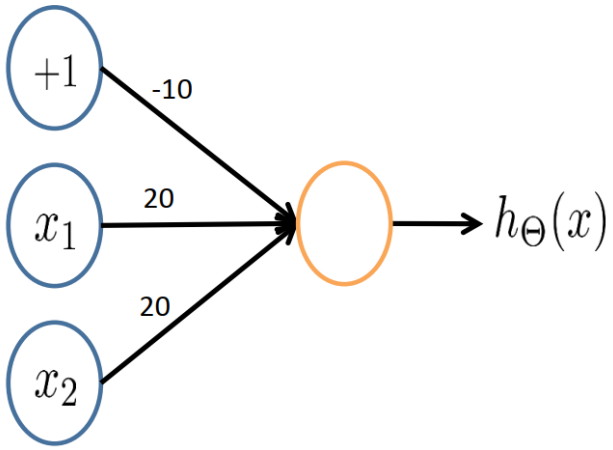
x_1	x_2	$h_\Theta(x)$
0	0	g(-30) ~0
0	1	g(-10) ~0
1	0	g(-10) ~0
1	1	g(10) ~1

AND function:

x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

Perceptron

Example: OR function



g(-10 + 20x_1 + 20x_2)

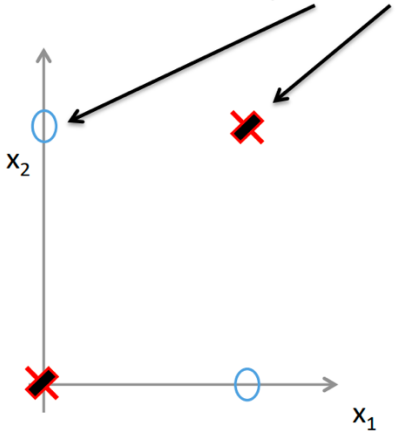
x_1	x_2	$h_\Theta(x)$
0	0	g(-10) ~0
0	1	g(10) ~1
1	0	g(10) ~1
1	1	g(30) ~1

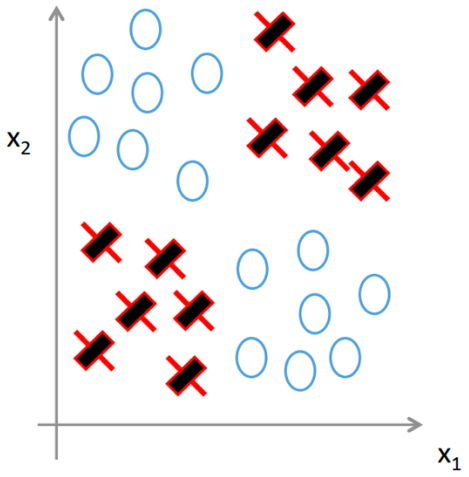
$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

A complex case

Non-linear classification example: XOR/XNOR

x_1, x_2 are binary (0 or 1).





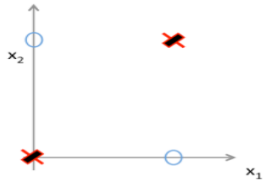
$$y = x_1 \text{ XOR } x_2$$
$$x_1 \text{ XNOR } x_2$$
$$\text{NOT } (x_1 \text{ XOR } x_2)$$

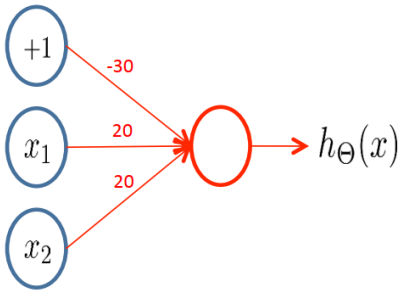
XOR function:

x1	x2	y
0	0	1
0	1	0
1	0	0
1	1	1

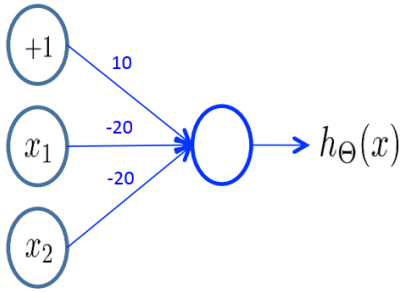
A complex case

Putting it together: x_1 XNOR x_2

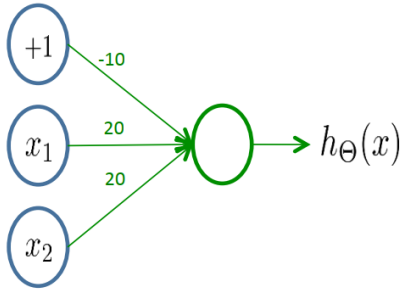




x_1 AND x_2



(NOT x_1) AND (NOT x_2)

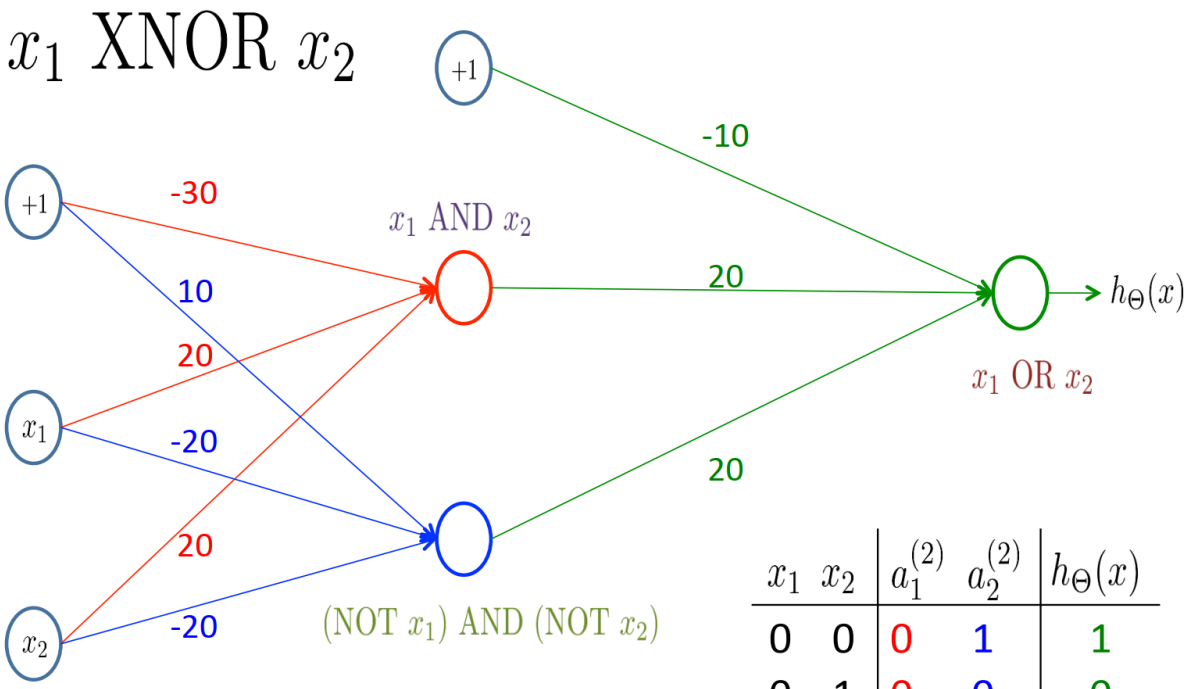


x_1 OR x_2



x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_\Theta(x)$
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

A complex case



x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_\Theta(x)$
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

A more complex case – multi output

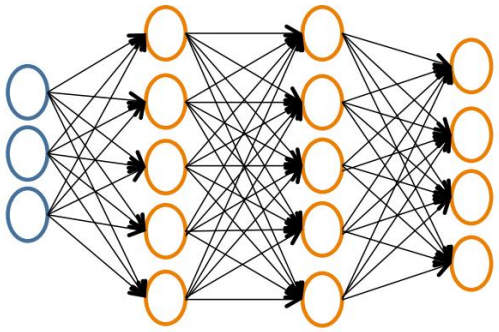
Multiple output units: One-vs-all.


Pedestrian


Car


Motorcycle


Truck



$$h_\Theta(x) \in \mathbb{R}^4$$

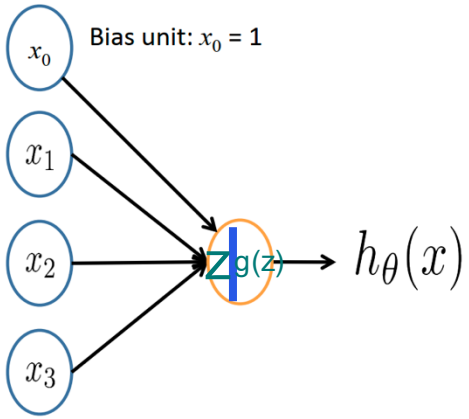
Want $h_\Theta(x) \approx \begin{bmatrix}1\\0\\0\\0\end{bmatrix}$, $h_\Theta(x) \approx \begin{bmatrix}0\\1\\0\\0\end{bmatrix}$, $h_\Theta(x) \approx \begin{bmatrix}0\\0\\1\\0\end{bmatrix}$, etc.

when pedestrian when car when motorcycle

Summary of implementation

- Design a structure
  - Number of input (input nodes)
  - Number of hidden layer
  - Number of nodes per hidden layer
  - Number of output nodes (targets)
- Decide the node activation function
  - From node input to node output
- Decide the parameters
  - Coefficients from previous layer output to next layer input
  - Parameters of activation function (if any)

ANN Model Representation



$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

Sigmoid (logistic) activation function.

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z)).$$

Model Representation

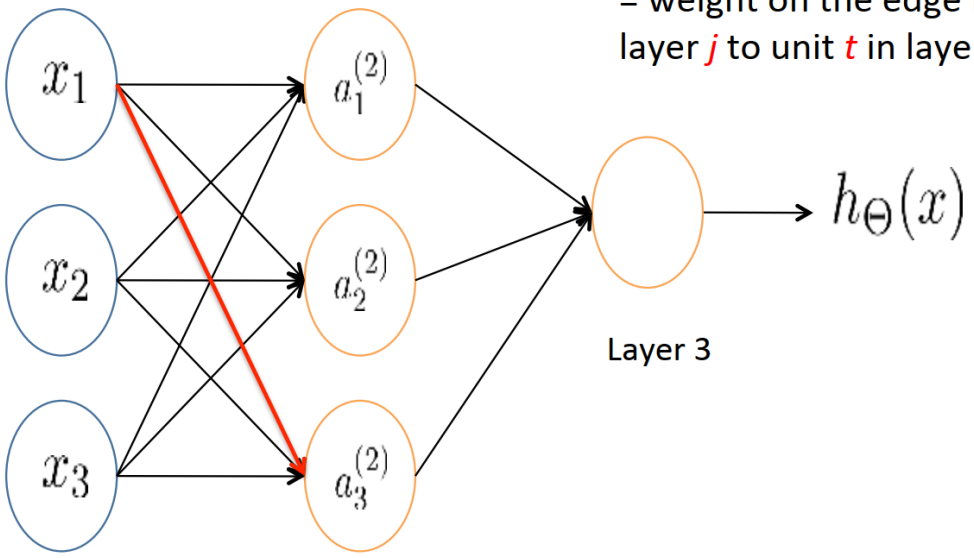
Neural Network

$a_i^{(j)}$ = "activation" of unit i in layer j

$\Theta^{(j)}$ = matrix of weights controlling function mapping from layer j to layer j+1

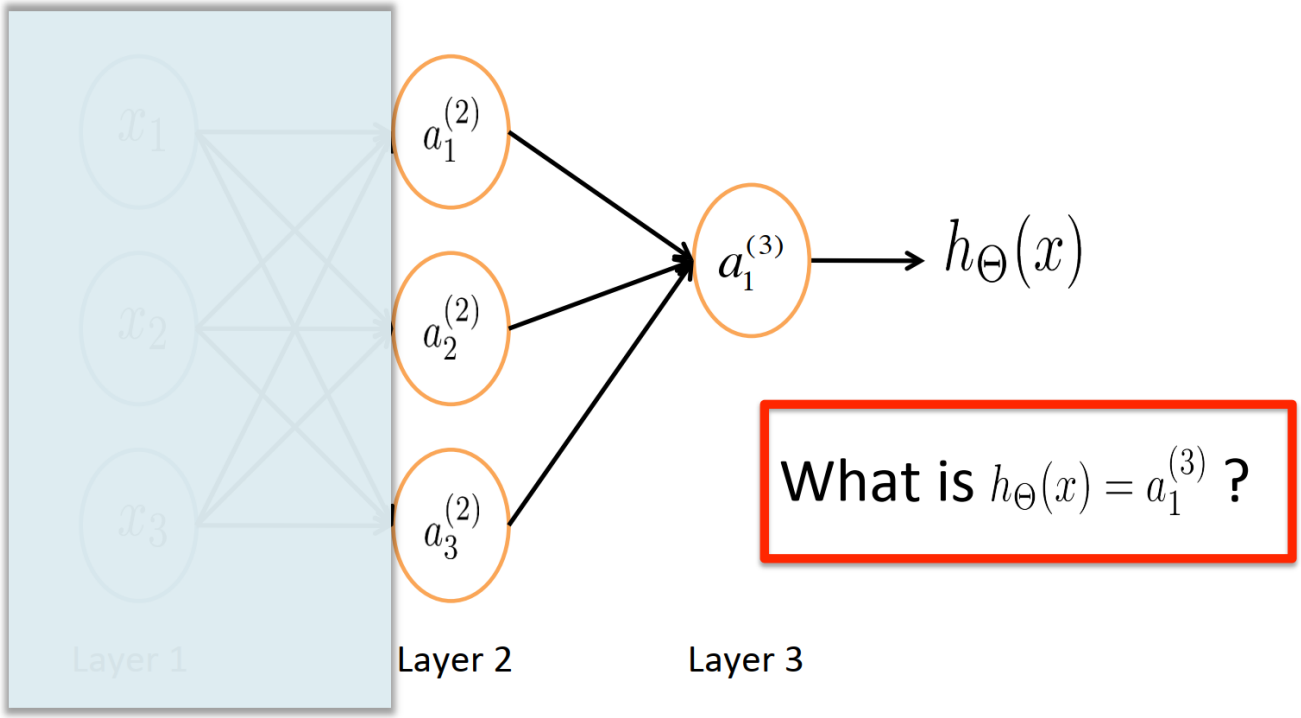
Where does $\theta_{3,1}^{(1)}$ go?

$\theta_{t,s}^{(j)}$ = entry in the matrix of weights
= weight on the edge from unit *s* in layer *j* to unit *t* in layer *j*+1



Model Representation

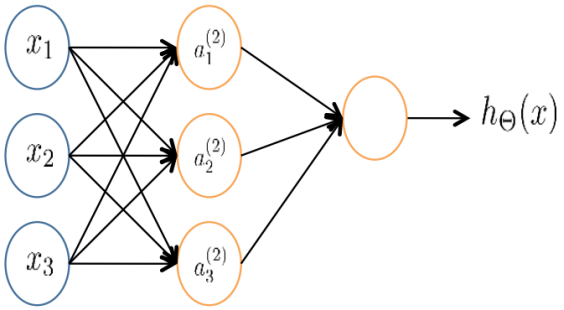
Neural network learns its own features



$$h_\Theta(x) = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

Model Representation

Neural Network



$a_i^{(j)}$ = "activation" of unit i in layer j

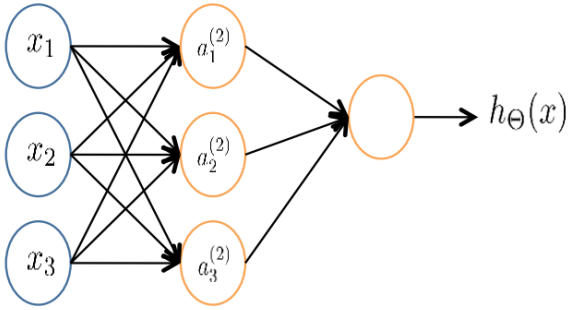
$\Theta^{(j)}$ = matrix of weights controlling function mapping from layer j to layer j+1

$$a_1^{(2)} = g(\Theta_{10}^{(1)}x_0 + \Theta_{11}^{(1)}x_1 + \Theta_{12}^{(1)}x_2 + \Theta_{13}^{(1)}x_3)$$
$$a_2^{(2)} = g(\Theta_{20}^{(1)}x_0 + \Theta_{21}^{(1)}x_1 + \Theta_{22}^{(1)}x_2 + \Theta_{23}^{(1)}x_3)$$
$$a_3^{(2)} = g(\Theta_{30}^{(1)}x_0 + \Theta_{31}^{(1)}x_1 + \Theta_{32}^{(1)}x_2 + \Theta_{33}^{(1)}x_3)$$
$$h_\Theta(x) = a_1^{(3)} = g(\Theta_{10}^{(2)}a_0^{(2)} + \Theta_{11}^{(2)}a_1^{(2)} + \Theta_{12}^{(2)}a_2^{(2)} + \Theta_{13}^{(2)}a_3^{(2)})$$

If network has s_j units in layer j, and s_{j+1} units in layer j+1, then $\Theta^{(j)}$ will be of dimension $s_{j+1} \times (s_j + 1)$

Model Representation

Forward propagation: Vectorized implementation



$$
\begin{aligned}
a_1^{(2)} &= g(\Theta_{10}^{(1)}x_0 + \Theta_{11}^{(1)}x_1 + \Theta_{12}^{(1)}x_2 + \Theta_{13}^{(1)}x_3) \\
a_2^{(2)} &= g(\Theta_{20}^{(1)}x_0 + \Theta_{21}^{(1)}x_1 + \Theta_{22}^{(1)}x_2 + \Theta_{23}^{(1)}x_3) \\
a_3^{(2)} &= g(\Theta_{30}^{(1)}x_0 + \Theta_{31}^{(1)}x_1 + \Theta_{32}^{(1)}x_2 + \Theta_{33}^{(1)}x_3)
\end{aligned}
$$

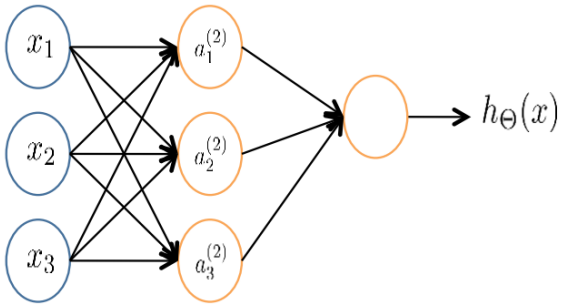
$$
x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \qquad z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix}
$$

$$
z^{(2)} = \Theta^{(1)}x
$$

$$
a^{(2)} = g(z^{(2)})
$$

Model Representation

Forward propagation: Vectorized implementation



$$a^{(1)} = x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \qquad z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix}$$

$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$
$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$
$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

$$h_\Theta(x) = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

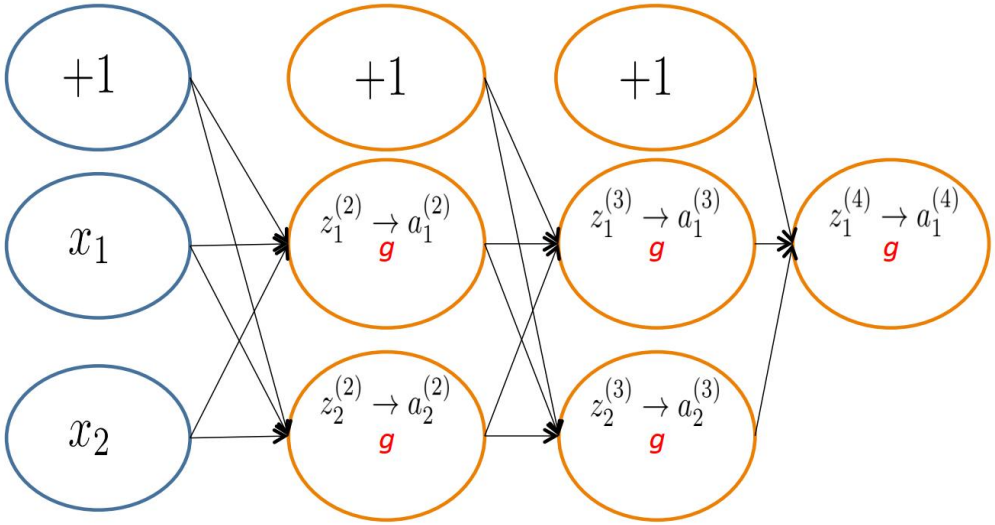
$$z^{(2)} = \Theta^{(1)} x$$
$$a^{(2)} = g(z^{(2)})$$

Add $a_0^{(2)} = 1.$

$$z^{(3)} = \Theta^{(2)} a^{(2)}$$
$$h_\Theta(x) = a^{(3)} = g(z^{(3)})$$

NN with 2 hidden layers

Forward Propagation

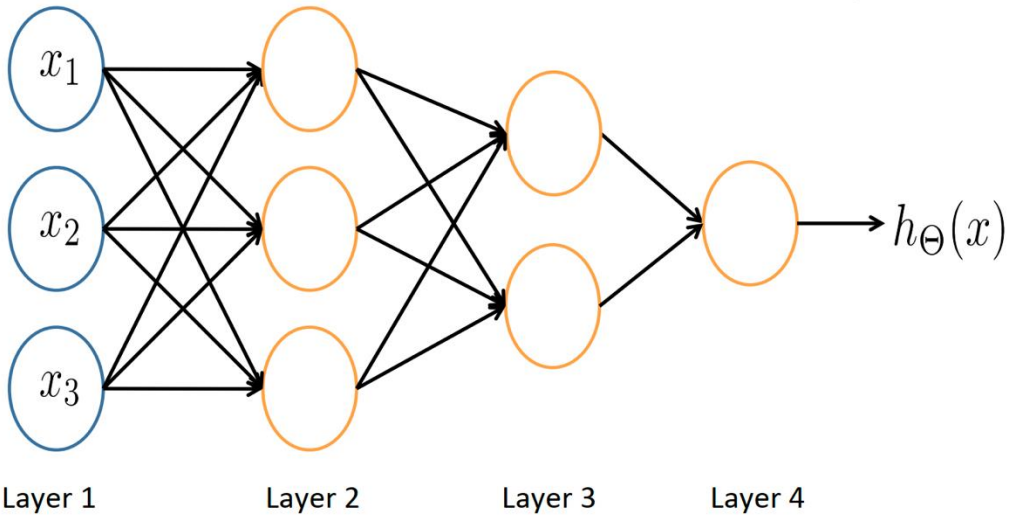


$$g(z) = \frac{1}{1 + e^{-z}}$$

NN learn complex features

Network architectures

1. Edges
2. Patches
3. Surfaces, and
4. Objects



Neural networks learn complex features through their hidden layers

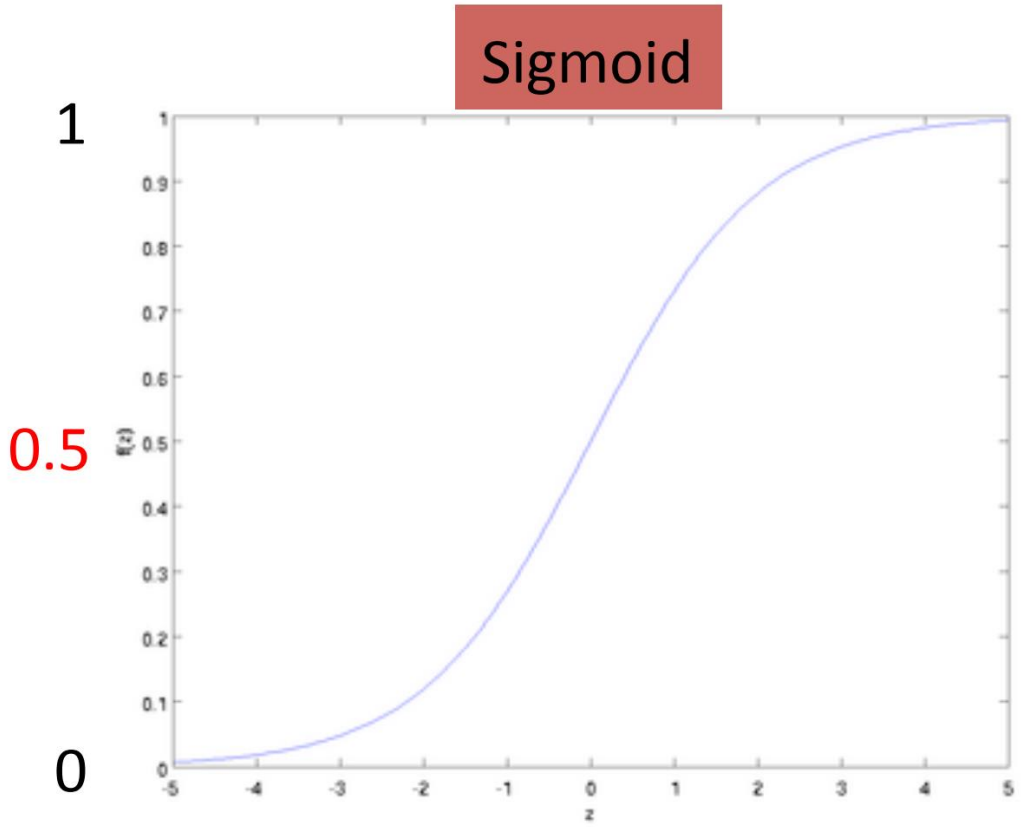
Activation Function

The activation function is a mathematical “gate” in between the input feeding the current neuron and its output going to the next layer. It can be as simple as a step function that turns the neuron output on and off, depending on a rule or threshold.

Sigmoid Activation Function

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z)).$$



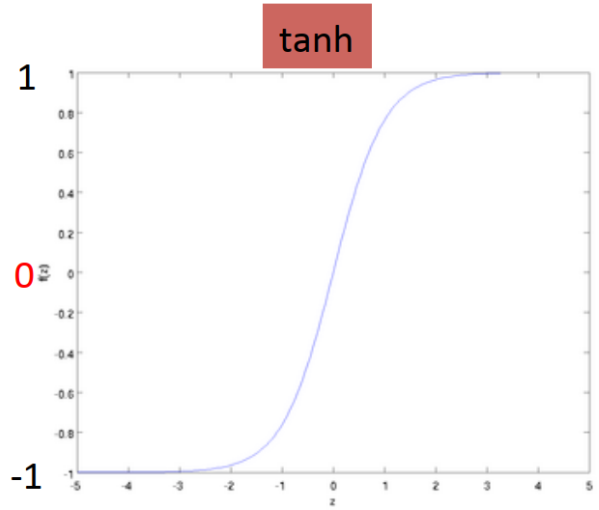
Tangent activation function

Another commonly used activation function is the hyperbolic tangent, tanh

- The tanh(z) function is a rescaled version of the sigmoid, and its output range is $[-1,1]$ instead of $[0,1]$.

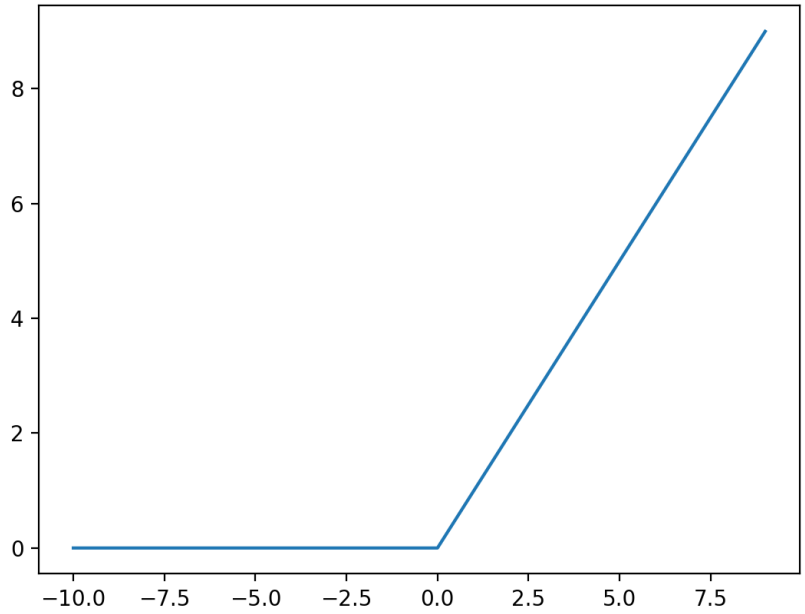
$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}},$$

- $f'(z) = 1 - (f(z))^2$

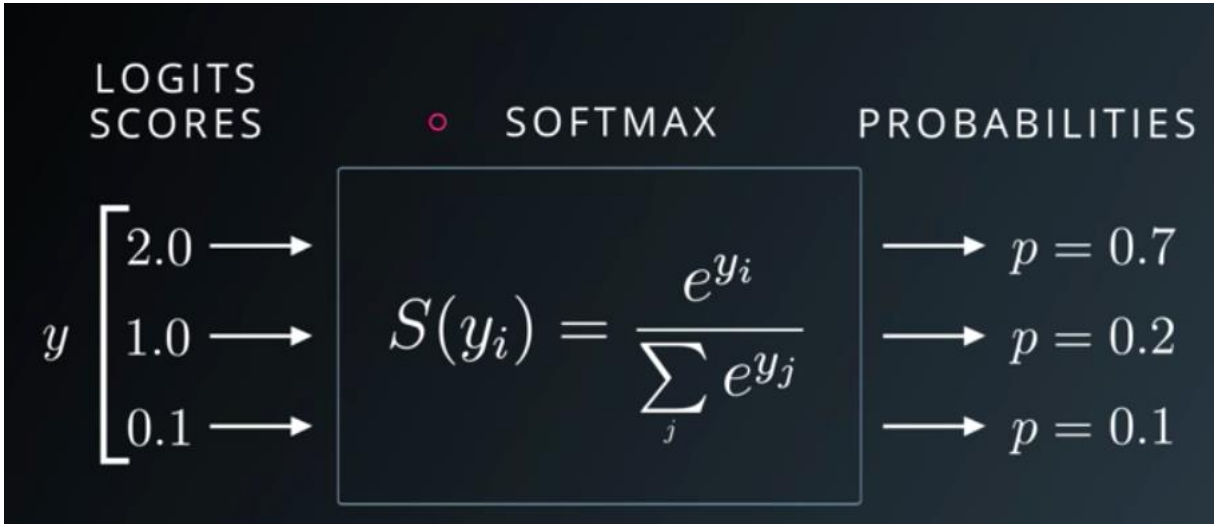


ReLU activation function

Rectifier Linear Unit



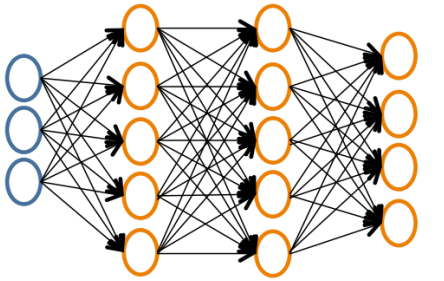
Softmax Activation



Usually used for last layer with its output representing the possibility of classes

Model Training

Neural Network (Classification)



$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})\}$

L = total no. of layers in network

s_l = # of units (not counting bias unit) in layer l

Binary classification

$y = 0$ or 1

1 output unit

Multi-class classification (K classes)

$y \in \mathbb{R}^K$ E.g. $\begin{bmatrix}1\\0\\0\\0\end{bmatrix}, \begin{bmatrix}0\\1\\0\\0\end{bmatrix}, \begin{bmatrix}0\\0\\1\\0\end{bmatrix}, \begin{bmatrix}0\\0\\0\\1\end{bmatrix}$

pedestrian car motorcycle truck

K output units

Cost Function (Objective)

Cost function

L = total no. of layers in network

s_l = no. of units (not counting bias unit) in layer l

One output

$$J(\theta) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))\right] + \frac{\lambda}{2m}\sum_{j=1}^{n} \theta_j^2$$

Training Error

Regularization

K outputs

$$h_\Theta(x) \in \mathbb{R}^K \quad (h_\Theta(x))_i = i^{th} \text{ output}$$

$$J(\Theta) = -\frac{1}{m}\left[\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log(h_\Theta(x^{(i)}))_k + (1 - y_k^{(i)}) \log(1 - (h_\Theta(x^{(i)}))_k)\right]$$

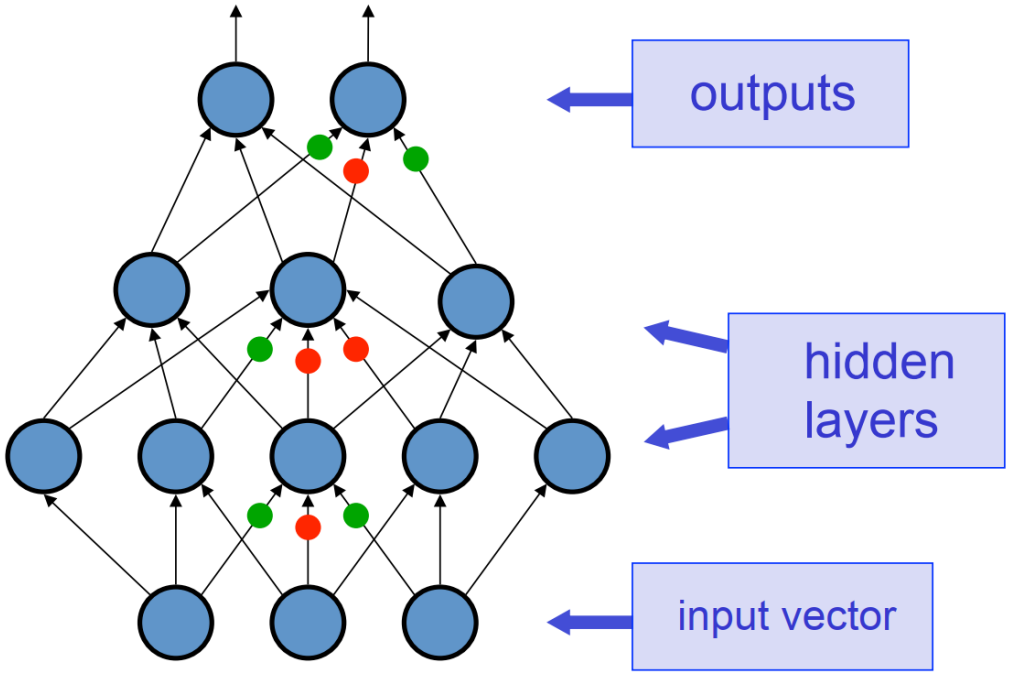
$$+\frac{\lambda}{2m}\sum_{l=1}^{L-1}\sum_{i=1}^{s_l}\sum_{j=1}^{s_{l+1}}(\Theta_{ji}^{(l)})^2$$

Back propagation

Back-propagate error signal to get derivatives for learning



Compare outputs with correct answer to get error signal



Slide by Geoff Hinton

Gradient Descent:
What if p is large? (e.g., n/2)

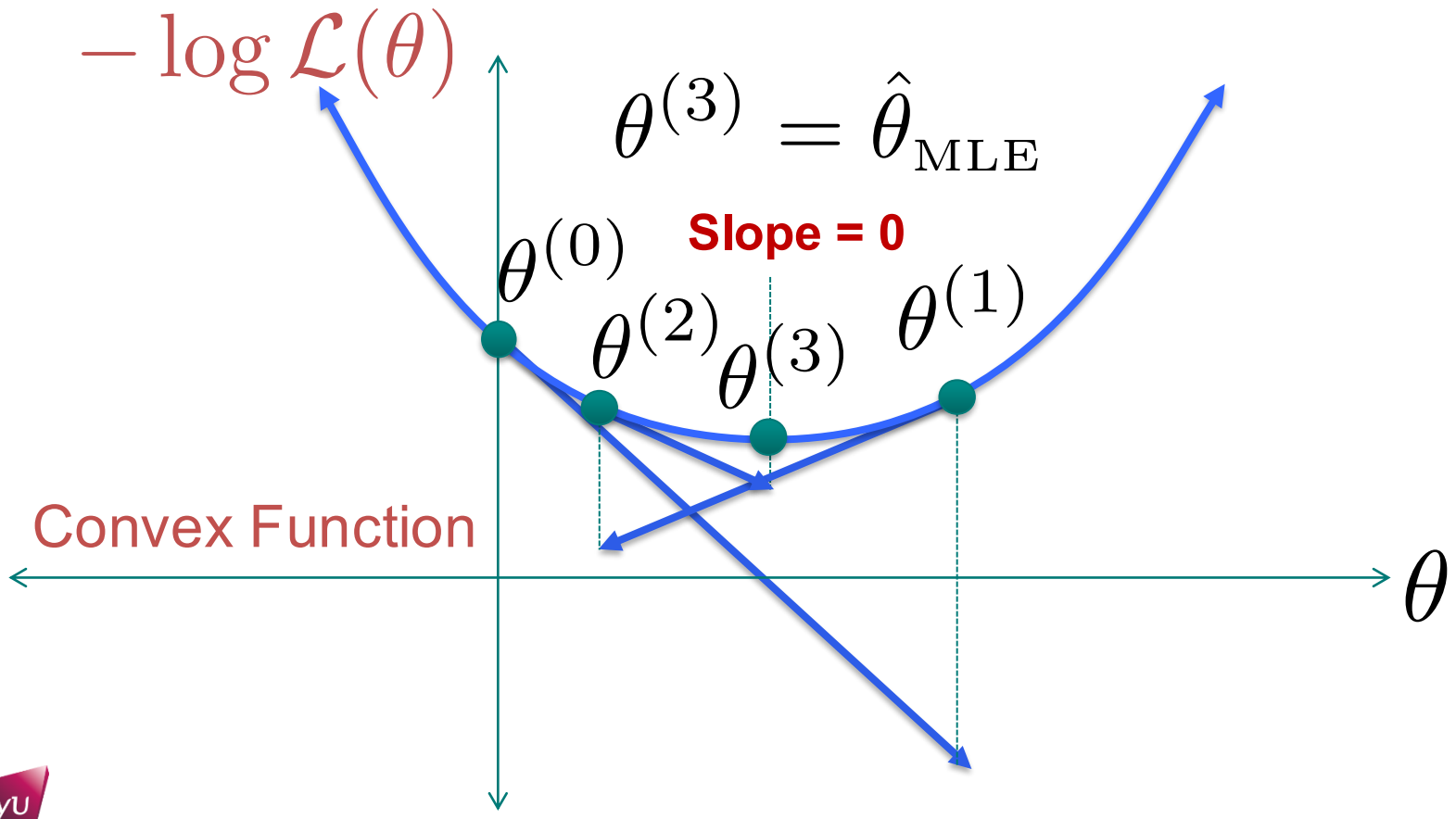
- The cost of $O(np^2) = O(n^3)$ could by prohibitive
- Solution: Iterative Methods
  - Gradient Descent:

    For τ from 0 until *convergence*

$$\theta^{(\tau+1)} = \theta^{(\tau)} - \rho(\tau)\nabla \log \mathcal{L}(\theta^{(\tau)}|D)$$

Learning rate

Gradient Descent Illustrated:



Gradient computation given cost function

Gradient computation

$$J(\Theta) = -\frac{1}{m}\left[\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log h_\theta(x^{(i)})_k + (1 - y_k^{(i)}) \log(1 - h_\theta(x^{(i)})_k)\right]$$
$$+\frac{\lambda}{2m}\sum_{l=1}^{L-1}\sum_{i=1}^{s_l}\sum_{j=1}^{s_{l+1}}(\Theta_j^{(l)})^2$$

$$\min_{\Theta} J(\Theta)$$

Need code to compute:

$J(\Theta)$	$\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta)$

$$\theta := \theta - \alpha \frac{d}{d\theta} J(\theta).$$

Overfitting of NN

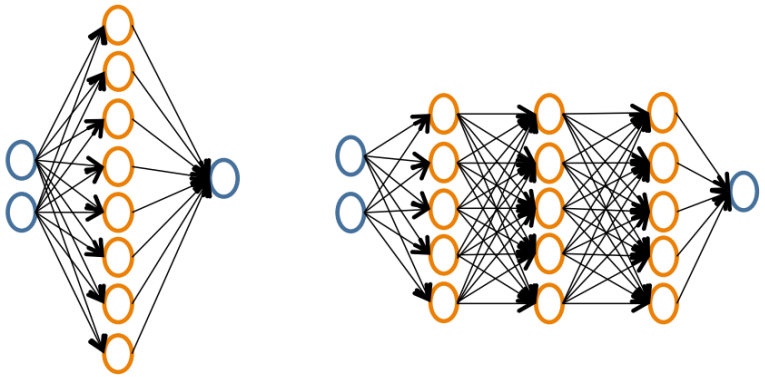
Neural networks and overfitting

"Small" neural network
(fewer parameters; more prone to underfitting)



Computationally cheaper

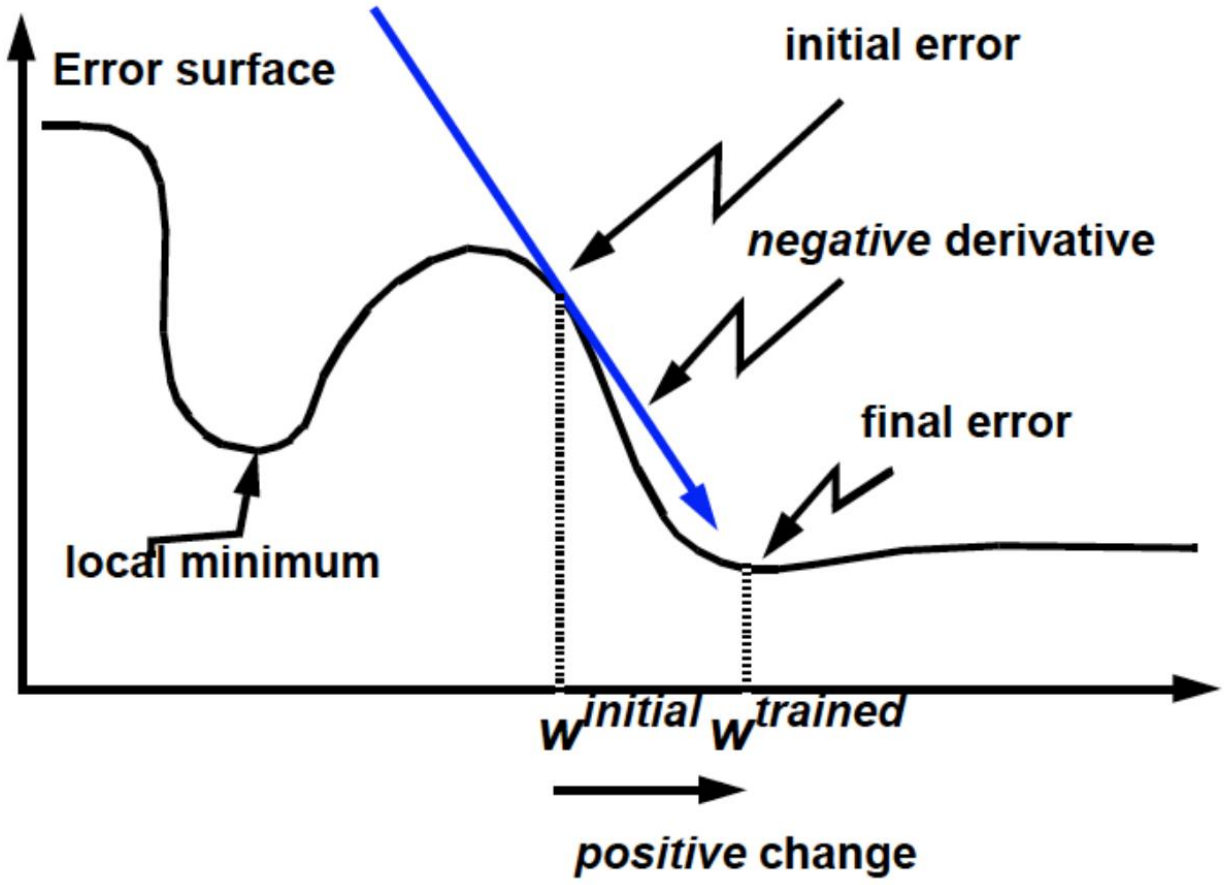
"Large" neural network
(more parameters; more prone to overfitting)



Computationally more expensive.

Use regularization (λ) to address overfitting.

Local Optimal Parameters



Example

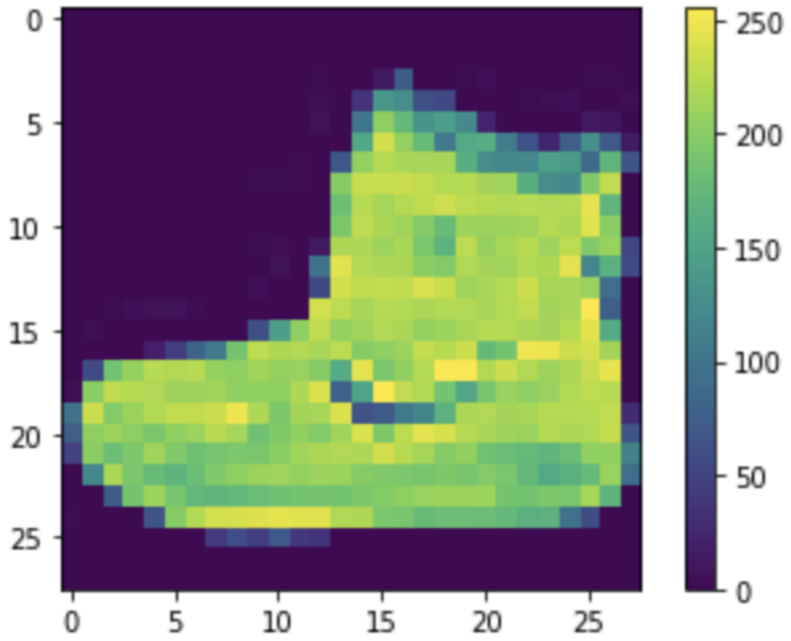
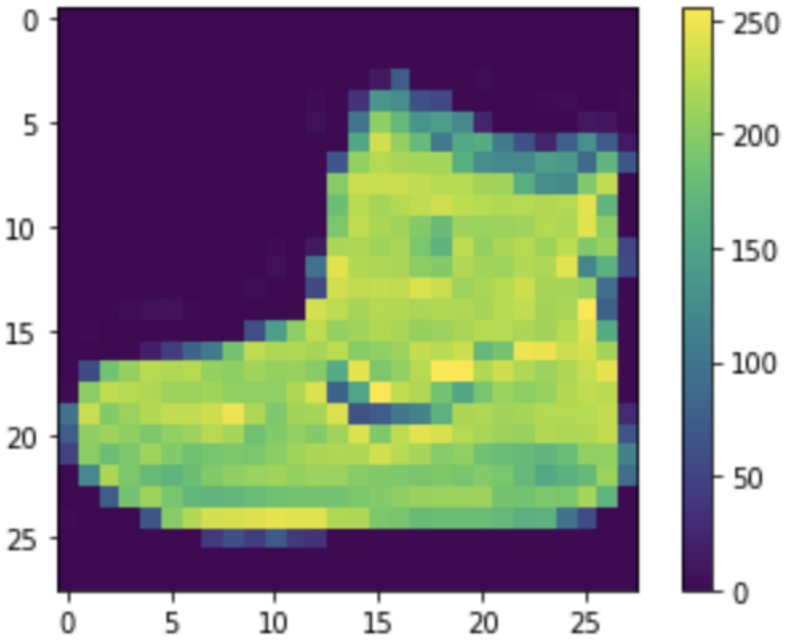


Image classification

Label	Class
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

Example



Object has 28*28 features (pixels)

Example

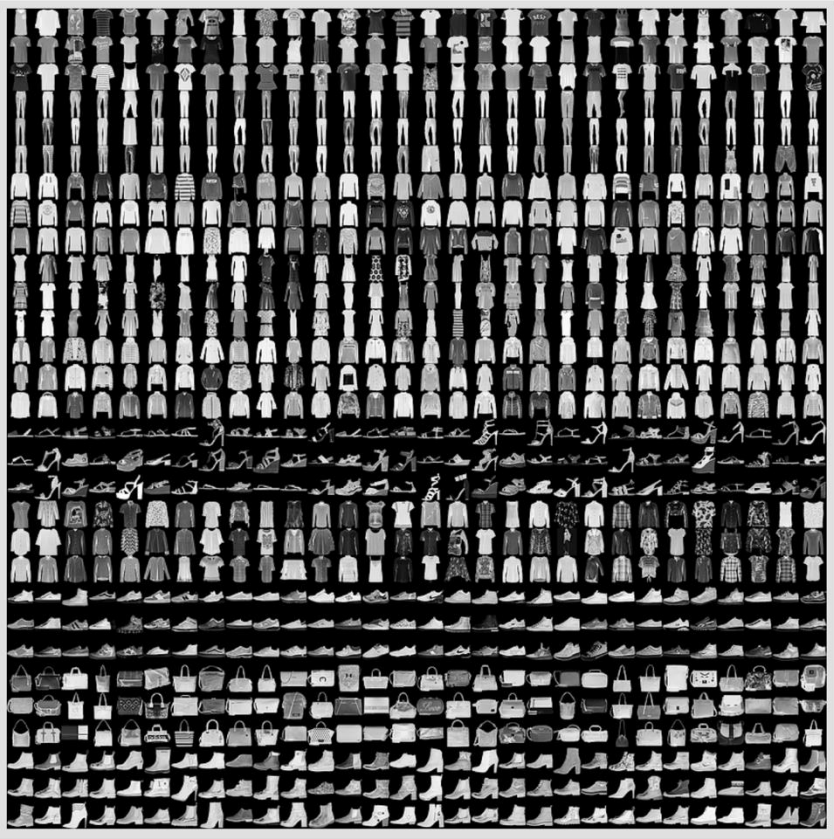
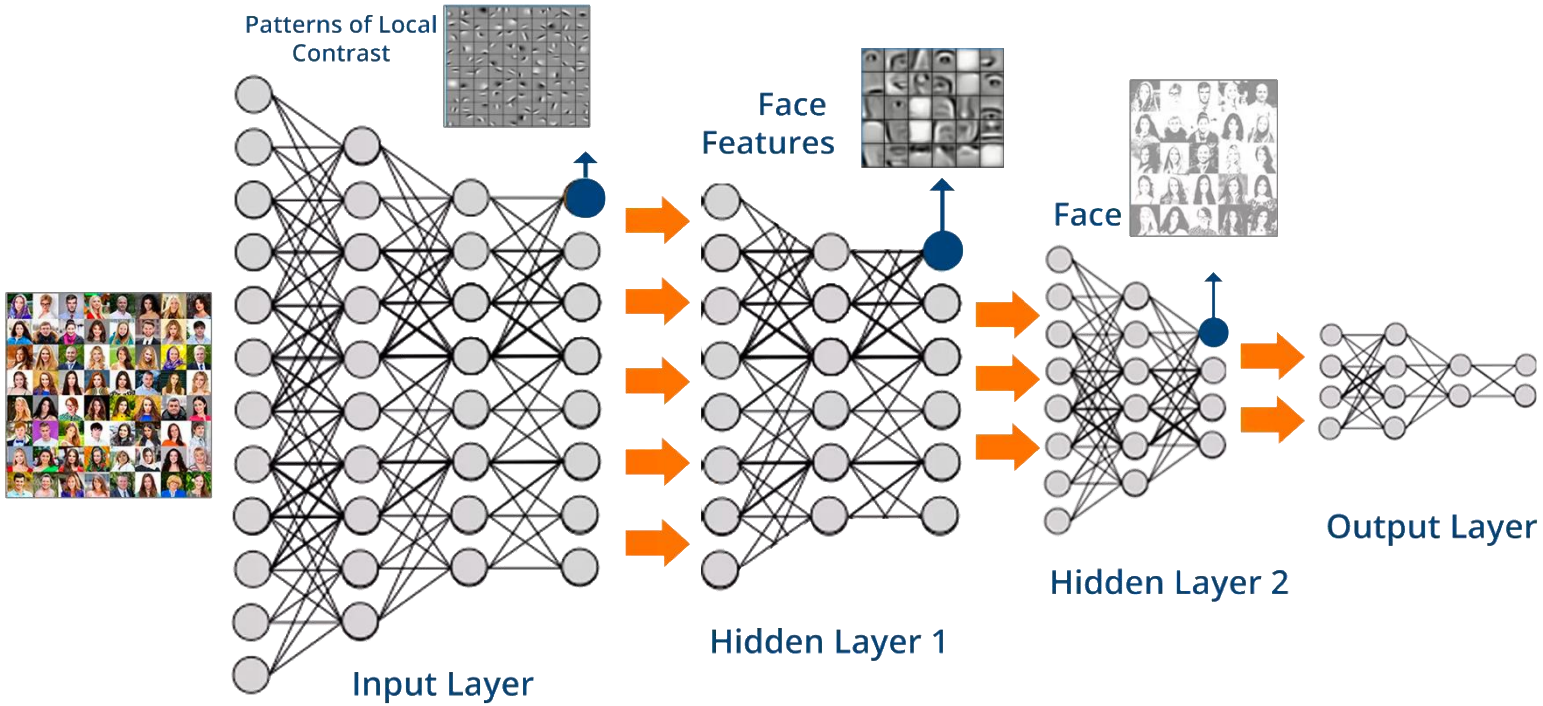


Figure 1. Fashion-MNIST samples (by Zalando, MIT License).

60,000 objects with labels for model training

Example

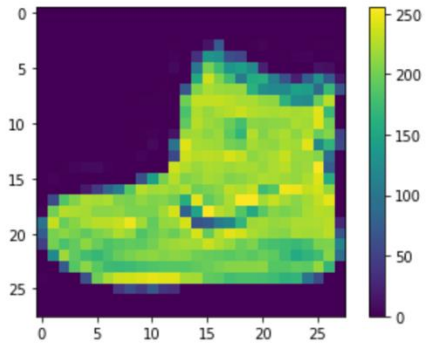


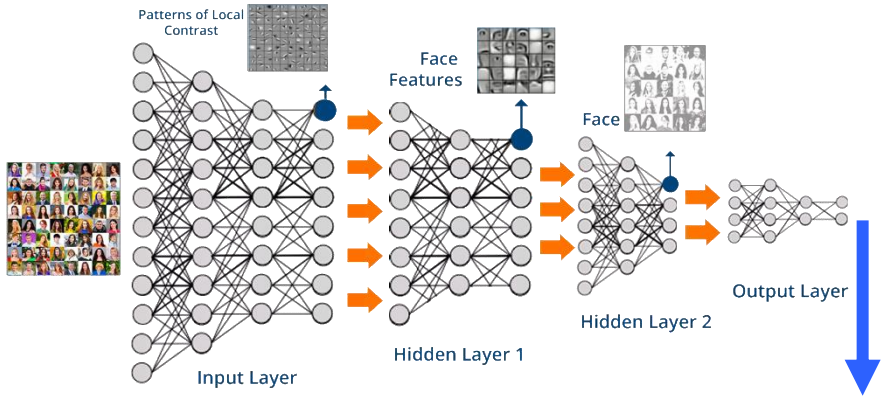
Identify the following parameters:

- Input layer dimension (for features input)
- Number of hidden layers
- Number of nodes in hidden layers
- Activation function
- Loss function (objective function)

Example

Testing object → Trained ANN model





Label	Class
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

Output y

$$y = (p_0, p_1, p_2, p_3, p_4, p_5, p_6, p_7, p_8, p_9)$$

$$p_i : probability\ of\ class\ i$$

If the max probability is p_j,
prediction result is class j

IS6400 Business Data Analytics

Dr. LIU Junming

City University of Hong Kong

Lecture 10

- Intro-Deep Learning
  - ANN
  - RNN, LSTM
  - NLP
- Image Mining

Reading Material:

-

FFNN

Static Input and Static Output

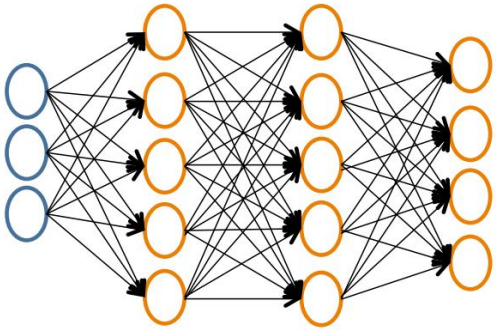
Multiple output units: One-vs-all.


Pedestrian


Car


Motorcycle


Truck



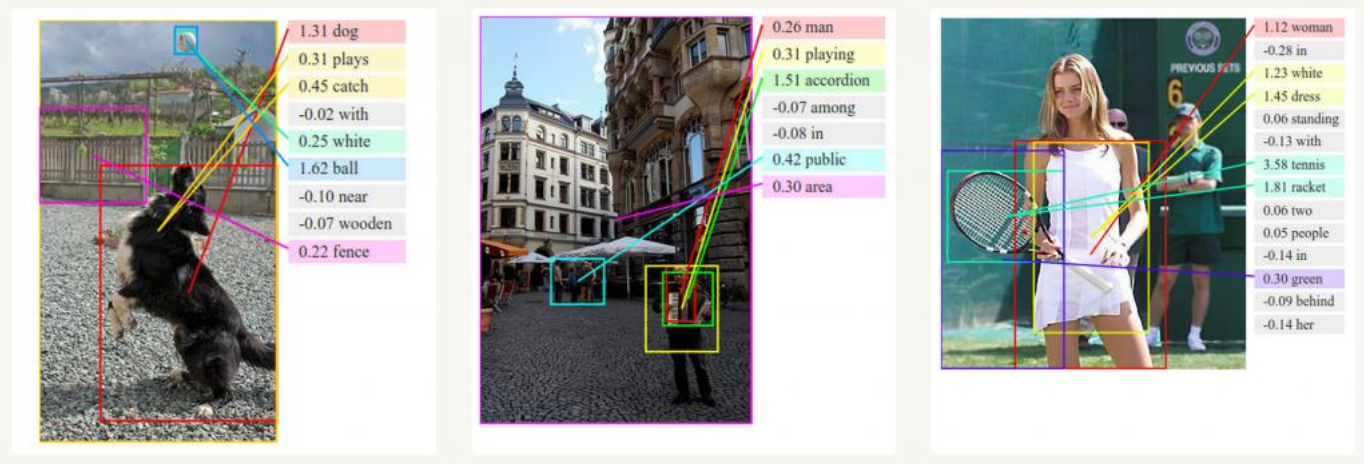
$$h_\Theta(x) \in \mathbb{R}^4$$

Want $h_\Theta(x) \approx \begin{bmatrix}1\\0\\0\\0\end{bmatrix}$, $h_\Theta(x) \approx \begin{bmatrix}0\\1\\0\\0\end{bmatrix}$, $h_\Theta(x) \approx \begin{bmatrix}0\\0\\1\\0\end{bmatrix}$, etc.

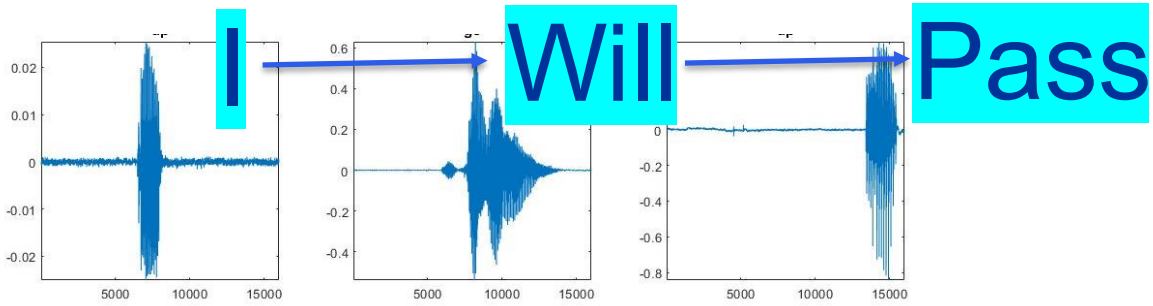
when pedestrian when car when motorcycle

NN for Sequence Analysis

What if I want to analyze something **in a sequence**?

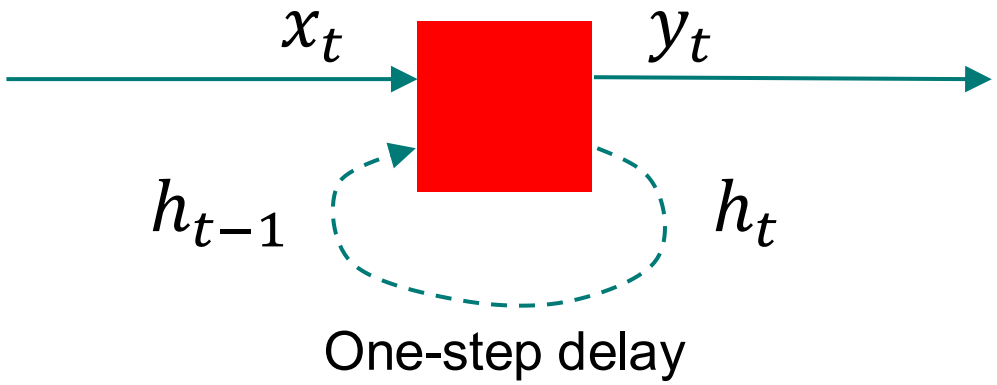


What if I want to analyze human speech?



Recurrent Neural Networks (RNNs)

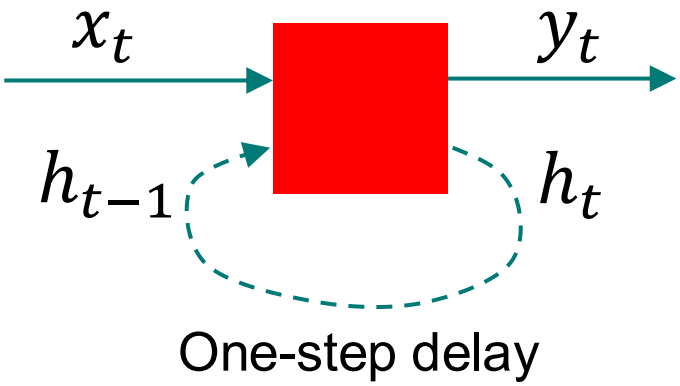
Recurrent networks introduce cycles and a notion of time.



- They are designed to process sequences of data $x_1, \ldots, x_n$ and can produce sequences of outputs $y_1, \ldots, y_m$.

Unrolling RNNs

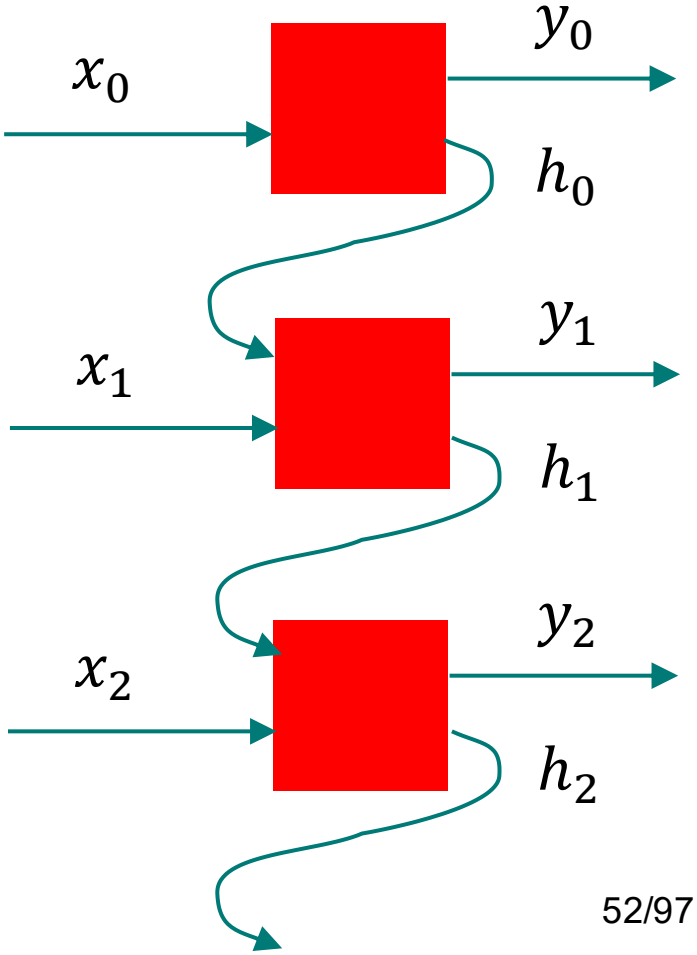
RNNs can be unrolled across multiple time steps.



One-step delay

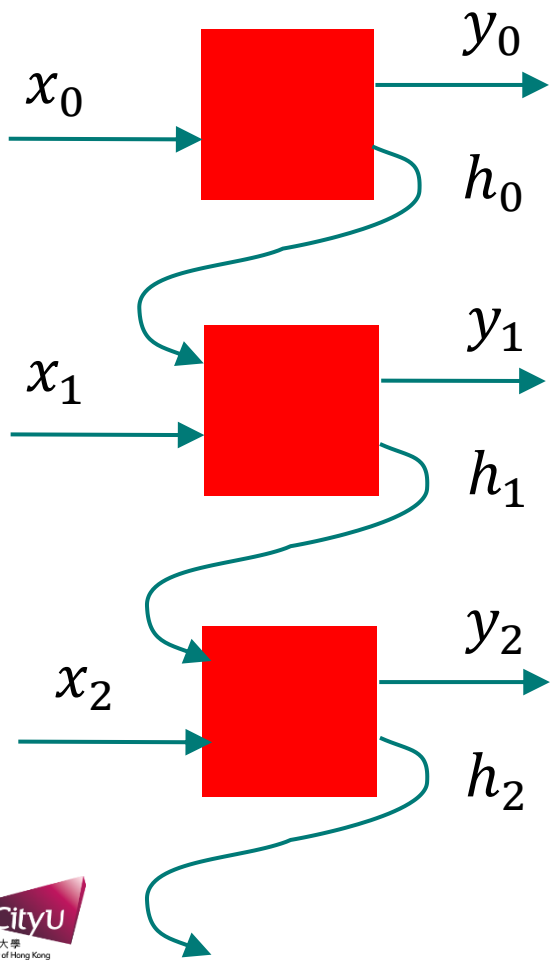
This produces a DAG which supports backpropagation.

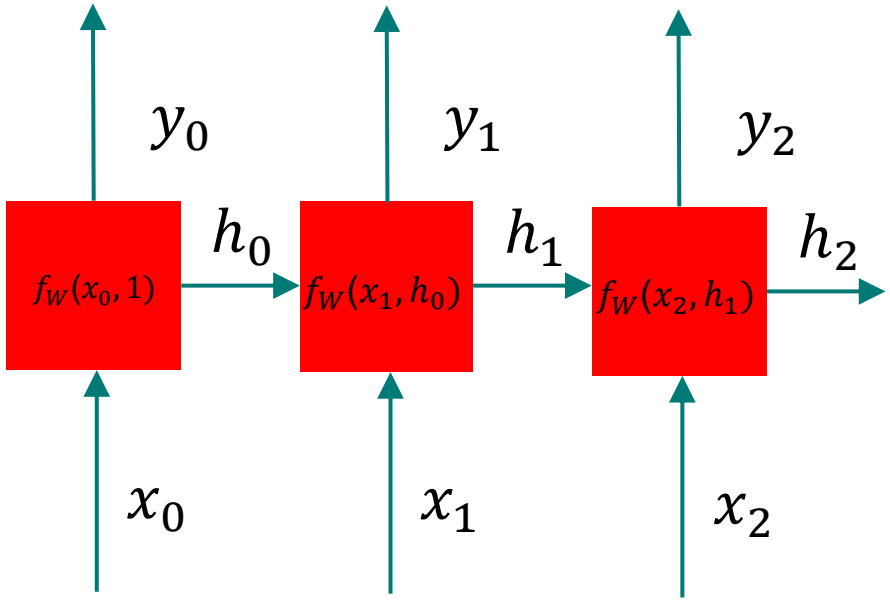
But its size depends on the input sequence length.



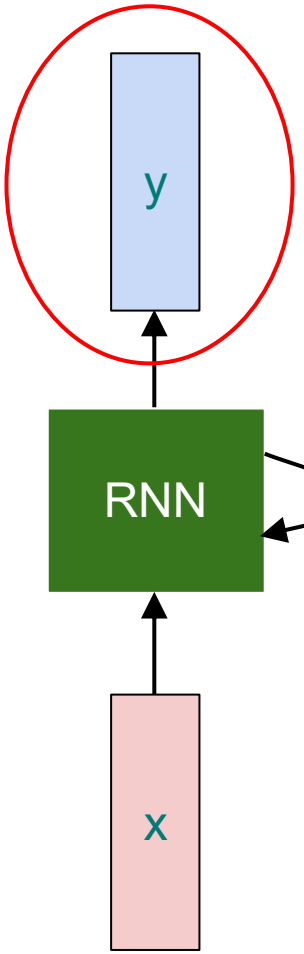
Unrolling RNNs

Usually drawn as:





RNN



Y: Target
(usually vector of a sequence)

h: Hidden State
(Information feedback to input layer)

X: Input
(Observations)

RNN for Time Series

We can process a sequence of vectors **x** by applying a recurrence formula at every time step:

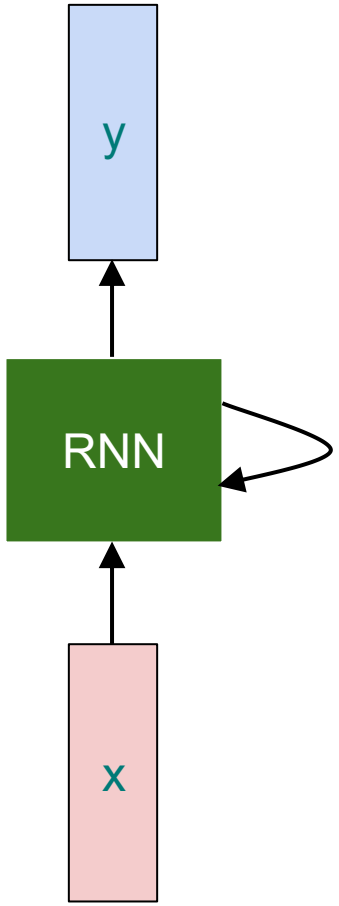
$$h_t = f_W(h_{t-1}, x_t)$$

new state

old state

input vector at some time step

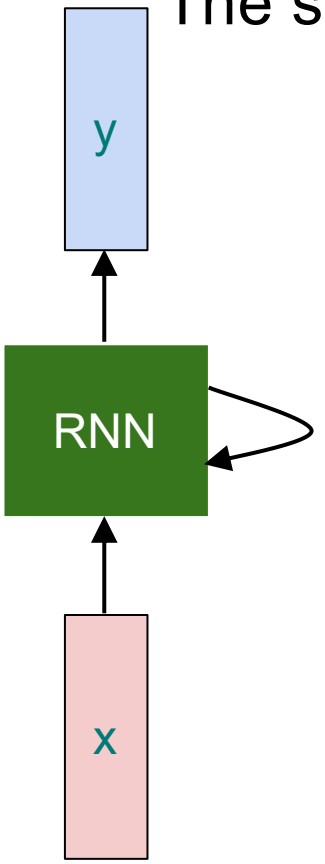
some function with parameters W



RNN Model is Simplified:
We have the same parameter set W for different time stamps t.

(Vanilla) RNN for Time Series

The state consists of a single *"hidden"* vector **h**:



$$h_t = f_W(h_{t-1}, x_t)$$

Tanh activation function

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

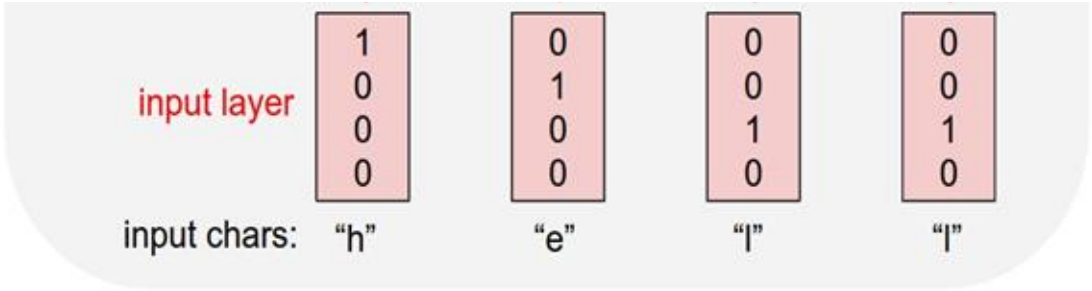
$$y_t = W_{hy}h_t$$

Example

Vocabulary:
[h,e,l,o]

Example training
Input sequence:
“hello”

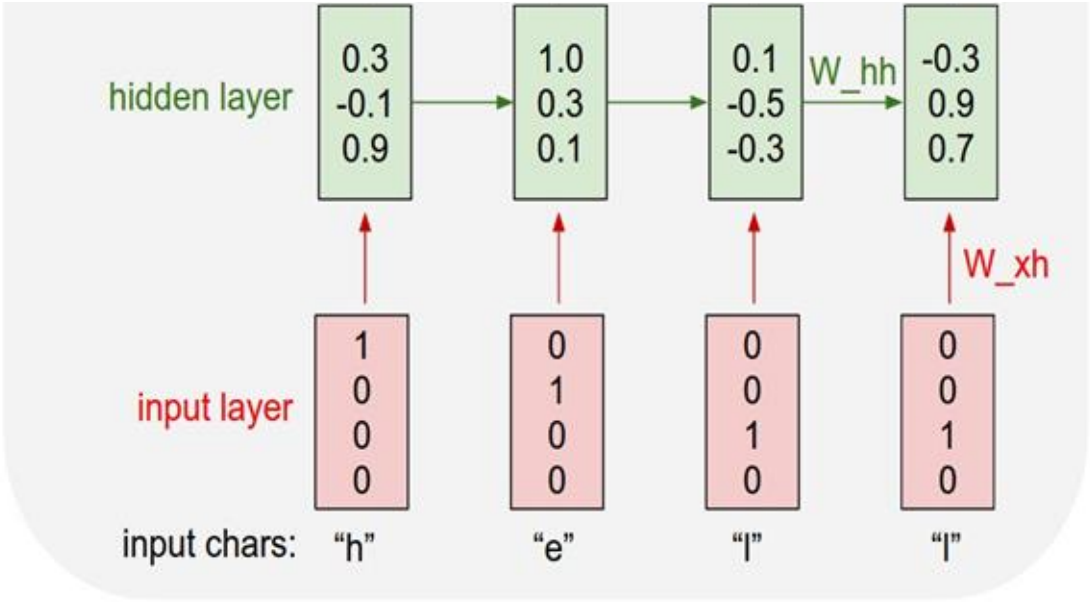




Example

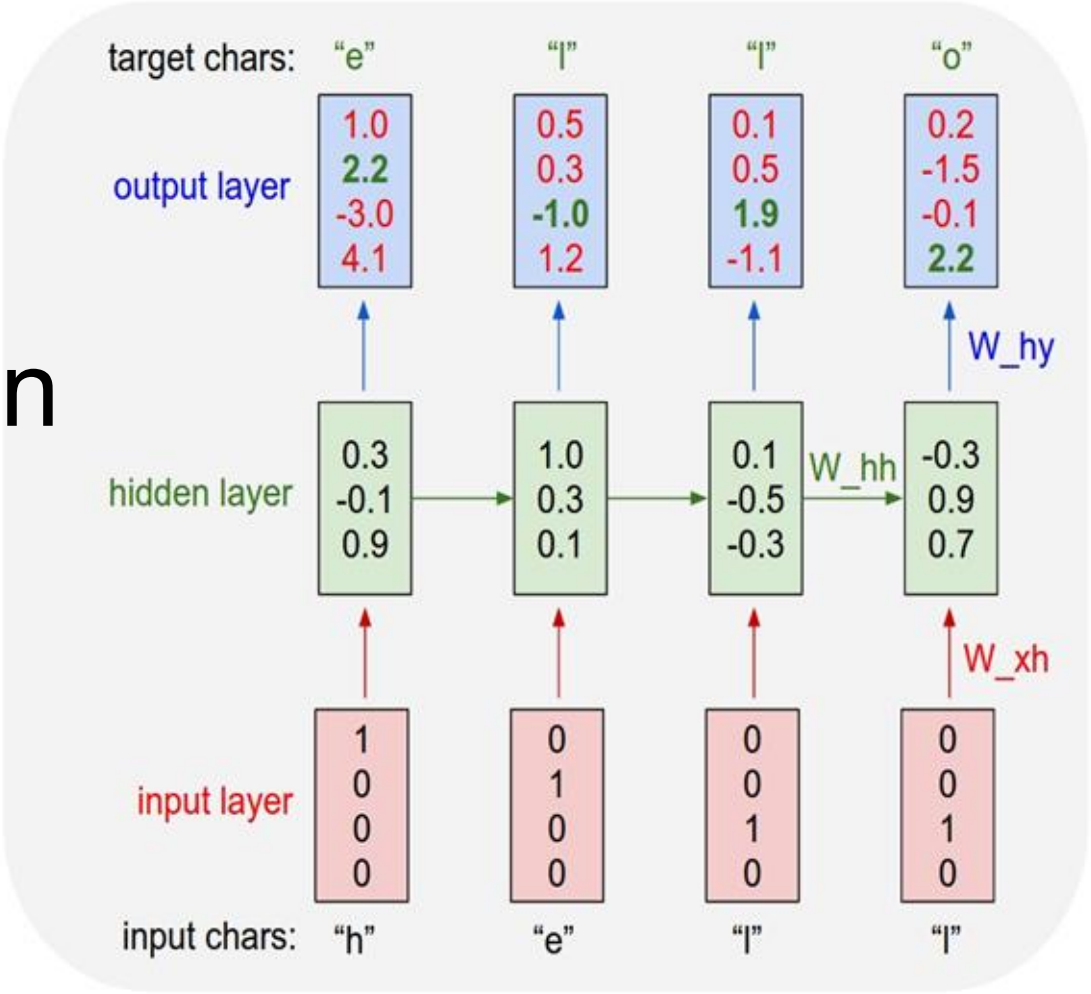
Example hidden states calculation

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

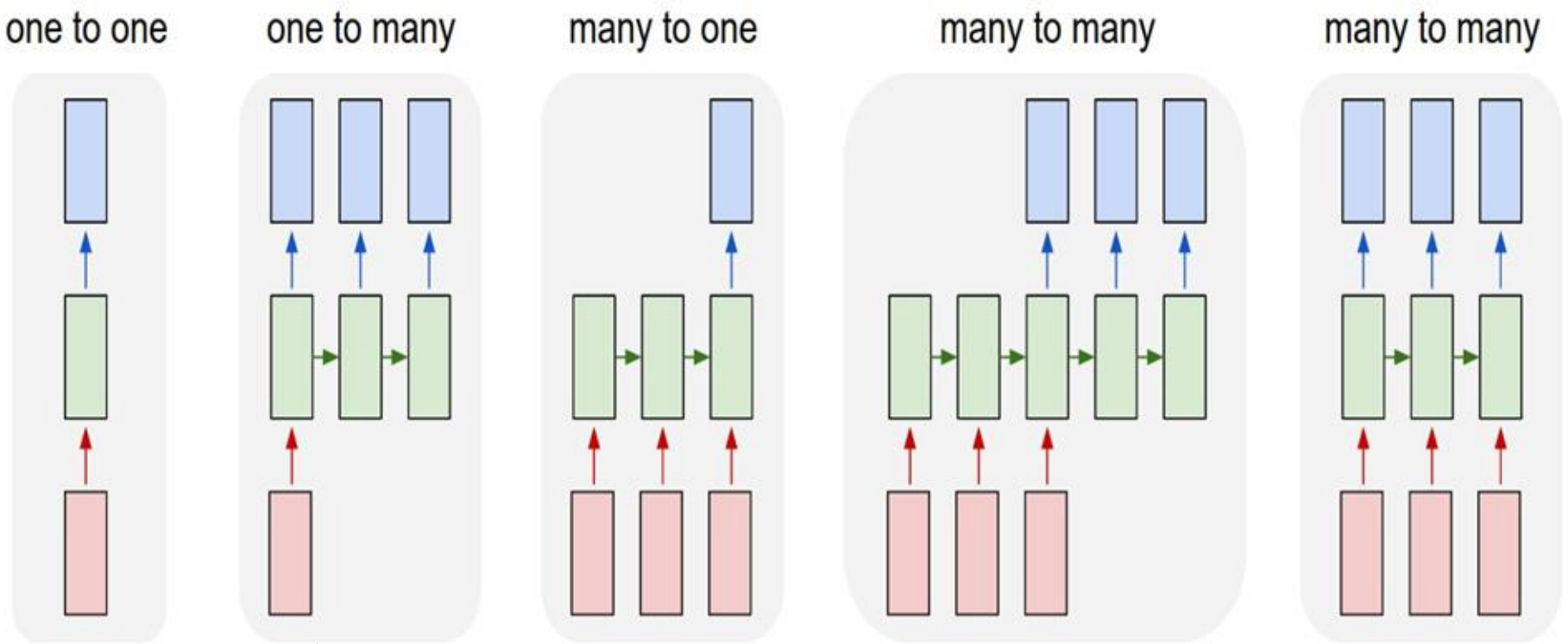


Example

Example output vector calculation

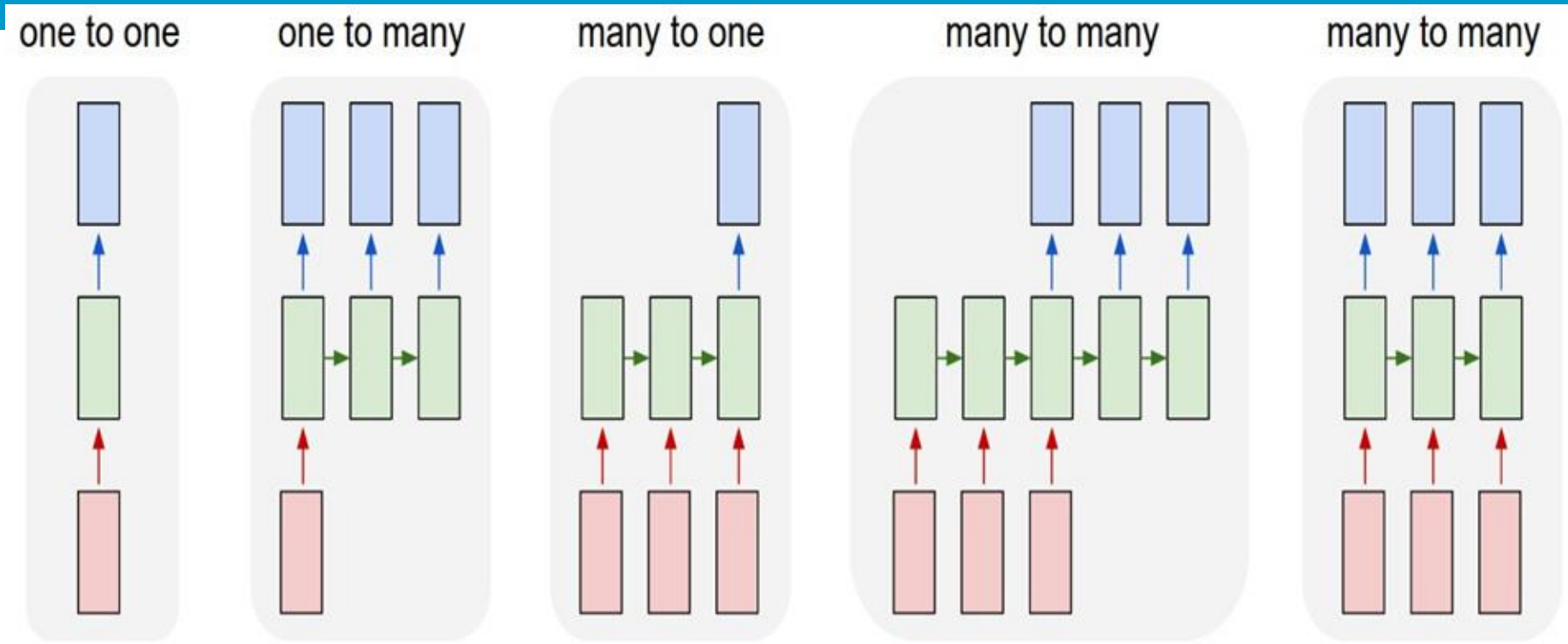


Recurrent Networks offer a lot of flexibility:



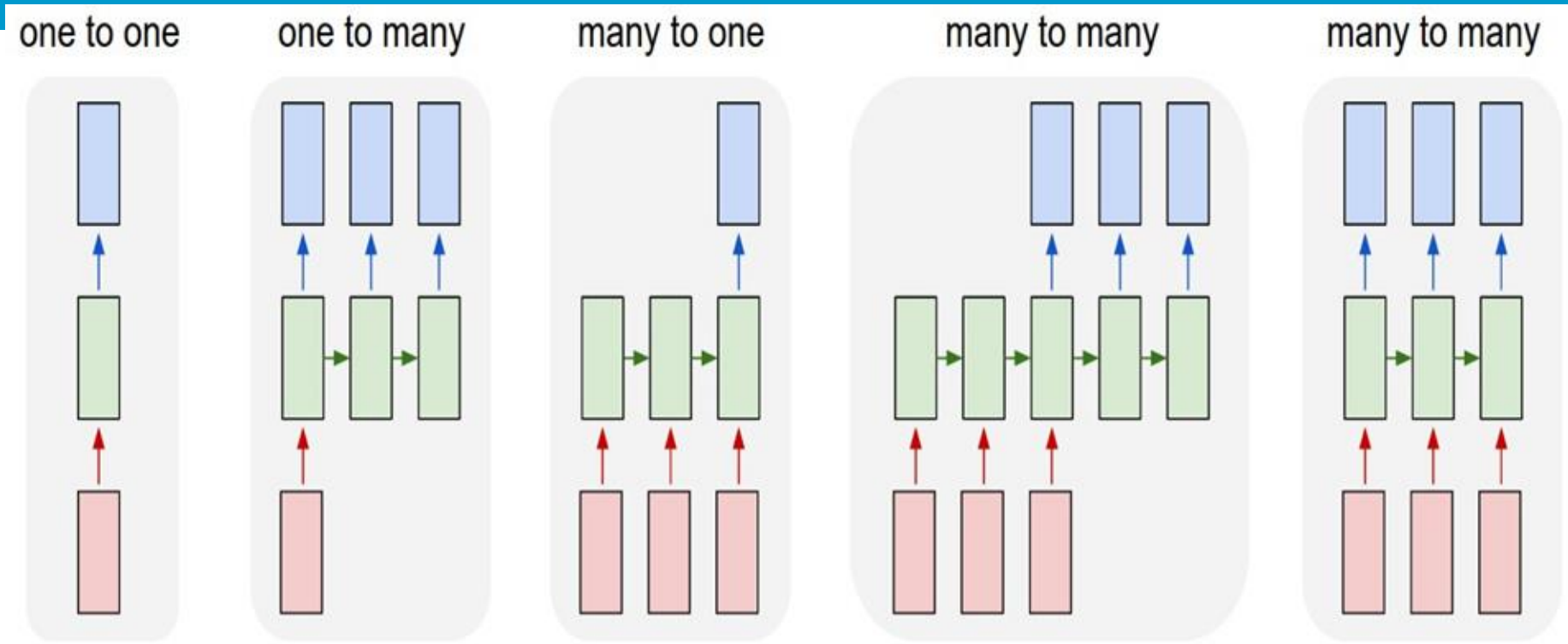
e.g. **Image Captioning**
image -> sequence of words

Recurrent Networks offer a lot of flexibility:



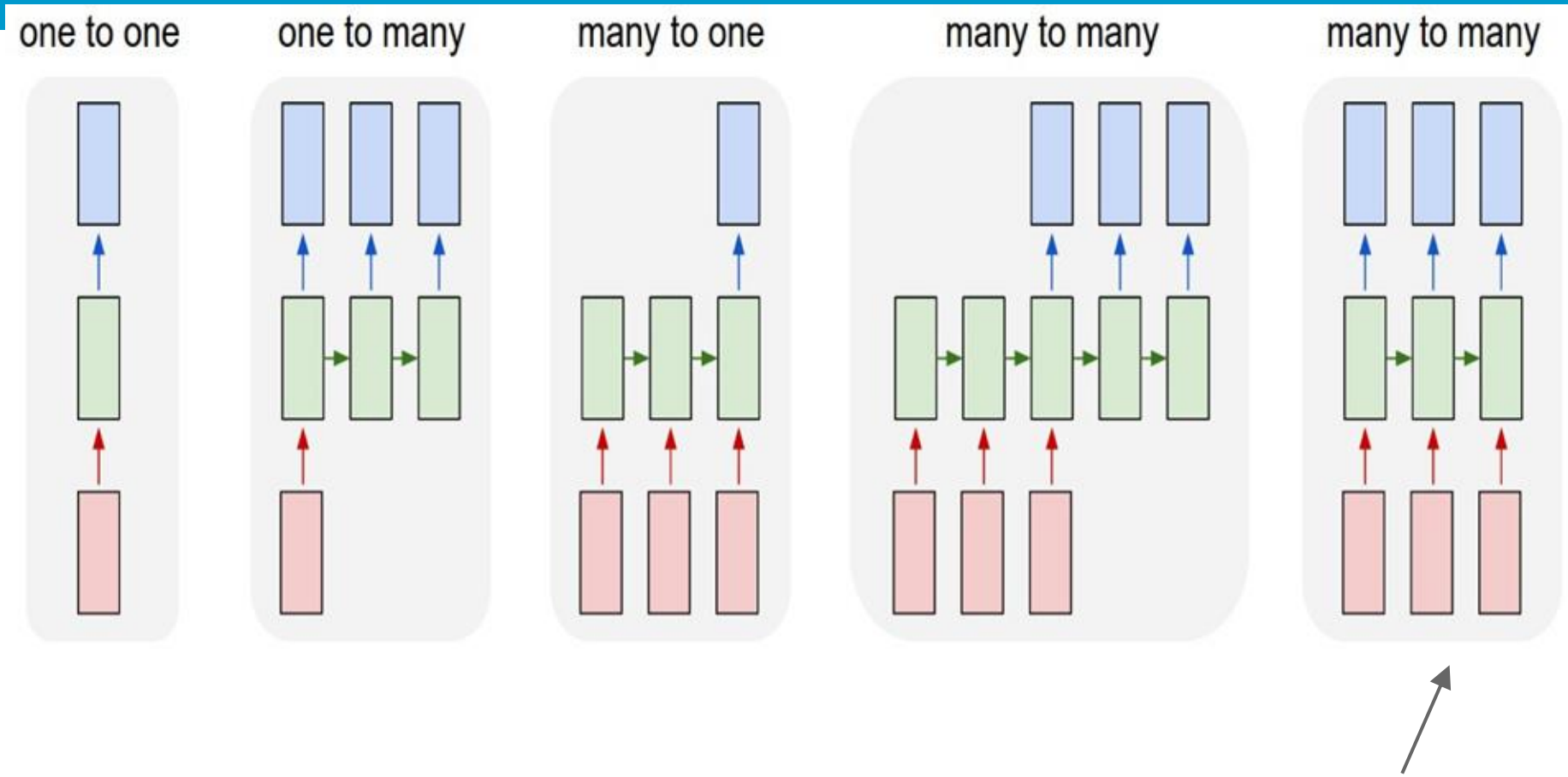
e.g. **Sentiment Classification**
sequence of words -> sentiment

Recurrent Networks offer a lot of flexibility:



e.g. **Machine Translation**
seq of words -> seq of words

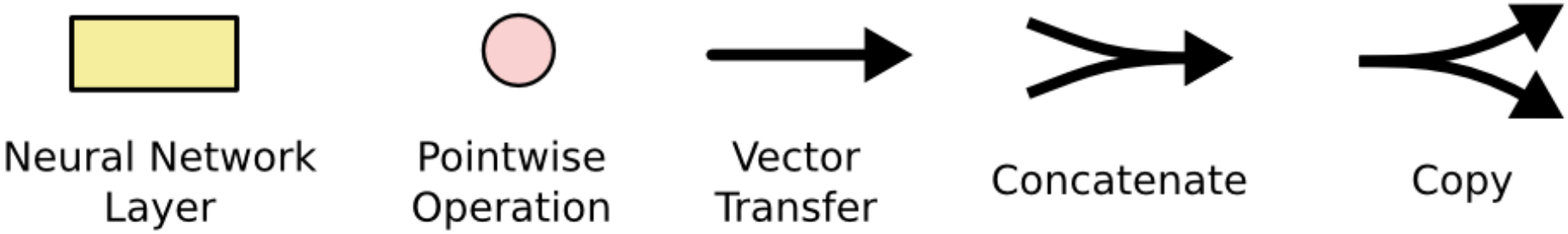
Recurrent Networks offer a lot of flexibility:

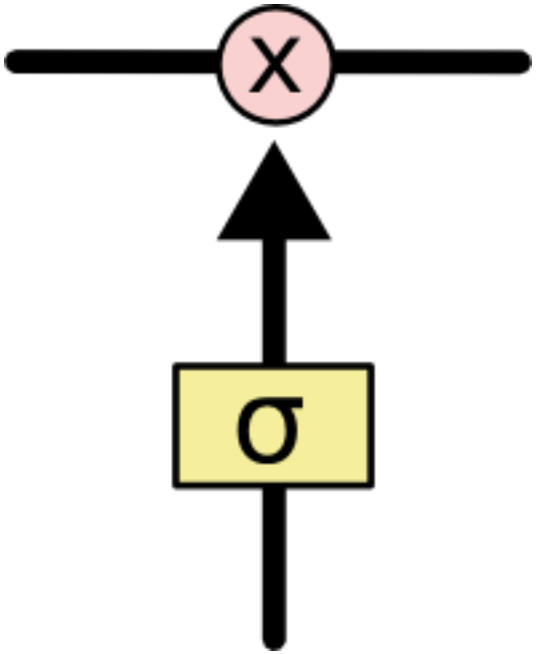


e.g. **Video classification**

Problems of RNN

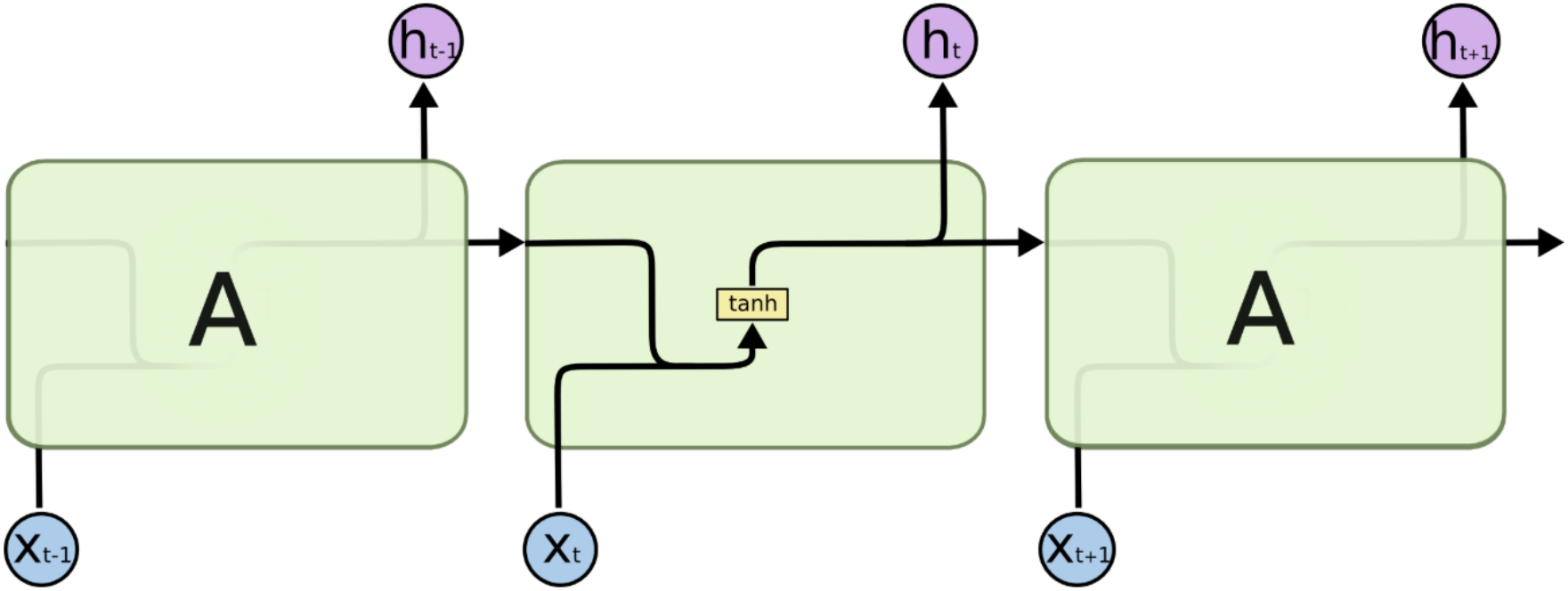
- When dealing with a time series, it tends to forget old information. When there is a distant relationship of unknown length, we wish to have a “memory” to it.
- Vanishing gradient problem.





The sigmoid layer outputs numbers between 0-1 determine how much each component should be let through. Pink X gate is point-wise multiplication.

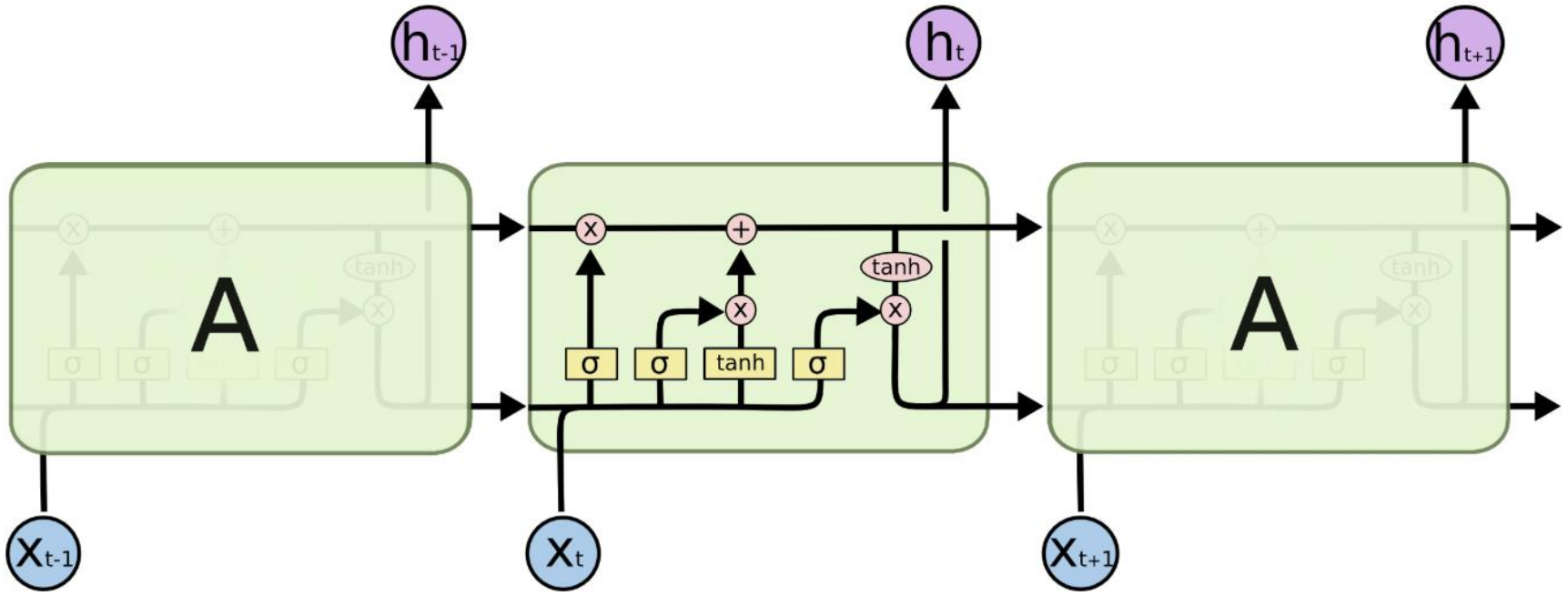
RNN Model



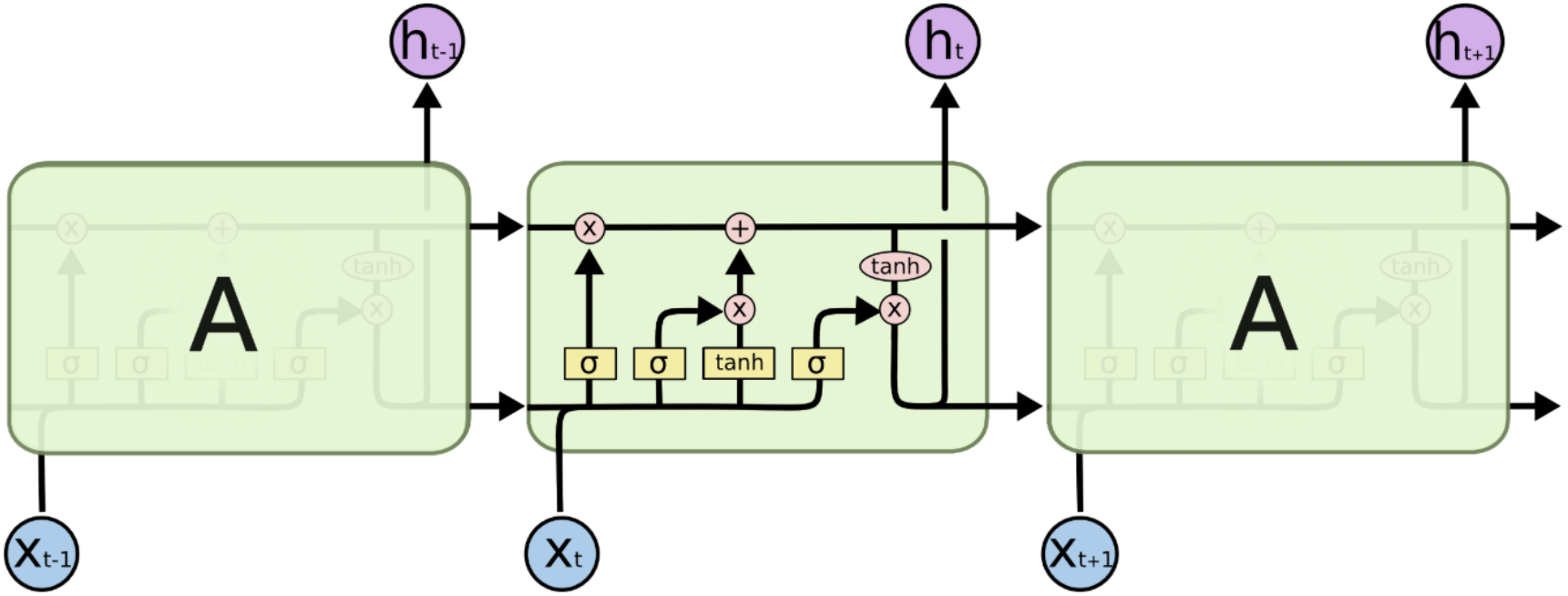
The repeating module in a standard RNN contains a single layer.

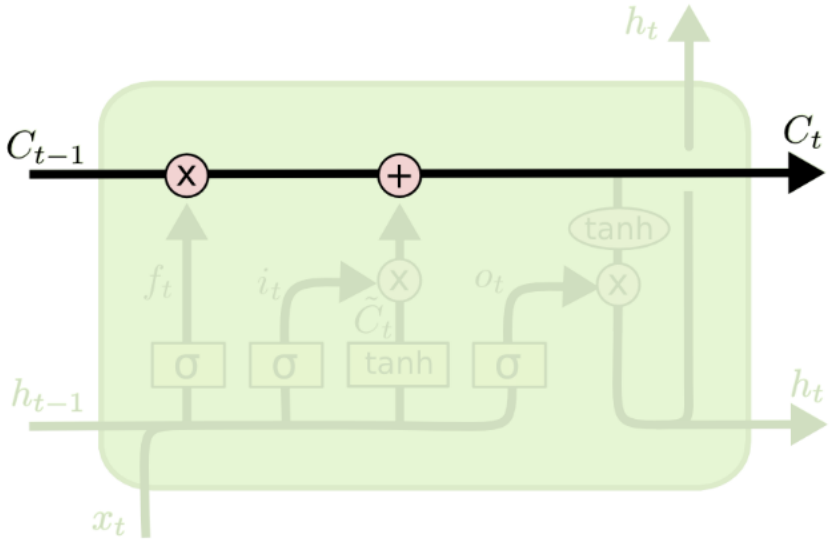
LSTM Model

LSTM: Long Term Short Term Memory



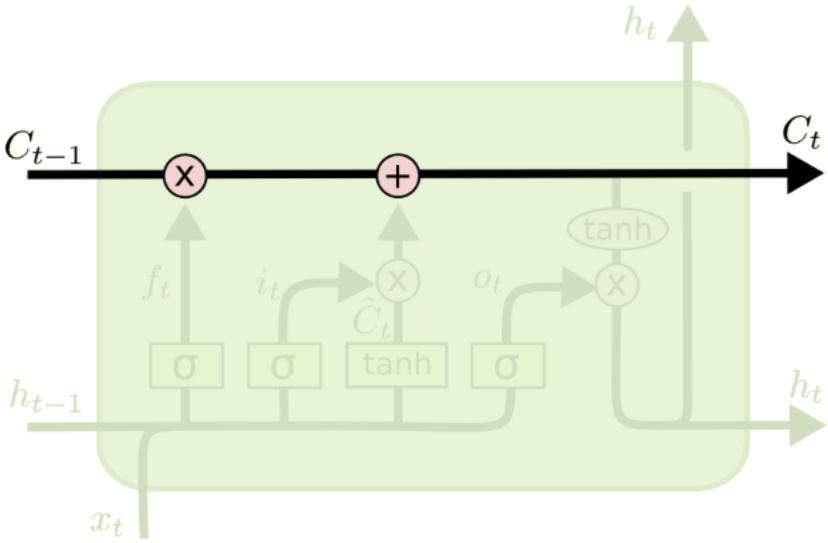
The repeating module in an LSTM contains four interacting layers.





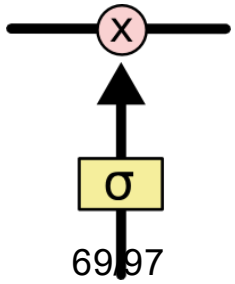
Cell State C:
like a conveyor belt connecting the entire structure

LSTM Cell State



Cell State C:
like a conveyor belt connecting the entire structure

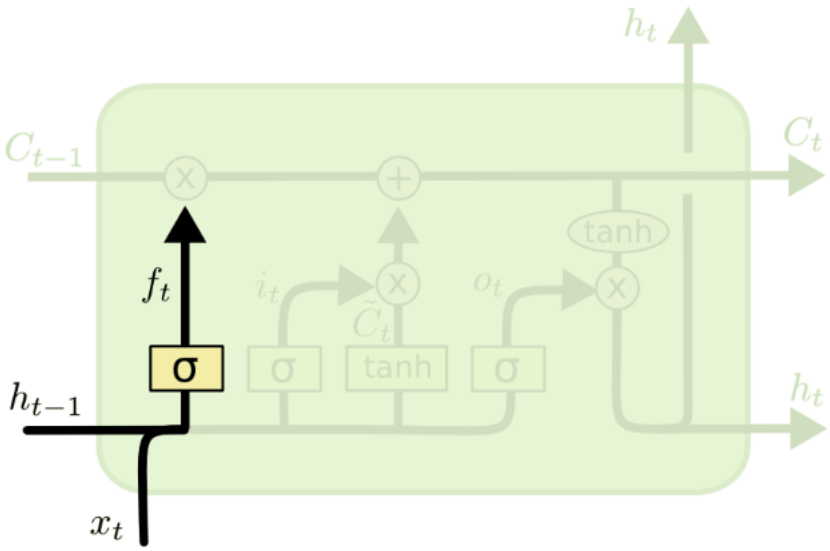
The LSTM does have the ability to remove or add long-term short-term information to the cell state, carefully regulated by structures called gates.



CityU
香港城市大學
City University of Hong Kong
專業 創新 與時俱進
Professional · Creative
For The World

LSTM Walk-through

- 1. decide what information will be thrown away from the old cell state



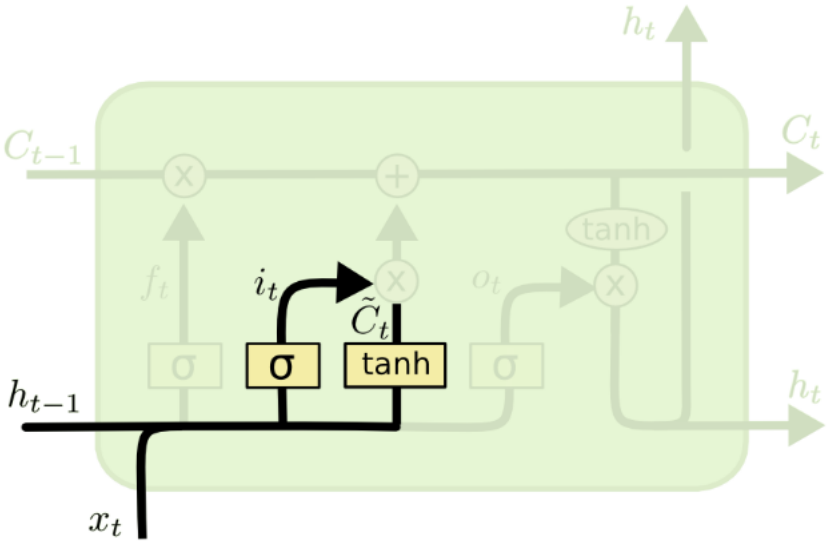
$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

$f_t = 1$: keep this information

$f_t = 0$: throw this information

LSTM Walk-through

- 2. decide what new information will be generated and stored in current state.

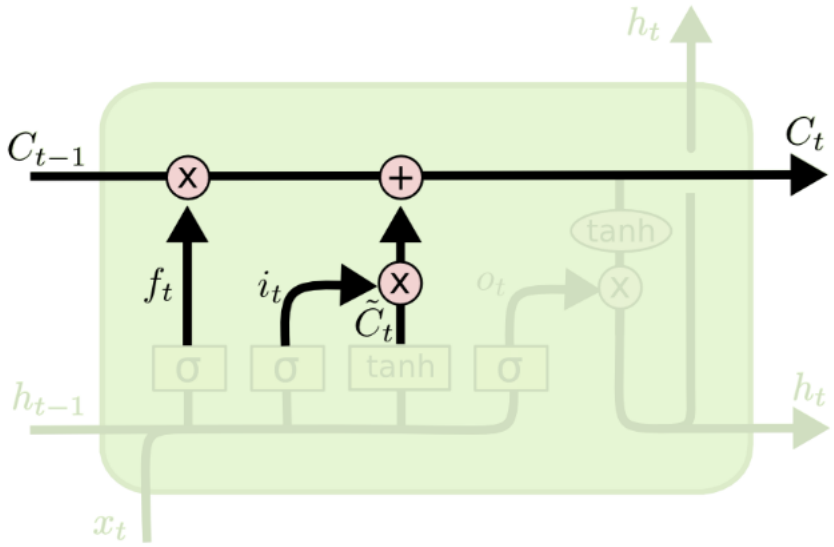


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

LSTM Walk-through

- 3. Update old cell state Ct-1 to new cell state Ct (to be passed to next cell)

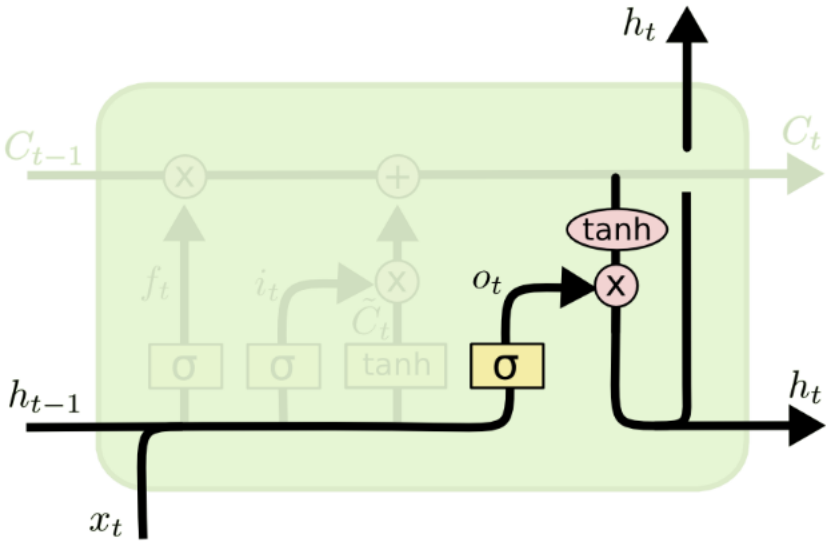


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Linear combination of old state and new created state information

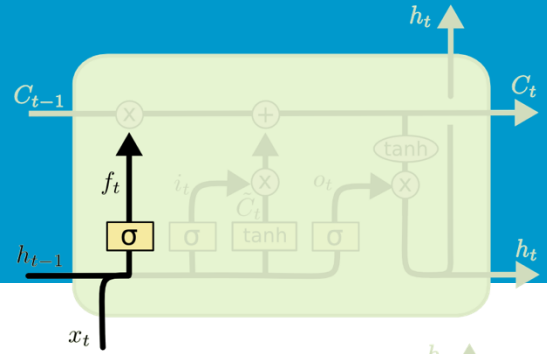
LSTM Walk-through

- 4. Generate output

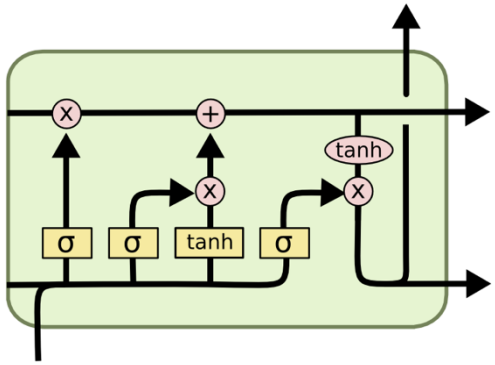


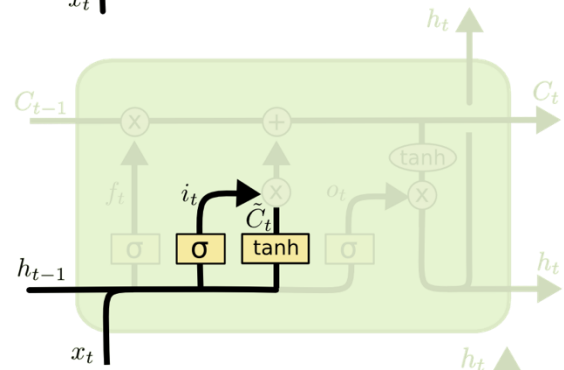
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$



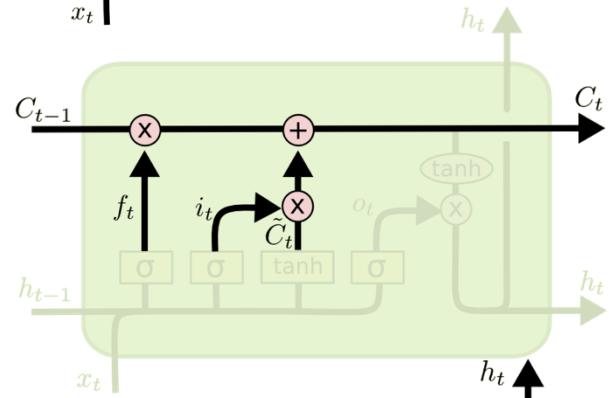
$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$





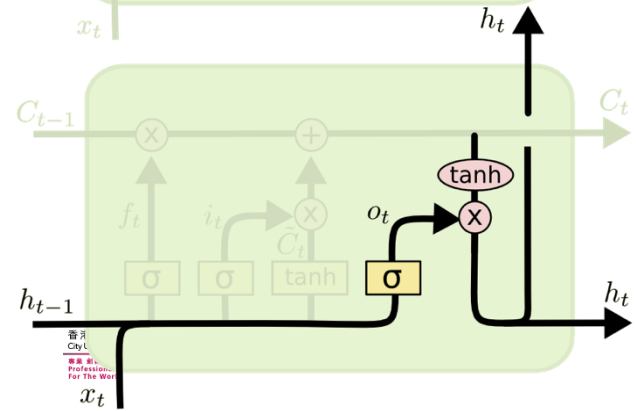
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

i_t decides what component is to be updated.
C'$_t$ provides change contents



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Updating the cell state



$$o_t = \sigma\left(W_o \; [h_{t-1}, x_t] \; + \; b_o\right)$$
$$h_t = o_t * \tanh\left(C_t\right)$$

Decide what part of the cell state to output

IS6400 Business Data Analytics

Dr. LIU Junming

City University of Hong Kong

Lecture 10

- Intro-Deep Learning
  - ANN
  - RNN, LSTM
  - NLP
- Image Mining

Reading Material:

-

What is Text Mining

- Text mining is the set of techniques and methods used for the automatic processing of natural language text data available in reasonably large quantities in the form of computer files, …

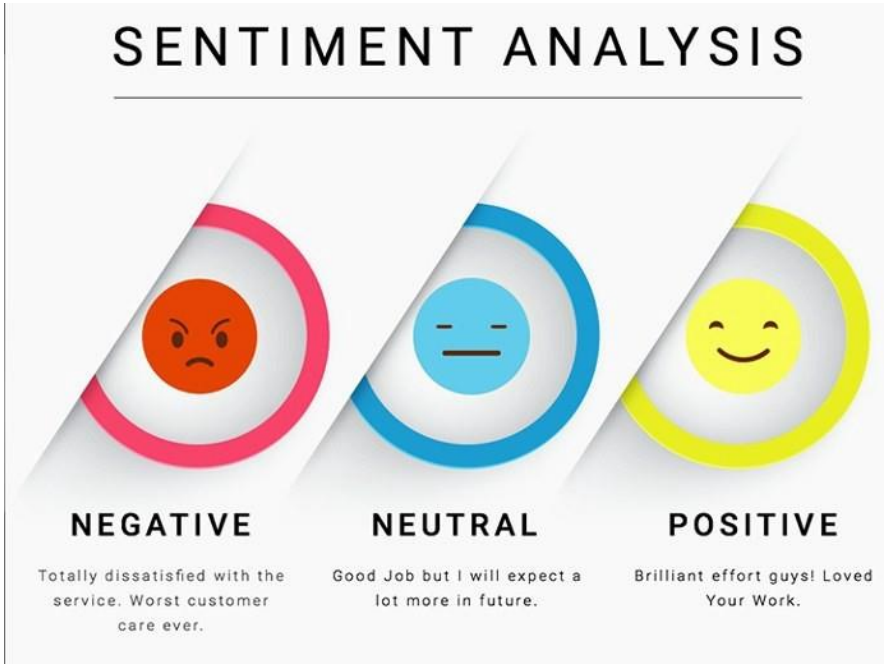
- with the aim of extracting and structuring their contents and themes, for the purposes of rapid (nonliterary) analysis, the discovery of hidden data, or automatic decision making.

Mining useful information from text data

- Same data mining tasks: clustering, classification, association analysis…
- Unstructured data: speech, document, social media articles, product review,…
- Text mining – first, impose structure to the data, then mine the structured data.

Examples of Text Mining 1

Sentiment analysis: a sequence of words -> sentiment



- Sentiment Analysis from Stanford University
- Tweet Sentiment Visualization from North Carolina State University:

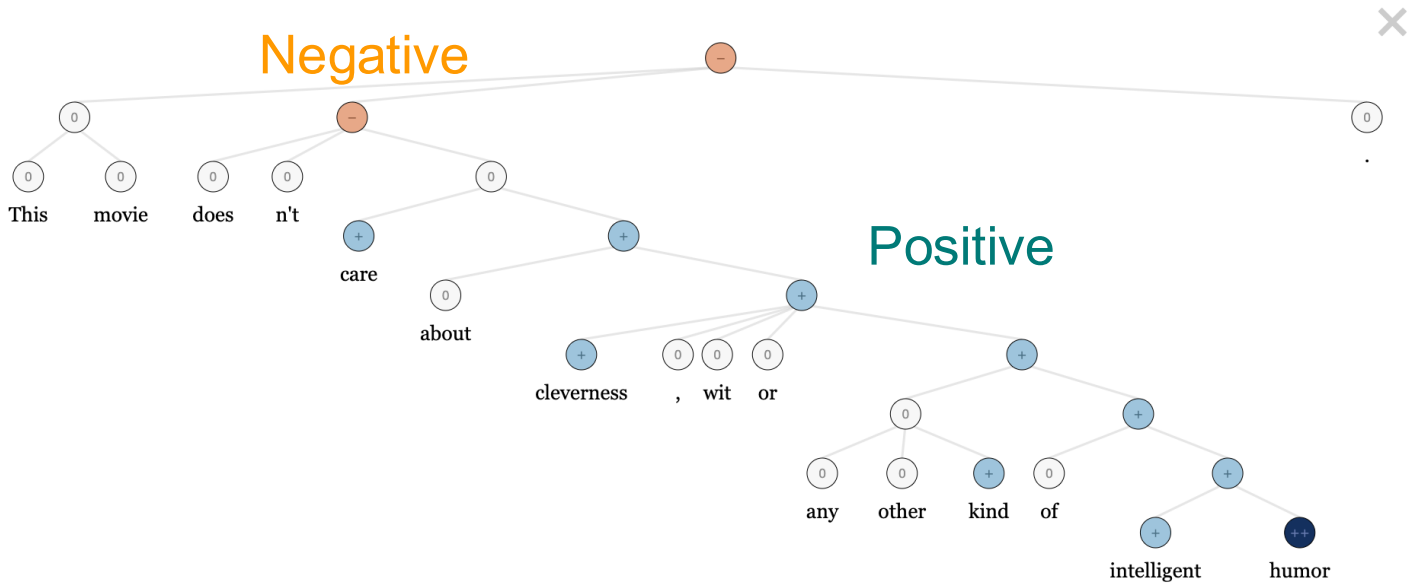
https://www.csc2.ncsu.edu/faculty/healey/tweet_viz/tweet_app/

Examples of Text Mining 1

Sentiment analysis: a sequence of words -> sentiment

- Sentiment Analysis from Stanford University:

INPUT:
This movie doesn't care about cleverness, wit or any other kind of intelligent humor.



Examples of Text Mining 2

- Stock price forecasting
- **HedgeChatter – Social Media Stock Sentiment Analysis**



Examples of Text Mining 3

- Question – Answer (QA) System

Text Data:

COVID-19 is the infectious disease caused by the most recently discovered coronavirus. This new virus and disease were unknown before the outbreak began in Wuhan, China, in December 2019. The most common symptoms of COVID-19 are fever, tiredness, and dry cough. Some patients may have aches and pains, nasal congestion, runny nose, sore throat or diarrhea. These symptoms are usually mild and begin gradually. Some people become infected but don't develop any symptoms and don't feel unwell. Most people (about 80%) recover from the disease without needing special treatment. Around 1 out of every 6 people who gets COVID-19 becomes seriously ill and develops difficulty breathing. Older people, and those with underlying medical problems like high blood pressure, heart problems or diabetes, are more likely to develop serious illness. People with fever, cough and difficulty breathing should seek medical attention.

QA system

Question 1 *

What is COVID-19?

the infectious disease caused by the most recently discovered coronavirus

Question 2

What are symptoms of COVID-19?

fever, tiredness, and dry cough

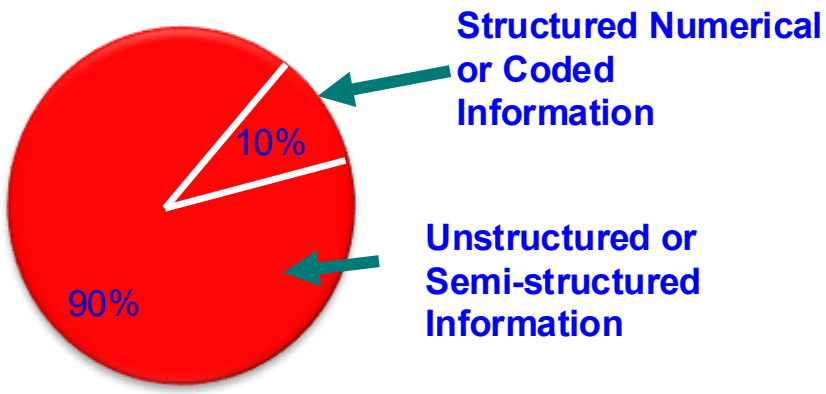
Question 3

How many people recover from COVID?

about 80%

Why Text Analytics/Mining ?

- Approximately **90%** of the world's data is held in unstructured formats (source: Oracle Corporation)
- Information intensive business processes demand that we transcend from simple document retrieval to "knowledge" discovery.



Text Analytics/Mining Methods (1)

- Information Retrieval Style: Information Search
  - *Search technology* (e.g. representation, indexing, ranking)
  - *TF-IDF* (Term Frequency –Inverse Document Frequency)
- Natural Language Processing Style: Information Extraction
  - *Entity extraction:* e.g. person names, organizations, locations, regulation codes)
  - *Relation extraction:* e.g. extracted relationships usually occur between two or more entities of a certain type such as person) and fall into a number of semantic categories such as *married to*, *employed by*, *live in*.
  - *Syntax (e.g. grammars, parsers, parse trees)*, *semantics and ontology (e.g. semantic classes, subcategorization frames)*
  - *Contextual analysis*

Challenges of Text Analytics

- Very high number of possible “dimensions”
  - All possible word and phrase types in the language!!
- Unlike data mining:
  - records (= docs) are not structurally identical
  - records are not statistically independent
- Complex and subtle relationships between concepts in text
  - “AOL merges with Time-Warner”
  - “Time-Warner is bought by AOL”
- Ambiguity and context sensitivity
  - automobile = car = vehicle = Toyota
  - Apple (the company) or apple (the fruit)

How to represent a document?

- A document representation aims to capture what the document is about
- Main approach:
  - Each entry in the table represents a document
  - Attributes represent the frequency in which a term appears in the document

Example: Document-Term frequency table/matrices

Documents

	Terms				
	Camera	Digital	Memory	Print	…
Document 1	3	2	0	1	
Document 2	0	4	0	3	
…	…	…	…	…	

Some Basic Information Retrieval Techniques

- Stemming: cut words to their root.
  - Taxes, taxation (both have the root “tax”)
- Stop words: irrelevant words for removal.
  - E.g. if, and, but, who, what, the, they, their, a, or…
- Weighting of terms (e.g., Term frequency–Inverse document frequency (TF-IDF))
  - *Corpus*: a collect of documents
  - An approach for weighting terms in a document based on the term’s frequency in the document and the document corpus.
  - A term would have a higher weight if it is found to be a good descriptor for a document, i.e., if it appears frequently in the document but is infrequent in the entire corpus.

Term Frequency (TF)

Term Frequency (TF): the frequency of a word appears in a document.

- Let $Count(i,j)$ be the count of the *term i* in the *document j*.
- Let $TF(i,j)$ be the proportion of the count of *term i* in the *document j.*

$$TF(i,j) = \frac{count(i,j)}{Total\ number\ of\ words\ in\ document\ j}$$

Note: the total number of words is used to normalize the direct counting of Count(i,j). If one document has much more words then the other, we need to normalize the Count(i,j) based on document size.

The size of a document, can also be replaced by the total frequency of terms (keywords) in this document.

Inverse Document Frequency (IDF)

- ***Inverse document frequency***: the number of documents which contain a specific word.
- Let *N* be the count of distinct documents in the corpus.
- Let *Count (i)* be the count of documents in the corpus in which the term *i* is present.
- The **inverse document frequency** is a measure of how much information the word provides, i.e., if it's common or rare across all documents.

$$IDF(i) = \log(\frac{N}{count(i)})$$

The Mighty TF-IDF

- Combine TF and IDF to enhance document-term frequency matrices:

$$\text{TF\&IDF}(i,j) = TF(i,j) * IDF(i)$$

attribute

	Terms				
	Camera	Digital	Memory	Print	…
Document 1	3	2	0	1	
Document 2	0	4	0	3	
…	…	…	…	…	

→ Clustering
Classification
Topic models
Regression

What is Natural Language Processing (NLP)?

- In 1950, Alan Turing Published an article titled "Machine and Intelligence" which advertised what is now called the Turing test as a subfield of intelligence.
- Process information contained in natural language text
- How to program computers to process and analyze large amounts of natural language data.
  - speech recognition
  - natural language understanding
  - natural language generation.



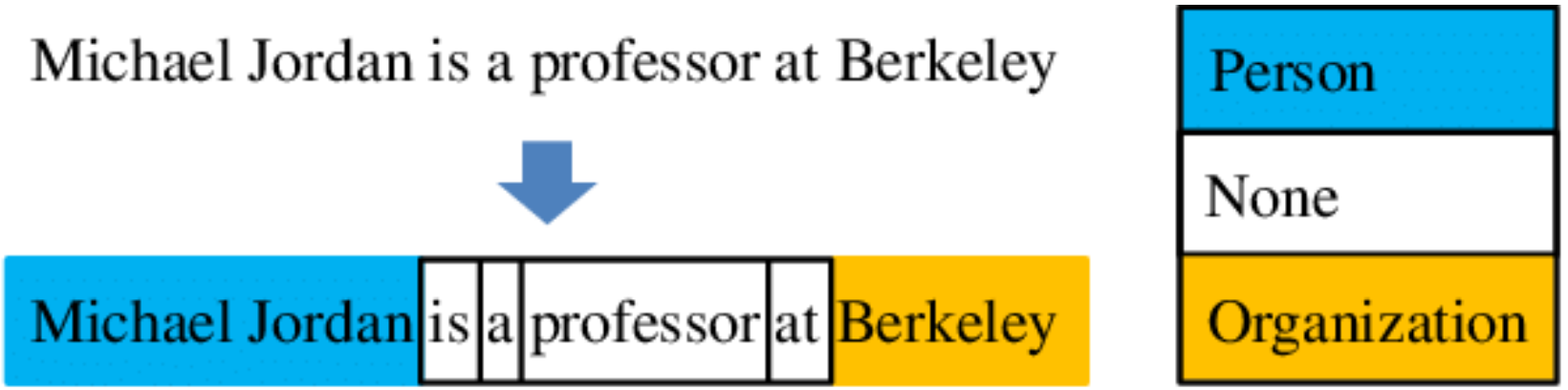
(source: Data Science Central)

Fundamental Tasks

- Word Segmentation
- Part-of-speech (POS) tagging
- Syntactic Analysis
- Semantic Analysis

Word Segmentation

Separate a sentence into segments

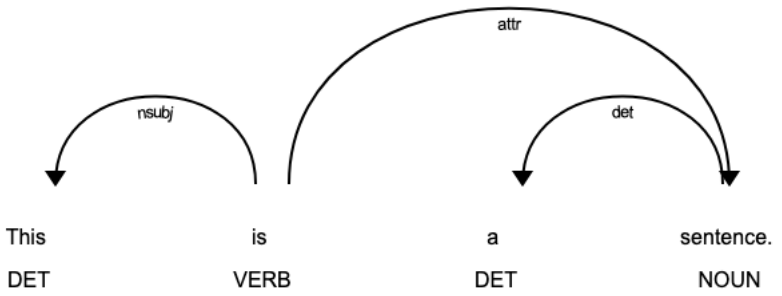


浦东开发与建设 → 浦东 / 开发 / 与 / 建设

Pudong development and construction

POS tagging

- Each word in a sentence can be classified in to classes, such as verbs, adjectives, nouns, etc
- POS Tagging is a process of tagging words in a sentences to part-of-speech, based on:
  - Its definition
  - Its context in the sentence
- Can be achieved by sequential labeling

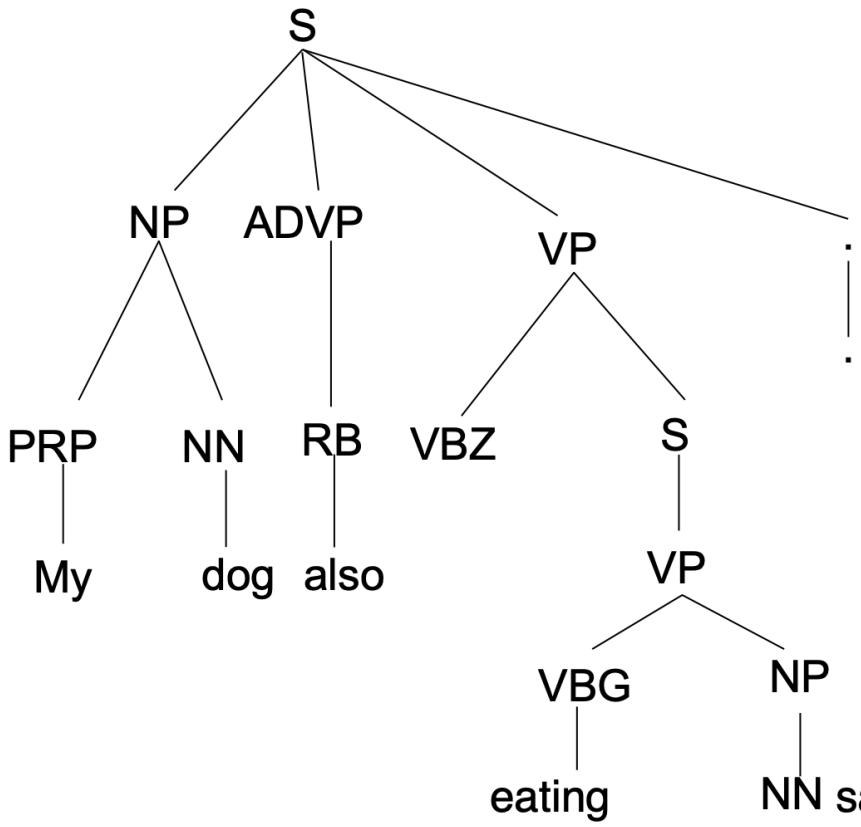


Syntactic Analysis

- The task of recognizing a sentence and assigning a syntactic structure to it.

My dog also likes eating sausage.

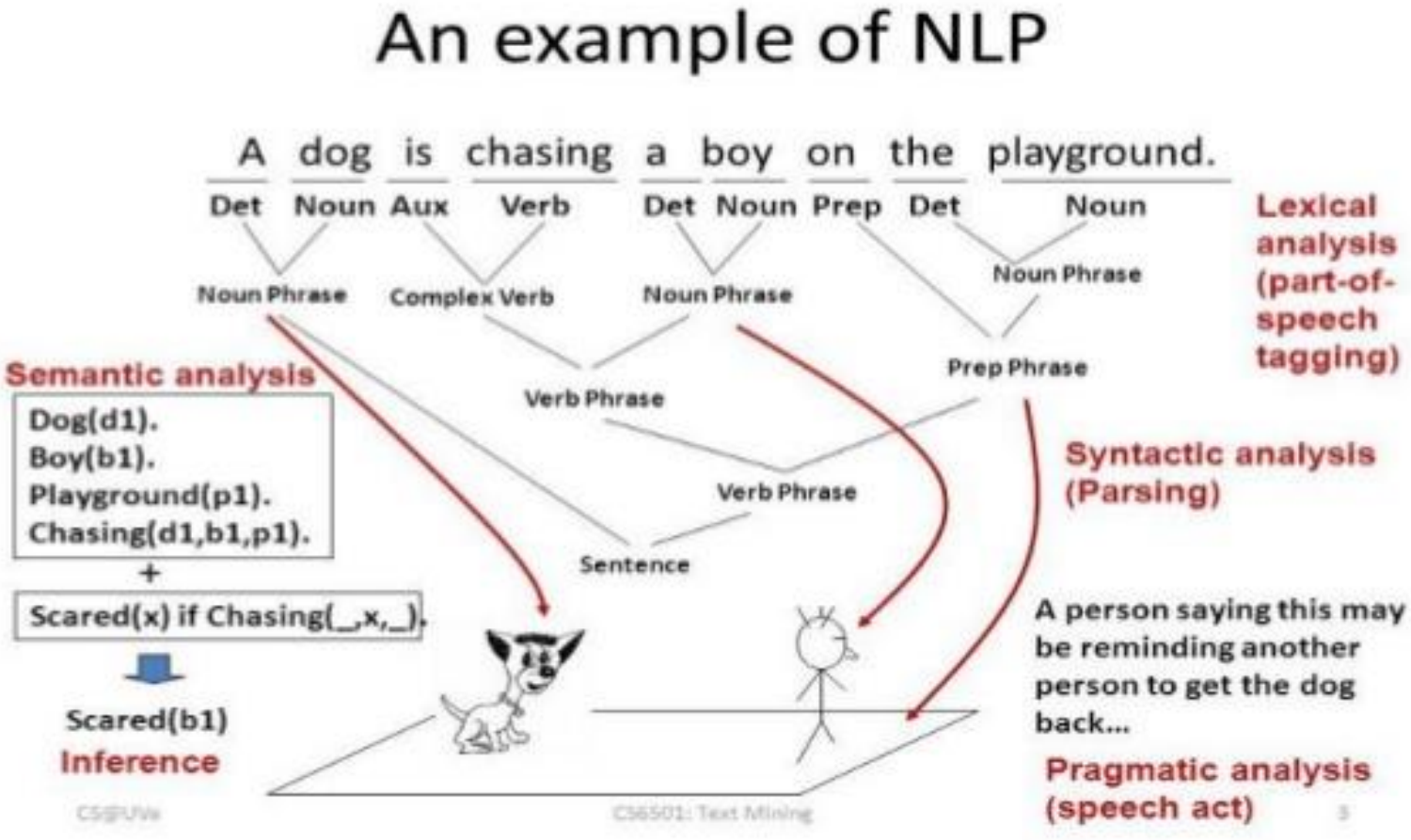
```
(ROOT
  (S
    (NP (PRP$ My) (NN dog))
    (ADVP (RB also))
    (VP (VBZ likes)
      (S
        (VP (VBG eating)
          (NP (NN sausage)))))
    (. .)))
```



Semantic Analysis

- Two levels
  - Lexical semantics − Representing meaning of words (e.g., word bank)
  - Compositional semantics − How words combined to form a larger meaning.

NLP Example


An example of NLP
A dog is chasing a boy on the playground.
Det Noun Aux Verb Det Noun Prep Det Noun
Noun Phrase
Complex Verb
Noun Phrase
Noun Phrase
Prep Phrase
Verb Phrase
Verb Phrase
Sentence
Lexical analysis (part-of-speech tagging)
Syntactic analysis (Parsing)
Semantic analysis
Dog(d1).
Boy(b1).
Playground(p1).
Chasing(d1,b1,p1).
+
Scared(x) if Chasing(_,x,_).
Scared(b1)
Inference
A person saying this may be reminding another person to get the dog back...
Pragmatic analysis (speech act)

Process of NLP

Morphological and Lexical Processing

- Analyze, identify and describe of structure of words.
- Divide a text into paragraphs, words and the sentences.

Syntactic Analysis

- Concerns the proper ordering of words and its effect on meaning.
- Involves analysis of the words in a sentence to describe grammatical structure of the sentence. E.g. "the girl the go to the school". This would definitely be rejected by the English syntactic analyzer.

Semantic Analysis

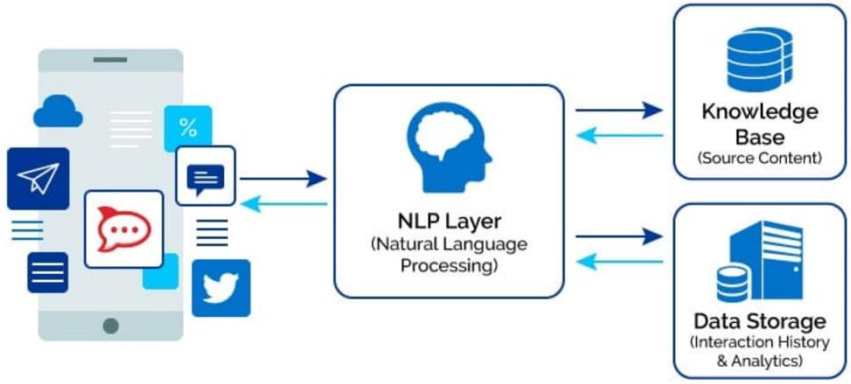
- Concerns the literal meaning of words, phrases, and sentences.
- The structures which are created by the syntactic analyzer are assigned meaning. E.g. "colorless blue idea". This would be rejected by the analyzer.

Context Processing Interpretation

- Sense of context
- The meaning of any single sentence depends on the sentences that precedes it and also invokes the meaning of the sentences that follow it. e.g. the word "it" in the sentence "she wanted it" depends on the prior discourse context.

Some NLP Platforms

- Dialog Flow (ex API.ai)
- Wit.ai
- Luis
- Lex
- Watson Assistant



IS6400 Business Data Analytics

Dr. LIU Junming

City University of Hong Kong

Lecture 10: Image Mining

- Overview
- From Traditional ML to CNN
- Common architectures for image classification

Overview

❖ Image Classification



Humans identify it:

Cat

Not a dog, a cow...

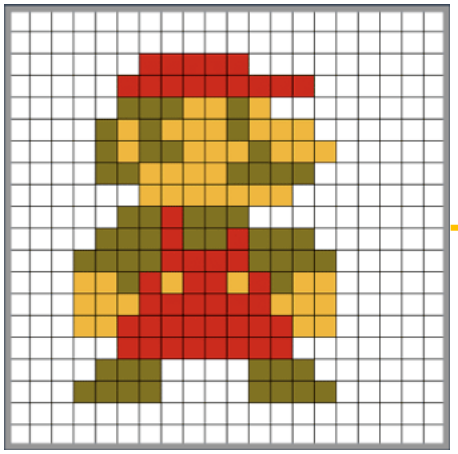
But how does a computer identify it?

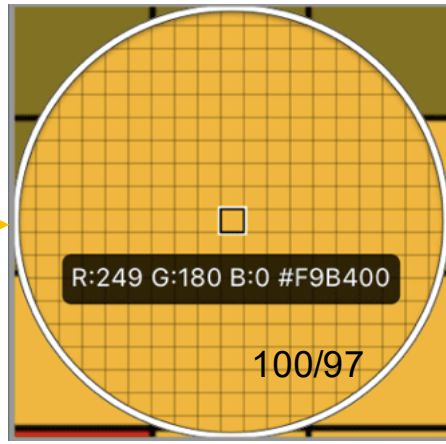
Overview

❖ Computer Vision(CV)

- **Computer vision**: An interdisciplinary scientific field that deals with how computers can gain high-level understanding from digital images and videos.

- Some common languages in CV
  - Pixel
  - RGB
  - Channel
  - Color Space





CityU
香港城市大學
City University of Hong Kong
專業 創新 與時俱進
Professional · Creative
For The World

Overview

❖ Computer Vision(CV)

- A Gray-Scale image



Overview

- **Computer Vision(CV)**
  - A Gray-Scale image

Dive deeper inside the image, it starts getting clearer that it is composed of little squares.

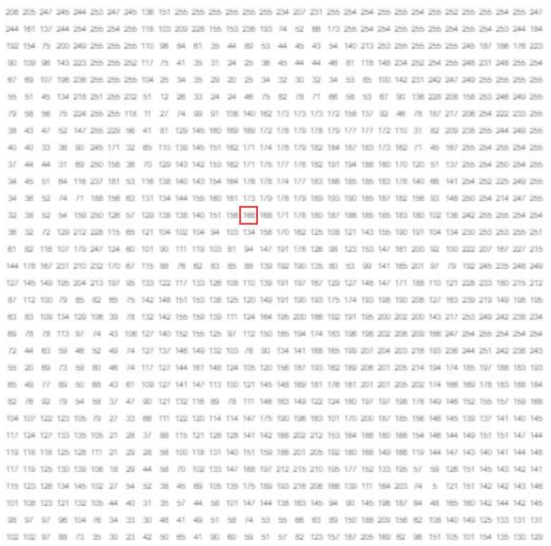


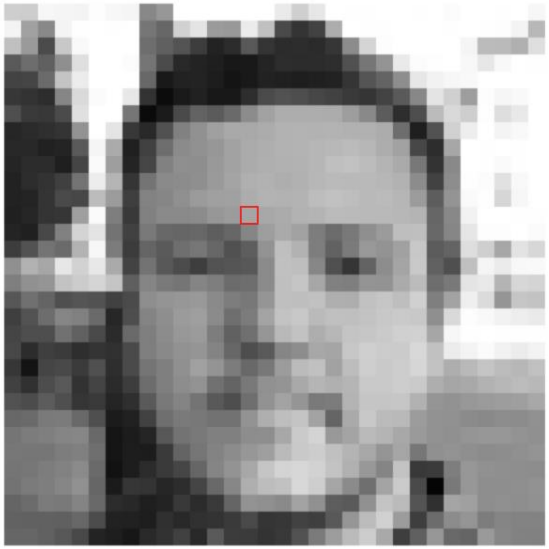
Pixel

Overview

❖ Computer Vision(CV)

- **Pixel:** a number inside a big matrix that represents an image





- Each pixel has a value between 0 and 255, representing the color intensity
- 0 → Completely black.
- 255 → Completely white

Gray-scale images: A matrix of pixels that have values between 0 and 255 representing the black intensity at a certain point.

What's a color image?

Image source: Image Kernels Explained Visually

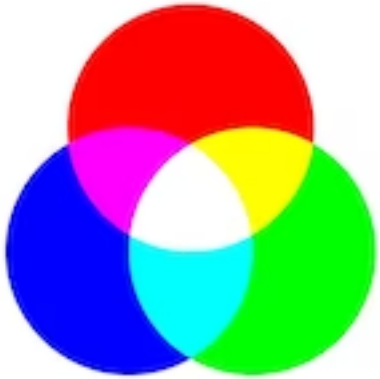
Overview

❖ Computer Vision(CV)

- Color images

Color images are a combination of **3 channels(RGB):**

- Red
- Green
- Blue

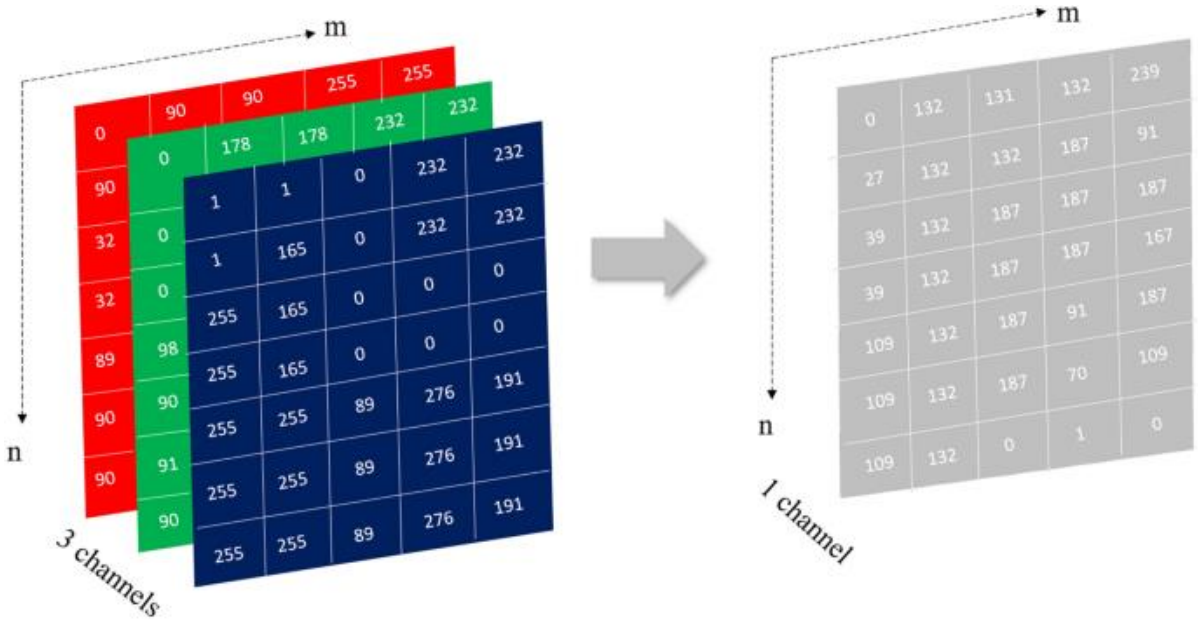


- By combining these three colors, you can produce any imaginable shade.

Overview

❖ Computer Vision(CV)

- Color images



RGB vs Gray-Scale image

Overview

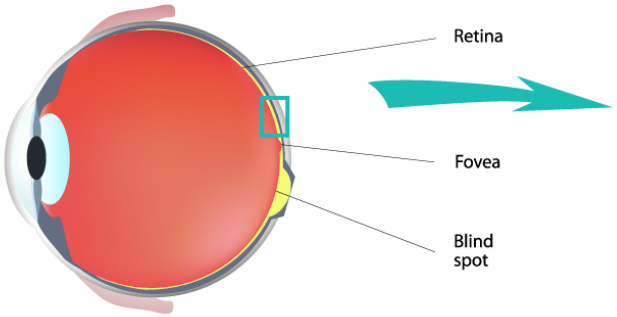
❖ Computer Vision(CV)

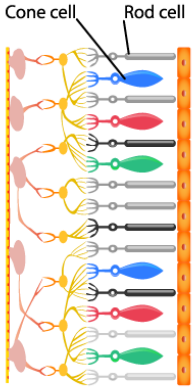
- **WHY Gray-scale images?**
  - The human visual system is more sensitive to brightness information,
    - so many machine learning tasks (such as medical imaging and X-ray recognition) can directly use grayscale images.
  - In computer vision tasks, the image is sometimes converted to grayscale to reduce computational complexity.

Overview

❖ Computer Vision(CV)

- **WHY RGB?**
  - Intuition: Human Vision



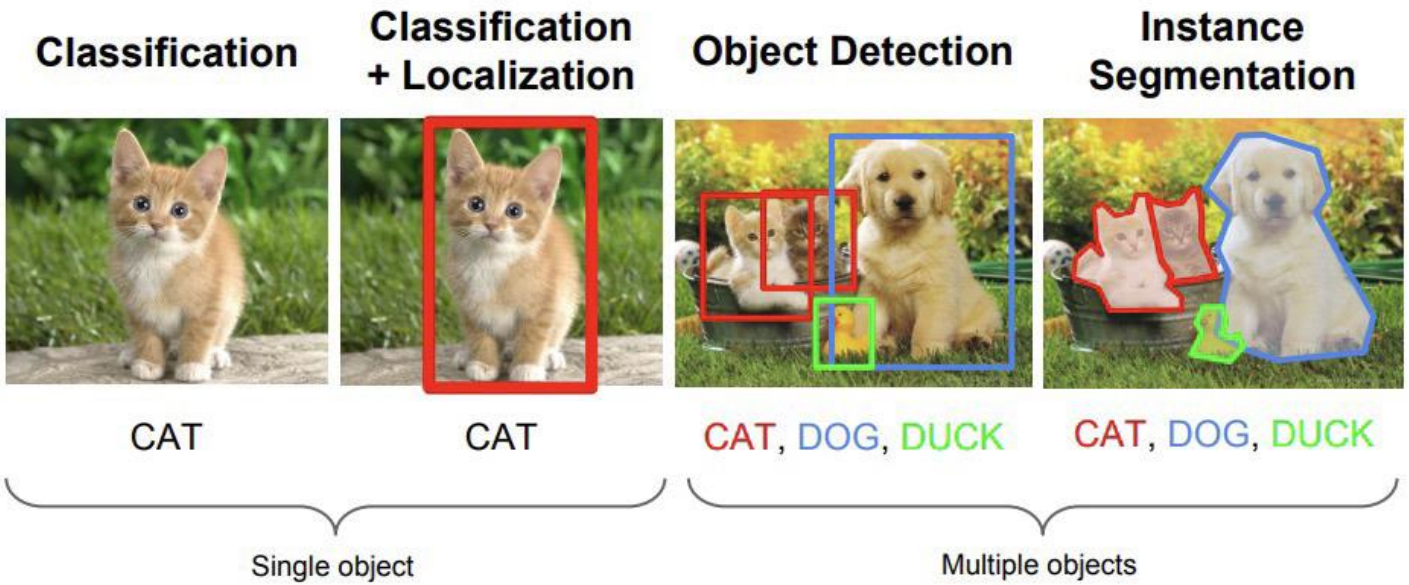


Our perception is based on the **three types of cone cells** we have in our eyes:

- **S (short-wavelength) cones** are most sensitive to light perceived as blue.
- **M (medium-wavelength) cones** are most sensitive to light perceived as green.
- **L (long-wavelength) cones** are most sensitive to light perceived as red.

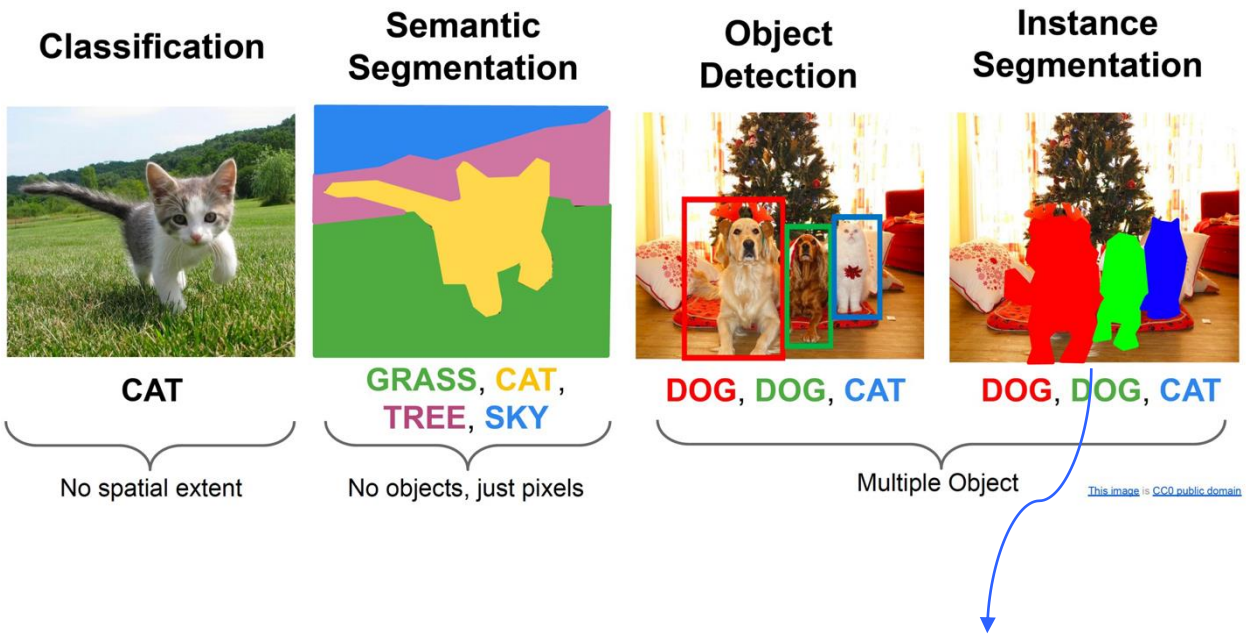
Overview

❖ Computer Vision(CV) Tasks



Overview

❖ Computer Vision(CV) Tasks

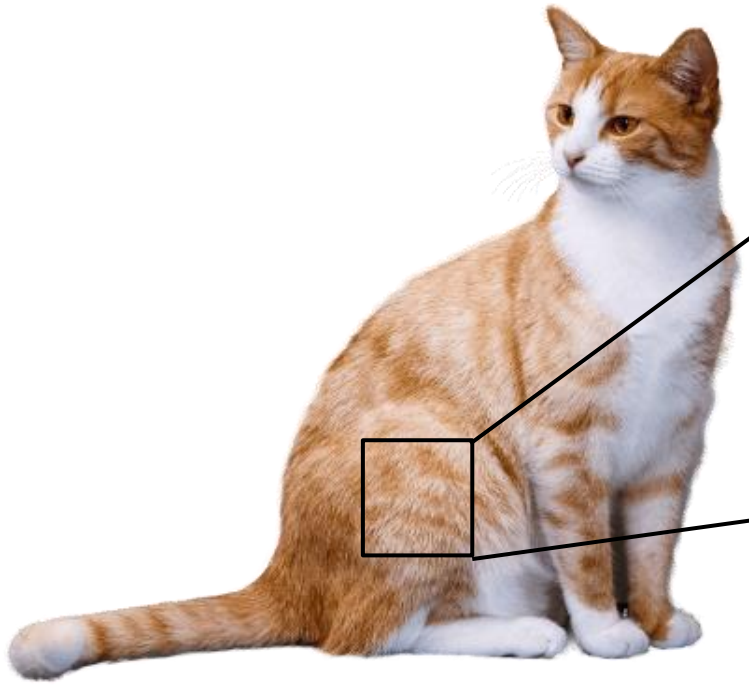


- Semantic Segmentation
  - Predict the class of each pixel

Overview

❖ How does the computer classify images?

- **1. describe it**



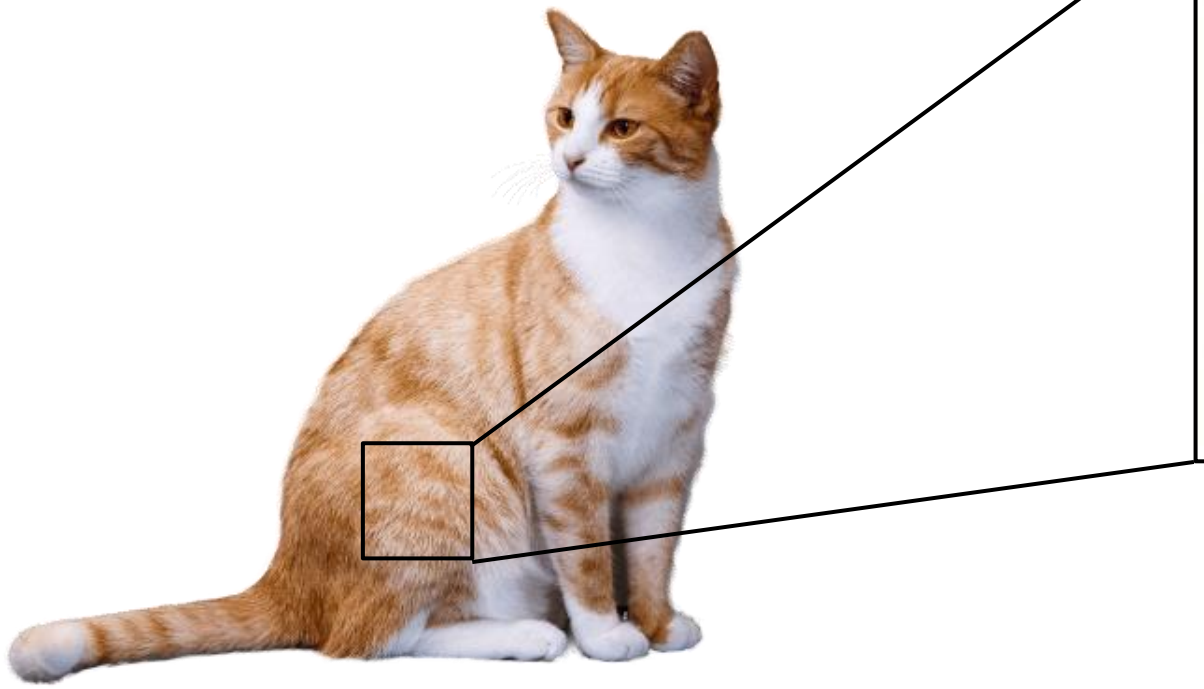
```
[[105 112 108 111 104  99 106  99  96 103 112 119 104  97  93  87]
 [ 91  98 102 106 104  79  98 103  99 105 123 136 110 105  94  85]
 [ 76  85  90 105 128 105  87  96  95  99 115 112 106 103  99  85]
 [ 99  81  81  93 120 131 127 100  95  98 102  99  96  93 101  94]
 [106  91  61  64  69  91  88  85 101 107 109  98  75  84  96  95]
 [114 108  85  55  55  69  64  54  64  87 112 129  98  74  84  91]
 [133 137 147 103  65  81  80  65  52  54  74  84 102  93  85  82]
 [128 137 144 140 109  95  86  70  62  65  63  63  60  73  86 101]
 [125 133 148 137 119 121 117  94  65  79  80  65  54  64  72  98]
 [127 125 131 147 133 127 126 131 111  96  89  75  61  64  72  84]
 [115 114 109 123 150 148 131 118 113 109 100  92  74  65  72  78]
 [ 89  93  90  97 108 147 131 118 113 114 113 109 106  95  77  80]
 [ 63  77  86  81  77  79 102 123 117 115 117 125 125 130 115  87]
 [ 62  65  82  89  78  71  80 101 124 126 119 101 107 114 131 119]
 [ 63  65  75  88  89  71  62  81 120 138 135 105  81  98 110 118]
 [ 87  65  71  87 106  95  69  45  76 130 126 107  92  94 105 112]
 [118  97  82  86 117 123 116  66  41  51  95  93  89  95 102 107]
 [164 146 112  80  82 120 124 104  76  48  45  66  88 101 102 109]
 [157 170 157 120  93  86 114 132 112  97  69  55  70  82  99  94]
 [130 128 134 161 139 100 109 118 121 134 114  87  65  53  69  86]
 [128 112  96 117 150 144 120 115 104 107 102  93  87  81  72  79]
 [123 107  96  86  83 112 153 149 122 109 104  75  80 107 112  99]
 [122 121 102  80  82  86  94 117 145 148 153 102  58  78  92 107]
 [122 164 148 103  71  56  78  83  93 103 119 139 102  61  69  84]]
```

- What the computer sees:
  - An image is a tensor of integers between [0, 255]:
  - e.g. 800 x 600 x 3 (3 channels RGB)

CityU 香港城市大學 City University of Hong Kong 專業 創新 與時俱進 Professional·Creative For The World

Overview

- How does the computer classify images?
  - 2. remember it



```
[[105 112 108 111 104  99 106  99  96 103 112 119 104  97  93  87]
 [ 91  98 102 106 104  79  98 103  99 105 123 136 110 105  94  85]
 [ 76  85  90 105 128 105  87  96  95  99 115 112 106 103  99  85]
 [ 99  81  81  93 120 131 127 100  95  98 102  99  96  93 101  94]
 [106  91  61  64  69  91  88  85 101 107 109  98  75  84  96  95]
 [114 108  85  55  55  69  64  54  64  87 112 129  98  74  84  91]
 [133 137 147 103  65  81  80  65  52  54  74  84 102  93  85  82]
 [128 137 144 140 109  95  86  70  62  65  63  63  60  73  86 101]
 [125 133 148 137 119 121 117  94  65  79  80  65  54  64  72  98]
 [127 125 131 147 133 127 126 131 111  96  89  75  61  64  72  84]
 [115 114 109 123 150 148 131 118 113 109 100  92  74  65  72  78]
 [ 89  93  90  97 108 147 131 118 113 114 113 109 106  95  77  80]
 [ 63  77  86  81  77  79 102 123 117 115 117 125 125 130 115  87]
 [ 62  65  82  89  78  71  80 101 124 126 119 101 107 114 131 119]
 [ 63  65  75  88  89  71  62  81 120 138 135 105  81  98 110 118]
 [ 87  65  71  87 106  95  69  45  76 130 126 107  92  94 105 112]
 [118  97  82  86 117 123 116  66  41  51  95  93  89  95 102 107]
 [164 146 112  80  82 120 124 104  76  48  45  66  88 101 102 109]
 [157 170 157 120  93  86 114 132 112  97  69  55  70  82  99  94]
 [130 128 134 161 139 100 109 118 121 134 114  87  65  53  69  86]
 [128 112  96 117 150 144 120 115 104 107 102  93  87  81  72  79]
 [123 107  96  86  83 112 153 149 122 109 104  75  80 107 112  99]
 [122 121 102  80  82  86  94 117 145 148 153 102  58  78  92 107]
 [122 164 148 103  71  56  78  83  93 103 119 139 102  61  69  84]]
```

Learning

cat

How to learn?

IS6400 Business Data Analytics

Dr. LIU Junming

City University of Hong Kong

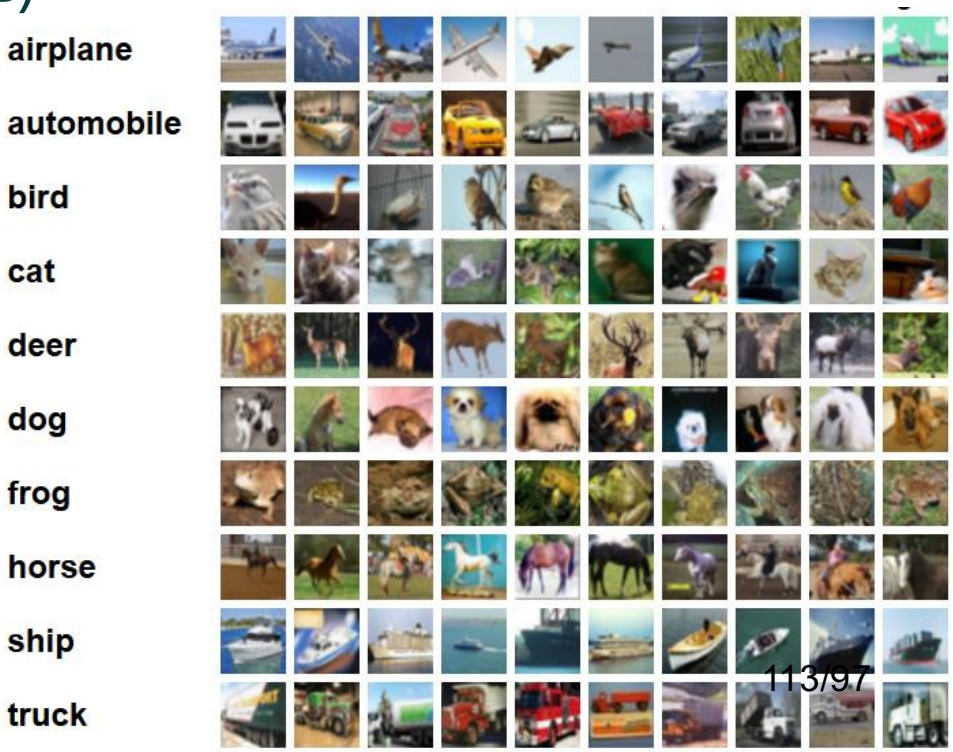
Lecture 10:Image Mining

- Overview
- From Traditional ML to CNN
  - Machine Learning For Image Classification
  - ANN for Image Classification

Overview

❖ Traditional Machine Learning

- **CIFAR-10 Dataset**
  - Image dimensions: 32 × 32 pixels
  - Number of channels: 3 (RGB)



Overview

❖ Traditional Machine Learning

- **Some Traditional Machine Learning methods for Classification**
  - SVM
  - KNN
  - …

ML for Image Classification

❖ SVM for Classification

- **pipeline**
  - **1. load data**

```
# Load the full CIFAR-10 dataset
trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)
```

  - **2. Create DataLoader**

```
# Create DataLoaders for the full dataset
trainloader = torch.utils.data.DataLoader(trainset, batch_size=len(trainset), shuffle=False)
testloader = torch.utils.data.DataLoader(testset, batch_size=len(testset), shuffle=False)
```

  - **3. Convert training and test data to NumPy arrays**

```
train_images, train_labels = convert_to_numpy(trainloader)
test_images, test_labels = convert_to_numpy(testloader)
```

High-dimension!

3072=32*32*3

```
Train data shape: (50000, 3072), Train labels shape: (50000,)
Test data shape: (10000, 3072), Test labels shape: (10000,)
```

ML for Image Classification

❖ PCA+SVM

- **PCA for Dimensionality Reduction**
  - A **dimensionality reduction** technique that transforms high-dimensional data into a lower-dimensional space **while preserving the most important information**.
  - **How PCA works?**
    - Identifying the **principal components** (directions of highest variance).
    - Transforming data into a **new coordinate system** with fewer dimensions.
    - Reducing computational complexity while **retaining critical features**.

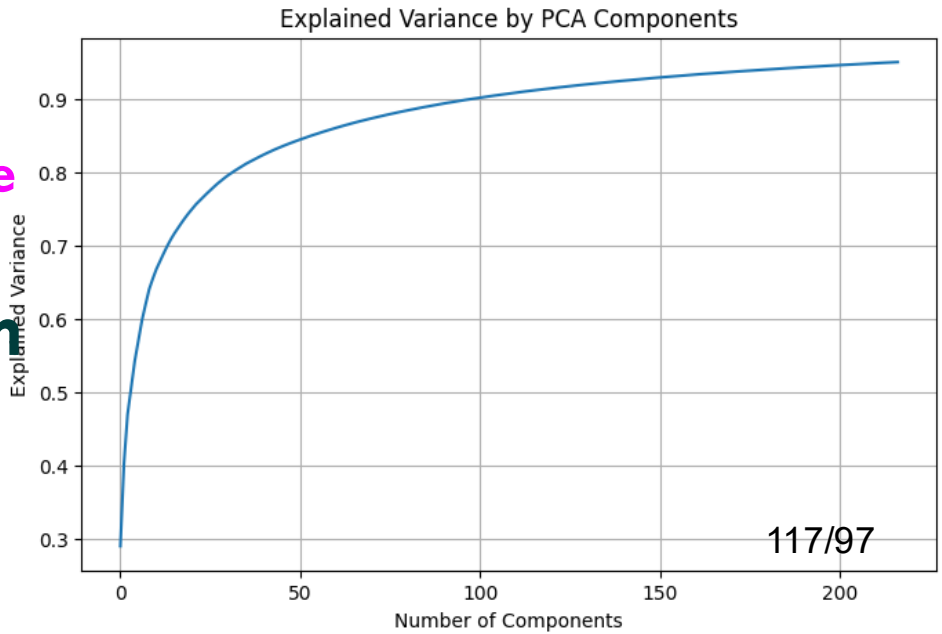
ML for Image Classification

❖ PCA+SVM

- **Pipeline**
  - **4. Apply PCA for dimensionality reduction**

```
pca = PCA(n_components=0.95, whiten=True)  # Retain 95% of variance
train_images_pca = pca.fit_transform(train_images)
test_images_pca = pca.transform(test_images)
```

200 components retain more than 95% of the dataset's variance



Reduce dimensionality from

3000+ ➔ 200+

ML for Image Classification

❖ PCA+SVM

- **Pipeline**
  - **4. Apply PCA for dimensionality reduction**

```
pca = PCA(n_components=0.95, whiten=True)  # Retain 95% of variance
train_images_pca = pca.fit_transform(train_images)
test_images_pca = pca.transform(test_images)
```

  - **5.Train SVM classifier**

```
svm_classifier = SVC(kernel='rbf', gamma='scale', C=1.0)
svm_classifier.fit(train_images_pca, train_labels)
```

ML for Image Classification

❖ PCA+SVM

- **Pipeline**
  - **6. Evaluate the trained classifier**

```python
# Predict on test set
test_predictions = svm_classifier.predict(test_images_pca)

# Compute accuracy
accuracy = accuracy_score(test_labels, test_predictions)

# Compute overall precision, recall, and F1-score (macro & weighted averages)
precision, recall, f1, _ = precision_recall_fscore_support(test_labels, test_predictions, average='weighted')

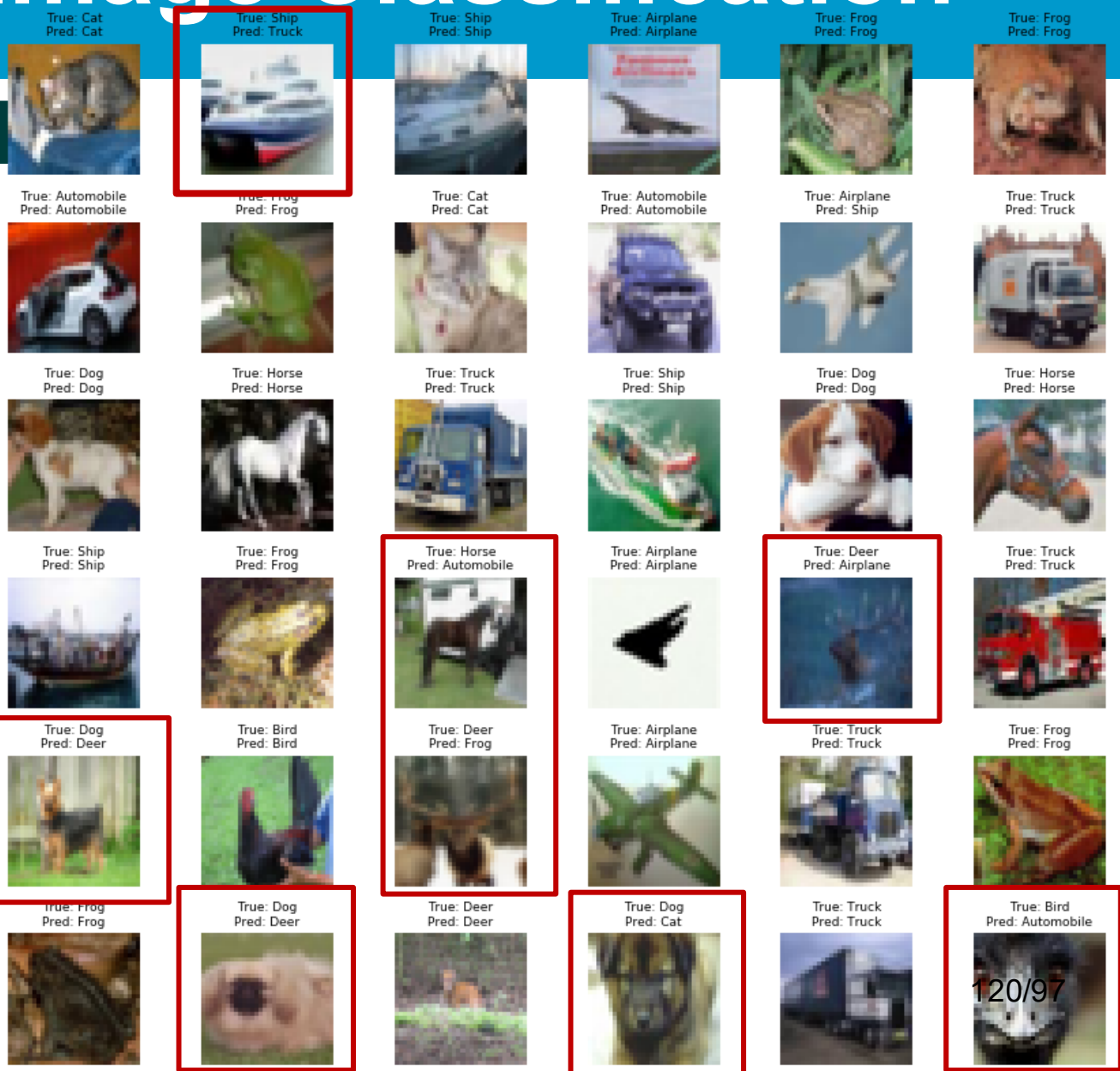
# Print overall metrics
print(f"Overall Accuracy: {accuracy:.4f}")
print(f"Overall Precision: {precision:.4f}")
print(f"Overall Recall: {recall:.4f}")
print(f"Overall F1 Score: {f1:.4f}")
```

Python

```
Overall Accuracy: 0.5553
Overall Precision: 0.5559
Overall Recall: 0.5553
Overall F1 Score: 0.5547
```

ML for Image Classification

- ❖ PCA+SVM
  - SVM Predictions



ML for Image Classification

❖ Summary

- **Problems**
  - Traditional machine learning methods, such as PCA and SVM, rely on handcrafted features and cannot automatically learn features from images.
  - Also, ignores the spatial structures within the image .
    - For example, PCA reduces dimensionality by preserving the most significant variance in pixel values but.
    - Similarly, SVM performs well in high-dimensional spaces but **fails to effectively capture local features (e.g., edges, textures) and spatial information**.

IS6400 Business Data Analytics

Dr. LIU Junming

City University of Hong Kong

Lecture 10:Image Mining

- Overview
- From Traditional ML to CNN
  - Machine Learning For Image Classification
  - ANN for Image Classification
  - CNN for Image Classification

Reading:

- PML Chapter 14.1

ANN for Image Classification

❖ Recall

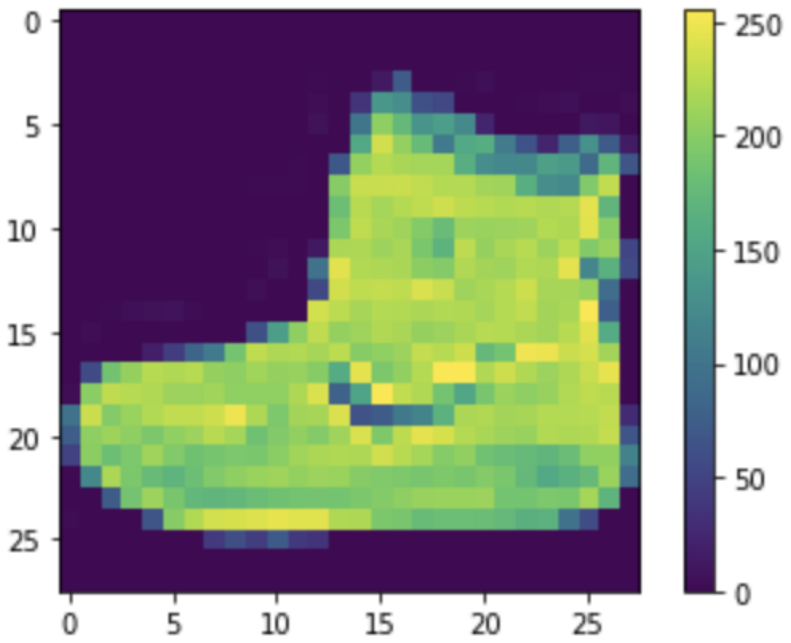
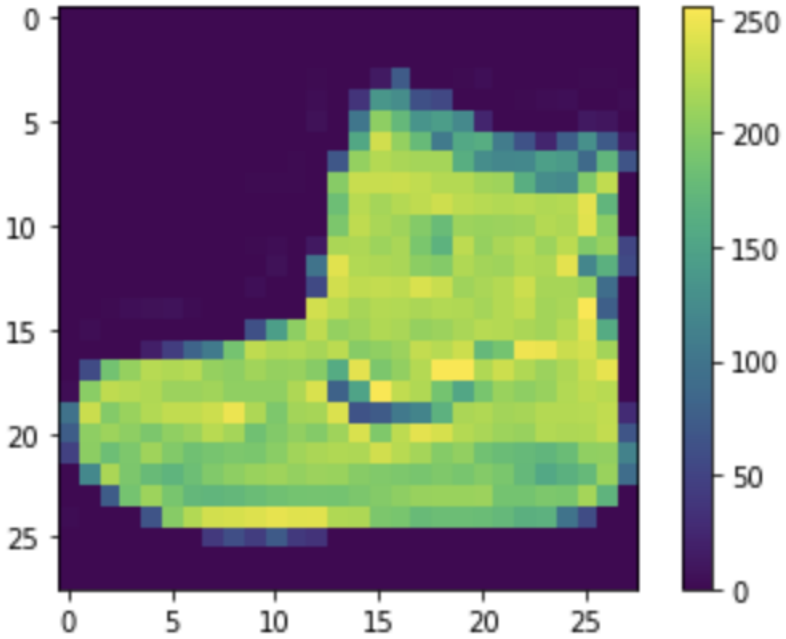


Image classification

Label	Class
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

ANN for Image Classification

❖ Recall



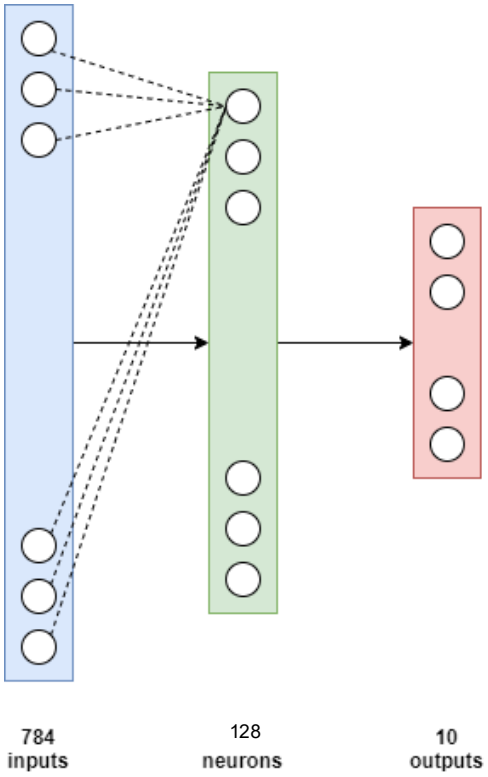
- The object has 28*28 features (pixels)
- We flatten it, i.e., reshape the images from a two-dimensional array (28x28 pixels) into a one-dimensional array of size 28 × 28 = 784
- Then put it into ANN model

ANN for Image Classification

❖ Recall

- **Step 0： define model**

- Input(Feature vector): 28*28 (pixels)=784



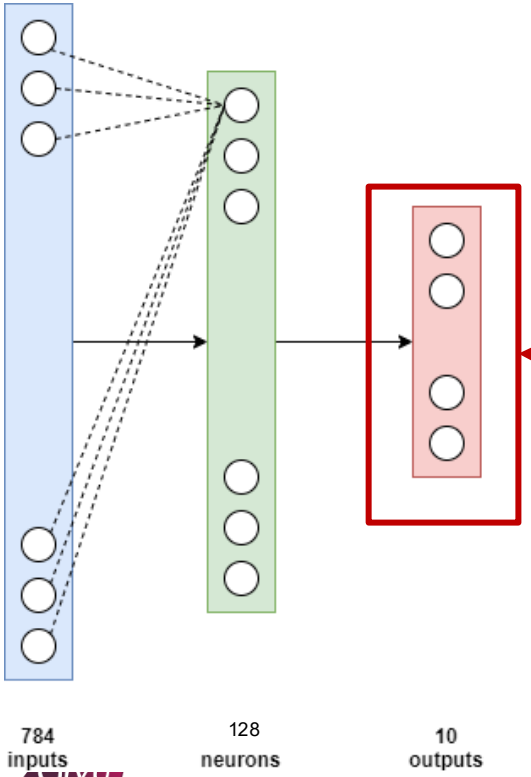
- Output: the prob of each class

Label	Class
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

ANN for Image Classification

❖ Recall

- **Step 1： FeedForward**




```
epochs = 10

for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:
        # Zero the parameter gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)
        loss = loss_function(outputs, labels)

        # Backward pass and optimize
        loss.backward()
        optimizer.step()

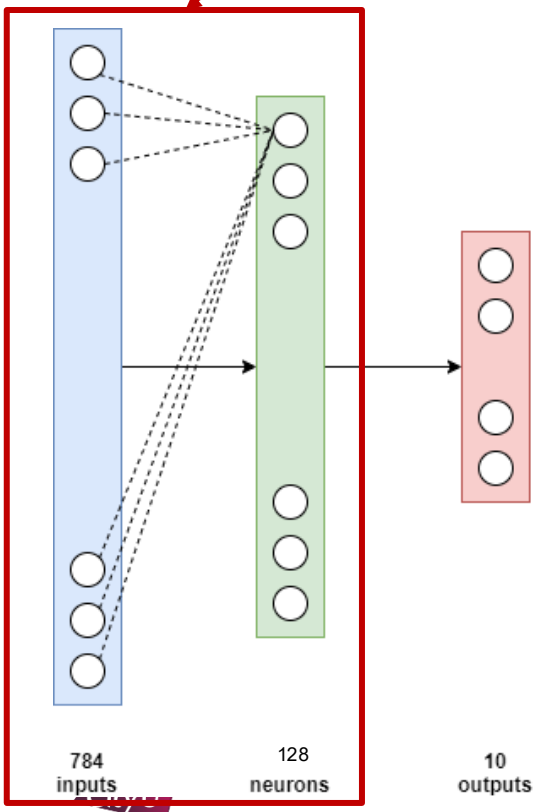
        # Track loss and accuracy
        running_loss += loss.item()
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
```

```
# Define the neural network lay
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28 * 28, 128),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(128, 10)
)
```

ANN for Image Classification

❖ Recall

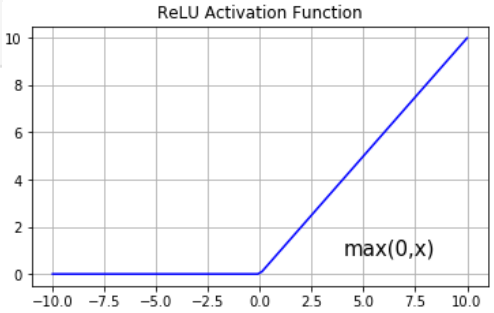
- **Step 1： FeedForward**




```
# Define the neural network lay
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28 * 28, 128),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(128, 10)
)
```

First layer and second layer

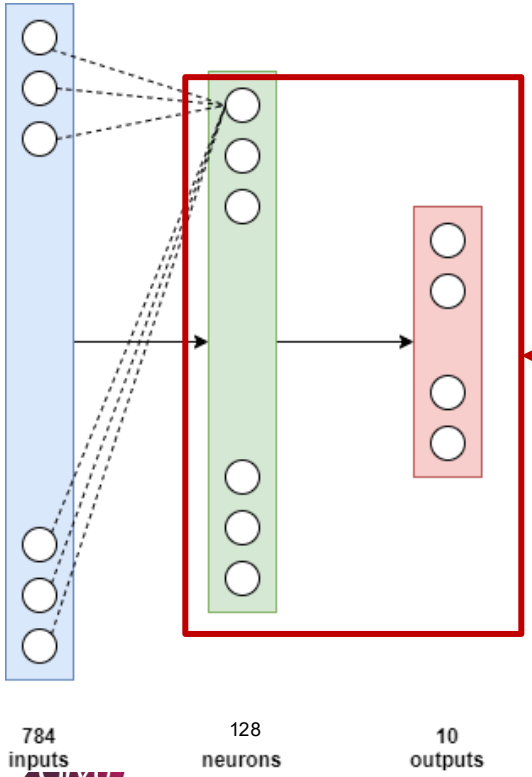
Activate function of second layer neurons.



ANN for Image Classification

❖ Recall

- **Step 1： FeedForward**



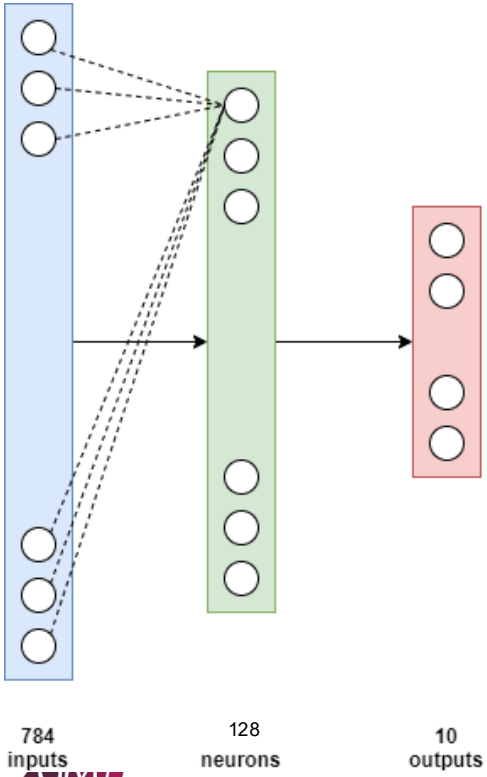

```
# Define the neural network lay
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28 * 28, 128),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(128, 10)
)
```

- The second and last layer

ANN for Image Classification

❖ Recall

- **Step 2 : Calculate loss**




```
epochs = 10

for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:
        # Zero the parameter gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)
        loss = loss_function(outputs, labels)
```

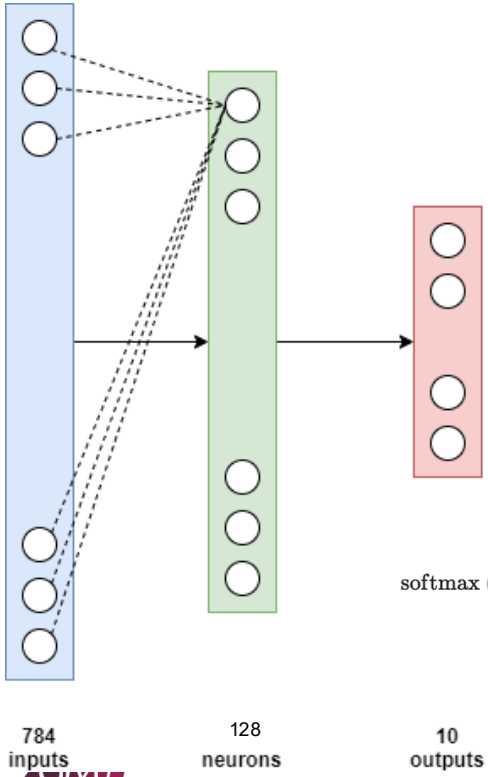
```
# Define the loss function and optimizer
loss_function = nn.CrossEntropyLoss()
```

ANN for Image Classification

❖ Recall

- **Step 2 : Calculate loss**

```
# Define the loss function and optimizer
loss_function = nn.CrossEntropyLoss()
```



- Recall: CrossEntropyLoss

The true label for sample i is class j, 0 or 1

$$CE\ Loss = -\frac{1}{n}\sum_{i=1}^{N}\sum_{j=1}^{M} y_{ij} \cdot log(p_{ij})$$

The predictive prob of sample i being j, ranges from 0 to 1

- Suppose the output of the model $\hat{\mathbf{y}} = [5, 1, 1, 1, 1, 1, 1, 1, 1, 1]$
  - Firstly, use Softmax to calculate the prob. of each class

$$\text{softmax}(\mathbf{x}) = \frac{e^{x_i}}{\sum_j e^{x_j}} \qquad \text{softmax}(\hat{\mathbf{y}}) = [0.8585, 0.0157, 0.0157, 0.0157, 0.0157, 0.0157, 0.0157, 0.0157, 0.0157, 0.0157]$$

  - If the ground truth is label 0, the loss would be

$$-\log(0.8585) = 0.153\ldots$$

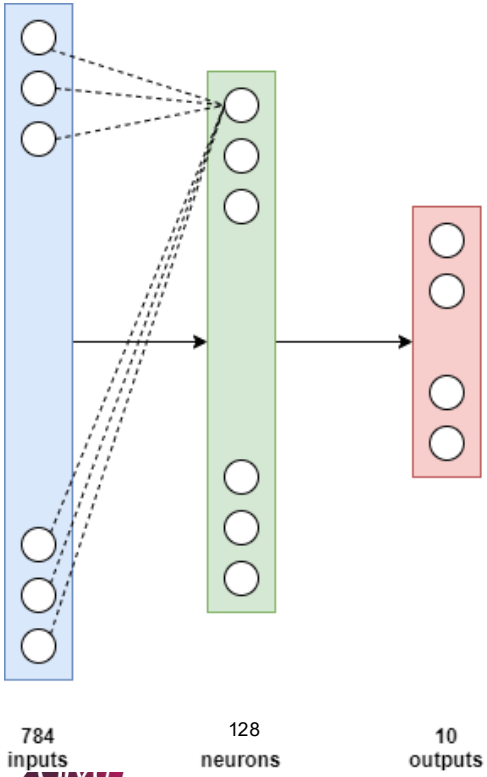
  - If the ground truth is label 5, the loss would be

$$-\log(0.0157) = 4.154\ldots$$

ANN for Image Classification

❖ Recall

- **Step 3: Loss backward**




```
epochs = 10

for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:
        # Zero the parameter gradients
        optimizer.zero_grad()

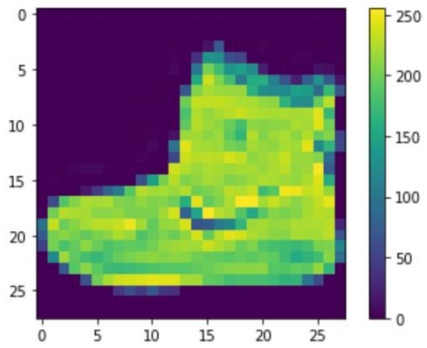
        # Forward pass
        outputs = model(images)
        loss = loss_function(outputs, labels)

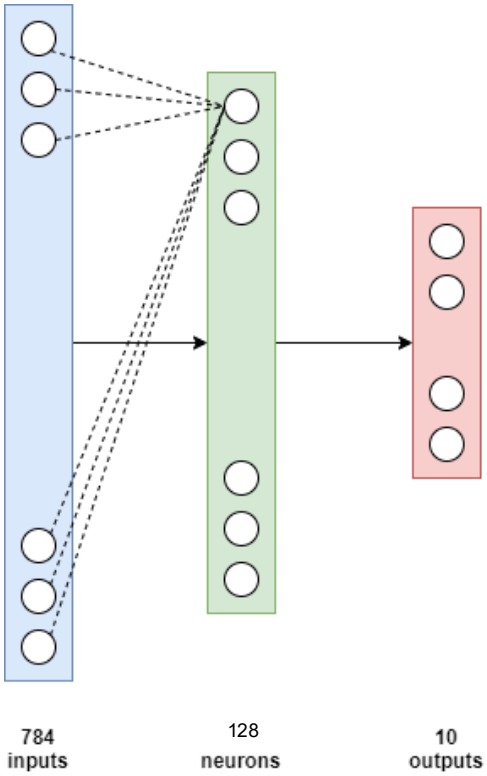
        # Backward pass and optimize
        loss.backward()
        optimizer.step()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

ANN for Image Classification

❖ Summary





Label	Class
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

```
epochs = 10

for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:
        # Zero the parameter gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)
        loss = loss_function(outputs, labels)

        # Backward pass and optimize
        loss.backward()
        optimizer.step()
```

ANN for Image Classification

❖ Problems-1

- ANN does not work well if the input is a variable-sized image

```
model = nn.Sequential(
    nn.Flatten(),                    # Flatten the input
    nn.Linear(28 * 28, 128),         # Fully connected layer with 128 neurons
    nn.ReLU(),                       # ReLU activation
    nn.Dropout(0.2),                 # Dropout with a rate of 0.2
    nn.Linear(128, 10)               # Fully connected layer with 10 neurons (output layer)
)
```

Fixed number of input neurons.

- We describe a picture (also, the input of NN model)

$$\boldsymbol{x} \in \mathbb{R}^{WHC}$$

Different pictures may have different size

  - W: the width,
  - H is the height, and
  - C is the number of input channels

source: PML Chapter 14.1

ANN for Image Classification

❖ Problems-2

- even if the input was fixed size, the number of parameters needed would be prohibitive for reasonably sized images
  - We describe a picture (also, the input of NN model)

$$\boldsymbol{x} \in \mathbb{R}^{WHC}$$

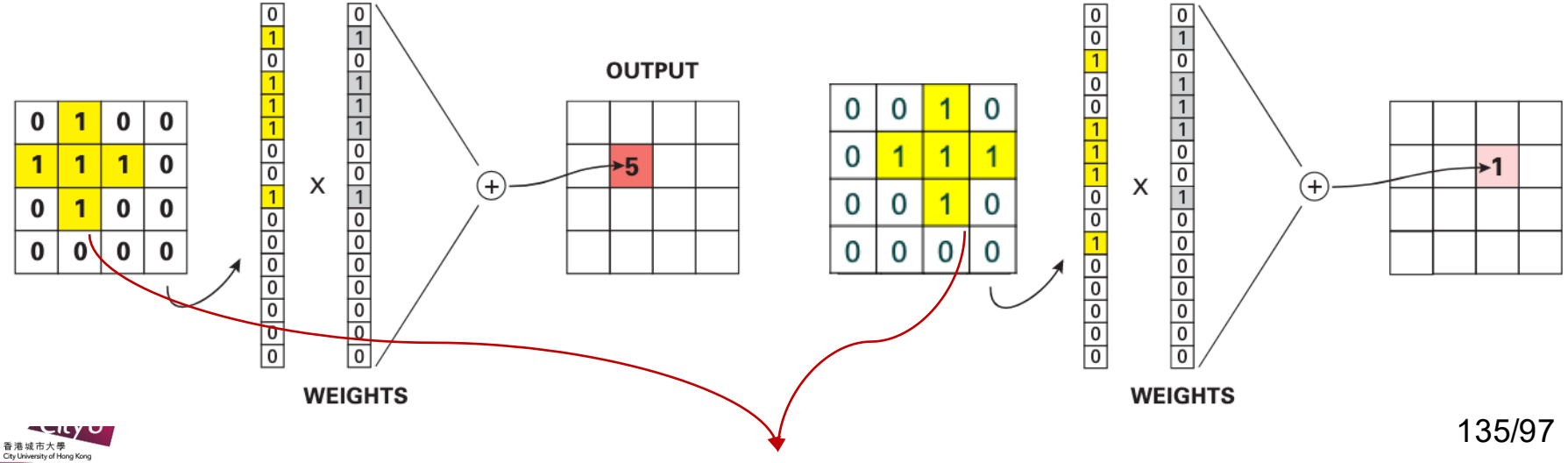
  - For classification, if # of class =D, then # of parameters=W*H*C*D

source: PML Chapter 14.1

ANN for Image Classification

❖ Problems-3

- ANN may not exhibit translation invariance
  - A pattern that occurs in one location may not be recognized when it occurs in a different location
  - because the weights are not shared across locations



The same patterns occur in different locations

source: PML Chapter 14.1

ANN for Image Classification

- The better Architecture ?

Convolutional Neural Network

source: PML Chapter 14.1

CNN Architecture: Summary

- Convolution + ReLU + Max Pooling →Flattening → Fully Connected Layers
- Layers with parameters: CONV & Fully Connected

